Instituto Tecnológico de Costa Rica

Curso: Taller de Programación | IC-1803

Profesor: William Mata Rodríguez

Programa 1:

Programa 1 · Lista Digital de Contactos

II Semestre

Estudiante:

Amy Jimena Morataya Sandoval

Carné:

2026801281

Fecha de entrega:

01 de noviembre del 2026

Contenido

Enunciado del Proyecto 3

1.1 Objetivos del Proyecto 3

1.2 Definición del Problema 4

1.3 Requerimientos Funcionales 4

Temas Investigados 6

2.1 Herramientas de Depuración (Debugger) 6

2.2 Depurador del IDE Utilizado y su Aplicación en el Proyecto 9

2.3 Importancia de la Documentación de Software 11

2.4 Expresiones Regulares (Regex) y Validación de Correo Electrónico 12

2.5 Emisión de información en archivos PDF 15

2.6 Herramientas de control de versiones y colaboración 18

2.7 Registro y Uso Regulado de Inteligencia Artificial (IA) 22

2.8 Otros aspectos investigados 44

Conclusiones y Trabajo Futuro 47

3.1 Problemas Encontrados y Soluciones Aplicadas 47

3.2 Aprendizajes Obtenidos 49

3.3 Aspectos a Mejorar / Hacer Diferente en Próximos Proyectos 49

Estadística de Tiempos 50

Lista de Revisión y Análisis de Resultados 54

Funcionalidades y Partes Adicionales Desarrolladas 116

Referencias 118

Enunciado del Proyecto

1.1 Objetivos del Proyecto

- Aplicar el ciclo completo de la metodología general de desarrollo de programas: entender el problema, diseñar el algoritmo, codificarlo, probar y evaluar el programa.
- Aplicar y reforzar el lenguaje Python 3: uso de funciones, programación estructurada (secuencia, condicional y ciclo), iteración y estructuras de datos (listas, tuplas, strings y rangos), con uso intensivo de listas y tuplas.
- Aplicar buenas prácticas de programación: documentación interna y externa, nombres significativos, reutilización de funciones, programación modular («divide y vencerás») y un estilo de programación consistente.
- Validar los datos de entrada, asegurando que cada dato cumpla con las características especificadas antes de ser procesado.
- Mantener la integridad referencial entre la lista de contactos, las áreas y los grupos de contactos, de modo que las relaciones entre los datos sean válidas y perduren en el tiempo.
- Implementar operaciones de mantenimiento de datos (CRUD: crear, consultar, modificar y eliminar) sobre listas y tuplas.
- Fomentar la habilidad de investigación sobre depuración, documentación, expresiones regulares, emisión de PDF y control de versiones, entre otros aspectos necesarios para la solución.
- Emitir información en archivos PDF a partir de datos filtrados.

1.2 Definición del Problema

- Se requiere un programa de consola (interfaz de usuario tipo texto, TUI) que permita registrar y consultar información de contactos. Cada contacto se identifica de forma única por su teléfono, que tiene dos partes: el área (código que indica la ubicación física a la que pertenece el número, por ejemplo 506 para Costa Rica) y el número de teléfono. Ningún teléfono se repite en dos contactos, de la misma forma en que el carné identifica a un estudiante.
- Además de los contactos, el programa administra las áreas disponibles, una configuración por omisión (área y tipo de teléfono) y grupos de contactos que agrupan teléfonos ya registrados. Con esos datos permite emitir un listado filtrado en formato PDF. La interacción se hace exclusivamente por teclado, con un menú principal desarrollado con la estructura match-case.
- Entendí que el programa debe permitir registrar y administrar contactos utilizando diferentes opciones de un menú. Para desarrollarlo, dividí el trabajo en funcionalidades, comenzando con Registrar áreas y Configuración de contactos. Esto me permitió trabajar cada parte por separado, realizar pruebas y corregir errores antes de continuar con la siguiente.

1.3 Requerimientos Funcionales

Opción del menú	Qué debe hacer	Validaciones y reglas
1. Registrar áreas	Mantenimiento (agregar, consultar, modificar, eliminar) de la lista areas, donde cada elemento es una tupla (área, nombre del área).	Área: entero de 1 a 999, único. Nombre: 1 a 40 caracteres. «C» cancela en el primer dato. No se elimina un área con contactos. Confirmación al eliminar.
2. Configuración	Establecer el área por omisión y el tipo de teléfono por omisión, que se usan al	El área debe existir en areas. El tipo debe existir en la tupla constante TIPOS_TELEFONO (M, C,

	registrar contactos cuando el usuario no digita esos datos.	T, O). Cancelar conserva los valores anteriores.
3. Registrar contactos	Mantenimiento de la lista contactos. Cada contacto: teléfono, área, tipo de teléfono, nombre, correo, dirección, fecha de nacimiento, pasatiempos y notas.	Teléfono: natural de 5 a 12 dígitos; teléfono y área únicos. Nombre: 1 a 50. Correo: formato parte1@parte2 sin espacios. Dirección: 0 a 80. Fecha dd/mm/aaaa válida (0000 si no se conoce el año; 0 si no se conoce la fecha). Pasatiempos y notas: 0 a 60.
4. Administrar grupos de contactos	Mantenimiento de las listas grupos y contactos_por_grupo (relación 1:1 por índice): agregar, modificar y eliminar grupos; agregar y eliminar contactos de los grupos.	Nombre del grupo: 1 a 40 caracteres, único. Solo se agregan contactos existentes. No hay contactos repetidos en un grupo. Confirmación al eliminar.
5. Lista de contactos	Emitir un listado en PDF con columnas por cada dato del contacto, según seis filtros: nombre, teléfono, área, fecha de nacimiento, pasatiempos y grupo.	Sin distinguir mayúsculas y minúsculas. Filtro vacío = cualquier valor. El carácter comodín % reemplaza cero o más caracteres.
6. Ayuda	Desplegar en pantalla el manual de usuario.	—
7. Acerca de	Mostrar nombre del programa, versión, fecha de creación y autor.	—

0. Fin	Terminar el programa.	Opciones inválidas muestran mensaje de error.

Temas Investigados

2.1 Herramientas de Depuración (Debugger)

¿De qué trata?

Un depurador (debugger) es una herramienta que permite ejecutar un programa de forma controlada para observar su comportamiento: se puede pausar en puntos determinados, avanzar instrucción por instrucción e inspeccionar el valor de las variables en cada momento. Su propósito es localizar la causa de los errores, sobre todo los de lógica, sin tener que llenar el código de instrucciones print. Probar un programa permite descubrir que hay un error; depurar permite encontrar dónde y por qué ocurre (Microsoft, s. f.-a; Python Software Foundation, s. f.-d).

Conceptos principales:

- **Punto de interrupción (breakpoint):** marca una línea donde la ejecución se detiene.
- **Ejecución paso a paso:** Step Over ejecuta la línea actual sin entrar en las funciones que llame; Step Into entra en la función llamada; Step Out termina la función actual y regresa a quien la llamó.
- **Continue:** reanuda la ejecución hasta el siguiente breakpoint o hasta el final.
- **Inspección de variables y expresiones (Watch):** permite ver cómo cambian los valores durante la ejecución.
- **Pila de llamadas (Call Stack):** muestra la secuencia de funciones que llevó al punto actual.
- **Breakpoint condicional:** solo detiene la ejecución cuando se cumple una condición.

Python también incluye un depurador de consola, el módulo pdb, y la función breakpoint() para pausar la ejecución desde el propio código.

¿Cómo se aplicó al proyecto?

1. Coloqué un breakpoint en la instrucción posicion = -1, para observar cómo funciona la búsqueda de áreas utilizando el depurador. Primero probé el código 506 cuando la lista estaba vacía. Después registré el área 506, Costa Rica, y busqué ese mismo código para observar cómo la variable posicion cambiaba a 0, porque el área estaba en la primera posición de la lista. Por último, busqué el código 207, que no estaba registrado, y observé que posición se mantenía en -1, indicando que el área no se había encontrado.

Se estaba buscando el código 506, pero areas estaba vacía ([]). Al ejecutar la búsqueda, la función devuelve -1 porque no encuentra ningún elemento.

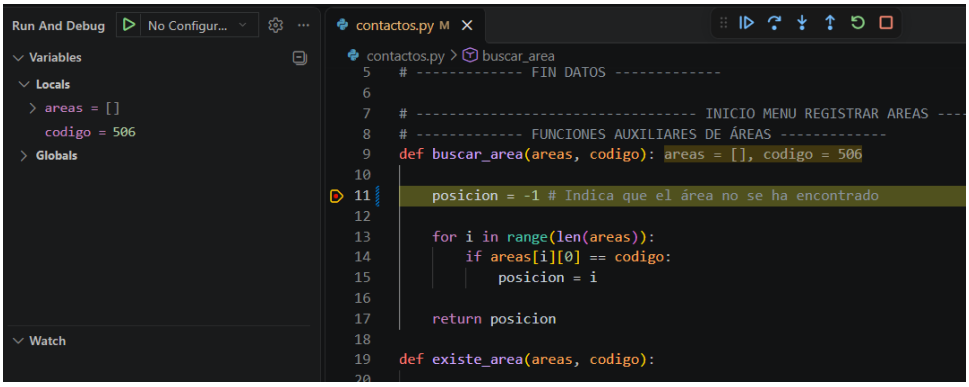


Figura 1. Depuración de buscar_area() con la lista areas vacía (posicion = -1).

La lista areas contenía (506, "Costa Rica") y que al buscar 506, posicion cambió a 0 porque encontró el área en la primera posición.

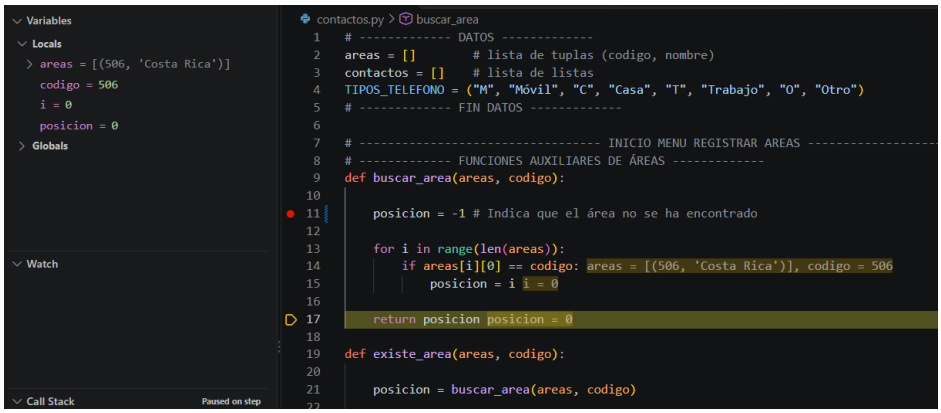


Figura 2. Depuración de buscar_area() al buscar el código 506 (posicion = 0).

207 es diferente de 506, por ende, se conservó la posicion = -1.

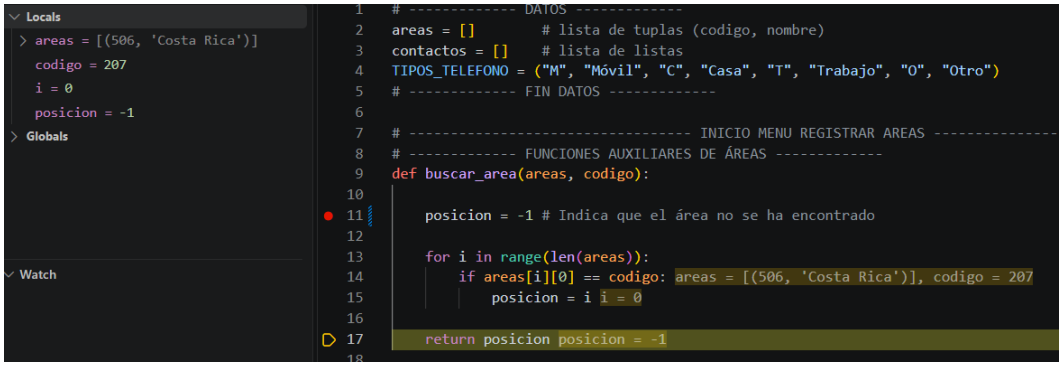


Figura 3. Depuración de buscar_area() al buscar el código 207 (posicion = -1).

Además, utilicé F5 para continuar la ejecución y F10 para avanzar la instrucción por instrucción.

2. ¿Por qué utilicé el depurador?

Utilicé el depurador durante las pruebas de la función buscar_area() porque quería observar cómo cambiaban las variables mientras se ejecutaba el programa. Con F10 avancé paso a paso y comprobé cuándo la variable posicion tomaba el valor 0 o permanecía en -1. Esto me ayudó a comprender mejor la búsqueda dentro de la lista sin tener que colocar instrucciones print() para mostrar los resultados.

2.2 Depurador del IDE Utilizado y su Aplicación en el Proyecto

Función del depurador	Qué hace	Atajo en VS Code (Windows/Linux)
Breakpoint	Detiene la ejecución en la línea marcada (clic a la izquierda del número de línea).	F9 (alterna)
Iniciar / Continue	Inicia la depuración o reanuda hasta el siguiente breakpoint.	F5
Step Over	Ejecuta la línea actual sin entrar en las funciones que contiene.	F10
Step Into	Entra en la función llamada en la línea actual.	F11
Step Out	Termina la función actual y regresa al código que la llamó.	Mayús + F11
Panel Variables	Muestra las variables locales y globales con su valor actual.	—
Watch	Permite vigilar expresiones o variables específicas durante toda la sesión.	—
Call Stack	Muestra la cadena de funciones activas y permite navegar entre ellas.	—
Breakpoint condicional	Solo pausa cuando una expresión es verdadera (clic	—

	derecho sobre el breakpoint).	

(Microsoft, s. f.-a, s. f.-b)

Funciones usadas en este programa

Función del programa depurada	**Opción del menú**	**Síntoma observado**	**Qué mostró el depurador**	**Corrección**
buscar_area()	1. Registrar áreas	Necesitaba comprobar que la búsqueda devolviera correctamente la posición de un área.	El código 506 se encontró en la posición 0 y el código 207 devolvió -1 al no estar registrado.	No fue necesario corregir esta función; se verificó su funcionamiento.
buscar_contacto()	Registrar contactos → Consultar	Ninguno. Se depuró para verificar que la búsqueda por teléfono y área recorre la lista correctamente.	Con telefono = 88886666 y area = 506, en la iteración i = 1 la variable posicion seguía en -1 justo antes de evaluar la condición	No fue necesaria.
buscar_contacto()	Registrar contactos → Consultar	Ninguno.	Al cumplirse la condición, posicion cambió de -1 a 1 y ese valor quedó listo para devolverse con return	No fue necesaria.

Las capturas de la depuración de buscar_area() se presentan en la sección 2.1, donde se observa el punto de interrupción y el panel de variables.

Las capturas de la depuración de buscar_contacto() se presentan a continuación. La primera muestra la iteración i = 1 antes de evaluar la condición, y la segunda muestra el valor de posicion una vez que la condición se cumplió.

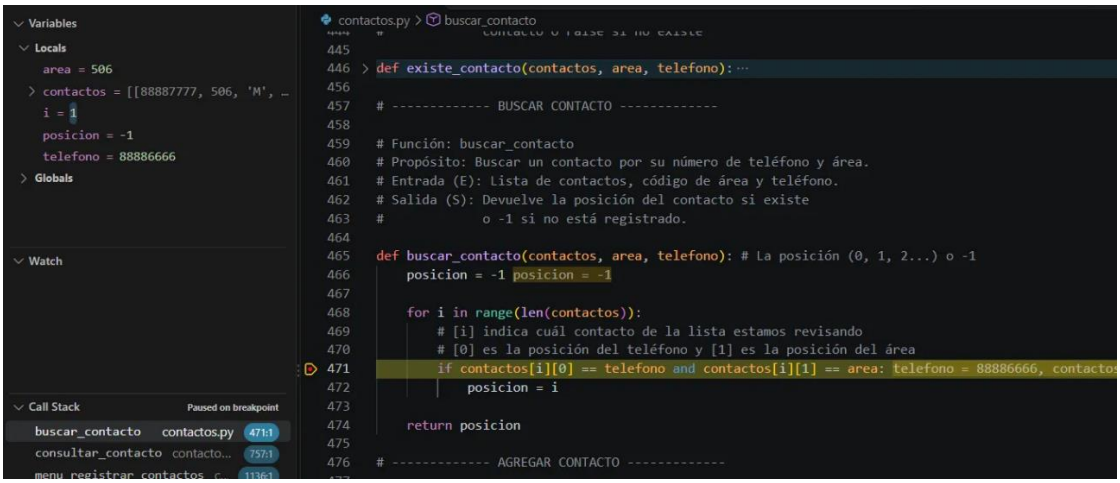


Figura 4. Depuración de buscar_contacto() en la iteración i = 1, antes de evaluar la condición.

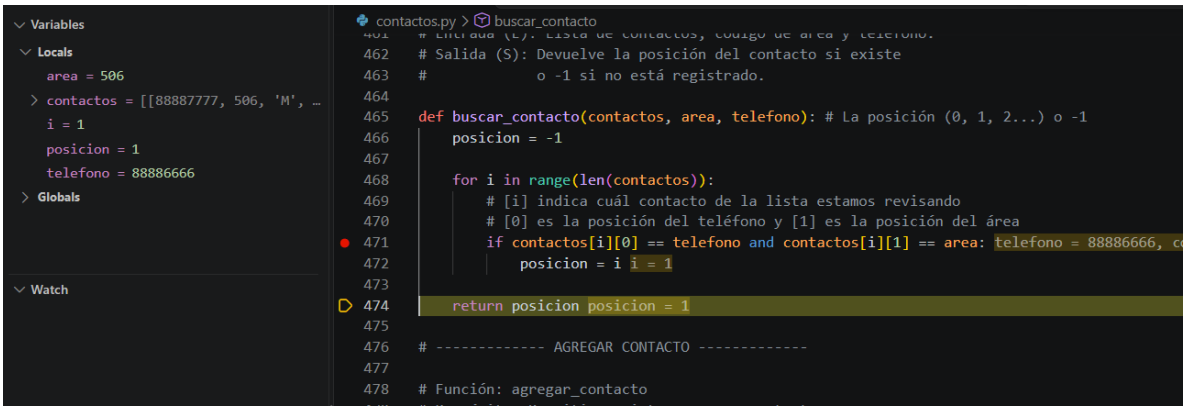


Figura 5. Depuración de buscar_contacto() después de cumplirse la condición: posicion = 1.

2.3 Importancia de la Documentación de Software

¿De qué trata?

La documentación de software es el conjunto de textos que explican qué hace un programa, cómo está construido y cómo se usa. Se divide en documentación interna (comentarios, nombres significativos de variables y funciones) y documentación externa (documentación técnica del proyecto y manual de usuario).

Es indispensable porque:

- Facilita el mantenimiento: quien lee el código meses después (incluso su propio autor) entiende rápidamente qué hace cada función, qué recibe y qué retorna.
- Permite el trabajo en equipo y la revisión de código, porque los acuerdos y supuestos quedan escritos.

- Reduce errores, ya que especificar entradas, salidas y restricciones obliga a pensar los casos límite antes de programar.
- Facilita la reutilización de funciones y las pruebas.
- Hace que el usuario final pueda usar el programa sin conocer su código (manual de usuario).

En Python existen guías oficiales para esto: PEP 8 indica cómo escribir el código de forma ordenada y fácil de leer (por ejemplo, nombres de funciones en minúscula separados por guion bajo, como buscar_area), y PEP 257 indica cómo documentar las funciones (Goodger y van Rossum, 2001; van Rossum et al., 2001).

¿Cómo se aplicó al proyecto?

En el programa utilicé comentarios para explicar las principales funciones, validaciones y operaciones. También procuré utilizar nombres de variables relacionados con los datos que se manejan, como codigo, nombre, areas y configuracion.

Por ejemplo, la función buscar_area(areas, codigo) recibe la lista de áreas y un código para buscarlo. Si encuentra el código, devuelve su posición dentro de la lista y, si no lo encuentra, devuelve -1.

Además, fui elaborando el manual de usuario conforme desarrollaba y probaba las funcionalidades, incluyendo instrucciones paso a paso y capturas de la consola.

2.4 Expresiones Regulares (Regex) y Validación de Correo Electrónico

Definición

Una expresión regular es una secuencia de caracteres que describe un patrón de texto. Sirve para comprobar si una cadena cumple cierto formato, buscar coincidencias dentro de un texto o reemplazar partes de él. En Python se trabajan con el módulo re, que incluye funciones como re.fullmatch (el patrón debe coincidir con toda la cadena), re.search (busca el patrón en cualquier parte), re.match (coincide desde el inicio) y re.escape (neutraliza los caracteres especiales de un texto). La bandera re.IGNORECASE ignora la diferencia entre mayúsculas y minúsculas (Python Software Foundation, s. f.-e, s. f.-f).

Símbolo	Significado
.	Cualquier carácter (excepto salto de línea).
* + ?	Repeticiones del elemento anterior: cero o más, una o más, cero o una.
{n,m}	Entre n y m repeticiones.
[abc] [^abc]	Uno de los caracteres del conjunto / cualquier carácter que NO esté en el conjunto.
^ $	Inicio y fin de la cadena.
\d \s	Un dígito / un espacio en blanco.
| ( )	Alternativa (o) / agrupación.
\	Escapa un carácter especial para que se interprete literalmente.

Aplicación en la validación del correo electrónico

El enunciado define el correo como un string con formato parte1@parte2, donde parte1 y parte2 no contienen espacios en blanco y pueden tener cualquier tamaño. Un patrón que expresa esa regla es:

[^@\s]+@[^@\s]+

- [^@\s]+ (primera vez): uno o más caracteres que no sean @ ni espacio en blanco, es decir, parte1.
- @: el símbolo @ literal, exactamente uno.
- [^@\s]+ (segunda vez): uno o más caracteres que no sean @ ni espacio, es decir, parte2.
- Se aplica con re.fullmatch para que el patrón cubra toda la cadena y no solo una parte.

Este patrón sigue el formato simplificado que pide el enunciado; no valida todas las reglas de una dirección real de correo (por ejemplo, la existencia del dominio).

En el programa utilicé esta expresión regular para validar el correo en agregar_contacto() y modificar_contacto(), con re.fullmatch(r"[^@\s]+@[^@\s]+", correo). Se probó con el correo vacío, con dos "@", sin parte final, sin parte inicial, con espacios y con el formato válido parte1@parte2: los casos inválidos se rechazaron con el mensaje de error y el válido se aceptó.

¿Hay otras partes del programa donde se pueden aplicar?

Parte del programa	**Posible uso de regex**	**¿Se aplicó?**
Teléfono (5 a 12 dígitos)	Comprobar que son solo dígitos y su cantidad, por ejemplo con \d{5,12}.	No se utilizó una expresión regular. El teléfono se valida con .isdigit() y len() (de 5 a 12 dígitos), y comprobando que sea mayor que cero.
Código de área (1 a 999)	Comprobar que son de 1 a 3 dígitos (el rango se valida después del cambio a entero).	No se utilizó una expresión regular. El código se valida con .isdigit() y posteriormente se comprueba que esté entre 1 y 999.
Fecha dd/mm/aaaa	Comprobar el formato con \d{2}/\d{2}/\d{4}; la validez real de la fecha se verifica aparte.	No se utilizó una expresión regular. El formato se valida con split("/"), len() e isdigit(), y la existencia de la fecha con datetime.date().

Parte del programa	Posible uso de regex	¿Se aplicó?
Filtros con comodín %	El comodín % equivale a «cualquier cantidad de caracteres» (.* en regex). Se debe tratar el texto del usuario con re.escape y usar re.IGNORECASE.	Sí. Se aplicó en la función coincide_con_filtro(): el filtro se separa con split("%"), cada parte se protege con re.escape(), las partes se unen con ".*" (cada % equivale a cero o más caracteres) y se compara con re.fullmatch() y la opción re.IGNORECASE, que además hace que la búsqueda no distinga mayúsculas de minúsculas. Un filtro sin % solo cumple si es exactamente igual al dato. Se probó con el comodín al inicio, al final, en el medio y en ambos lados (ver Pruebas de funcionamiento de Lista de contactos).

2.5 Emisión de información en archivos PDF

¿De qué trata?

PDF (Portable Document Format) es un formato de documento que conserva la apariencia del contenido sin importar el programa o dispositivo con el que se abra, por lo que es adecuado para emitir listados e informes. Python no lo genera por sí solo, así que se usa una biblioteca externa instalada con pip. Dos opciones frecuentes son fpdf2 y ReportLab (py-pdf, s. f.; ReportLab, s. f.).

Aspecto	fpdf2	ReportLab
¿Qué es?	Biblioteca para generar PDF en Python, escrita en Python y con pocas dependencias (Pillow, defusedxml y fontTools).	Biblioteca que permite crear PDF directamente desde Python; ofrece una capa de bajo nivel (canvas) y una de alto nivel (Platypus) para maquetar documentos.
Instalación	pip install fpdf2	pip install reportlab
Tablas	Método table() integrado, con saltos de página automáticos.	Clases Table y TableStyle de Platypus, junto con SimpleDocTemplate.
Documentación oficial	py-pdf.github.io/fpdf2	Guía de usuario en reportlab.com/docs

Biblioteca elegida y razón:

fpdf2 (versión 2.8.9). Se eligió porque se instala con un solo comando (pip install fpdf2), tiene pocas dependencias, su forma de uso es sencilla (crear el documento, agregar una página, elegir la fuente, escribir y guardar) e incluye un método table() para crear tablas, que es lo que necesita el listado de contactos. Además, su fuente básica Helvetica muestra correctamente las tildes, la ñ y los signos ¿ y ¡. Antes de aplicarla en el programa se hizo una prueba en un archivo aparte (prueba_pdf.py, fuera de la carpeta del proyecto) que generó un PDF de diez líneas con estos caracteres.

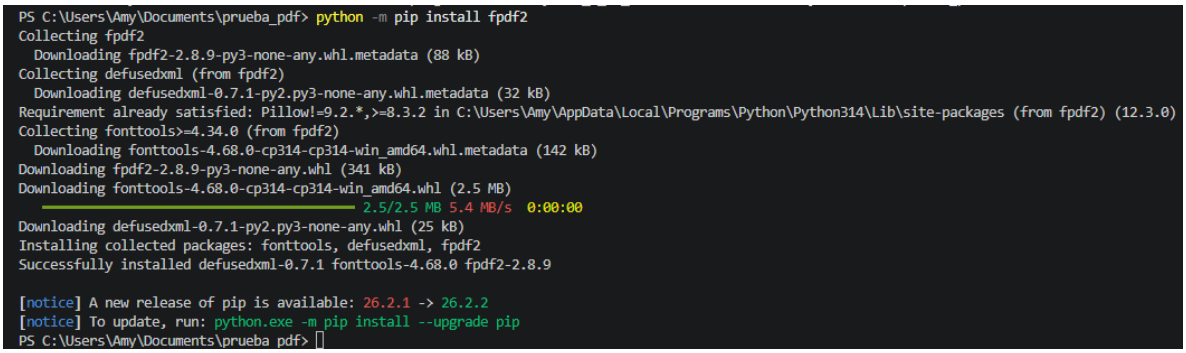


Figura 6. Instalación de fpdf2 2.8.9.

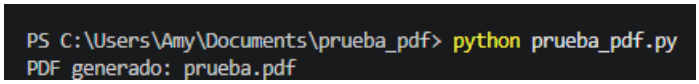


Figura 7. Ejecución de la prueba de generación de PDF

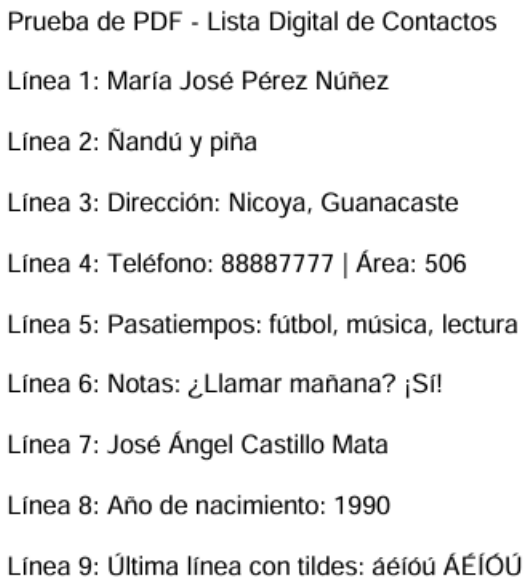


Figura 8. PDF de prueba con tildes, ñ y signos ¿ ¡.

¿Cómo se aplicó al proyecto?

La biblioteca se aplicó en la función emitir_listado(), que recibe por parámetro la lista de contactos que ya cumplen los filtros (obtenida por filtrar_contactos()) y genera el archivo PDF. Si la lista está vacía, no se genera el archivo y se muestra el mensaje “No hay contactos que cumplan con los filtros.”. Antes de usar la biblioteca, el programa comprueba con

importlib.util.find_spec() que fpdf2 esté instalada; si no lo está, muestra un mensaje con el comando para instalarla en lugar de detenerse con un error.

El PDF se crea en hoja horizontal para que quepan las ocho columnas que pide el enunciado: nombre, teléfono y área, tipo de teléfono, correo electrónico, dirección física, fecha de nacimiento, pasatiempos y notas. Lleva un título, la cantidad de contactos encontrados y una tabla creada con el método table() de fpdf2, en la que la primera fila es el encabezado y se repite en cada página; los textos largos bajan de línea dentro de su celda. El tipo de teléfono se muestra con su descripción (Móvil, Casa, Trabajo u Otro) usando la tupla TIPOS_TELEFONO.

Como la fuente Helvetica solo maneja el alfabeto latin-1, la función auxiliar texto_pdf() convierte cada dato en texto y reemplaza por "?" los caracteres que la fuente no puede dibujar (por ejemplo, un emoji), para que el programa no falle. Las tildes, la ñ y los signos ¿ y ¡ se muestran correctamente.

El archivo se guarda en la misma carpeta que contactos.py con el nombre lista_contactos_1.pdf; si ese nombre ya existe, se usa el siguiente número (lista_contactos_2.pdf, etc.) para no reemplazar un listado anterior. Al terminar, el programa muestra en pantalla el nombre del archivo generado y la carpeta donde se guardó.

LISTA DIGITAL DE CONTACTOS - LISTA DE CONTACTOS

Contactos encontrados: 6

Nombre	Teléfono y área	Tipo	Correo electrónico	Dirección física	Fecha de nacimiento	Pasatiempos	Notas
María Fernández	88887777 (506)	Móvil	maria@correo.com	San José, del parque 200 m al norte, casa esquinera	15/06/1990	leer, música	Compañera del colegio
Ana Mora	88886666 (506)	Casa	ana@correo.com	Heredia	20/03/1995	futbol, cine	¿Llamar mañana? ¡Sí!
William Mata	88885555 (47)	Trabajo	william@correo.no	Oslo	0	futbol	Nota con emoji ?
Alberto Castillo Matamoros	22223333 (506)	Otro	a_castillo@yahoo.com	Cartago centro	30/01/2001	futbol, basquet, videojuegos	está de vacaciones
Luis Castillo	88884444 (502)	Móvil	luis@correo.gt		15/08/0000		
Ñandú Núñez Ávila	88883333 (506)	Móvil	nandu@correo.com	Nicoya, Guanacaste	01/12/1988	pesca	Prueba de ñ y tildes

Figura 9. Listado de contactos generado en PDF

2.6 Herramientas de control de versiones y colaboración

¿A qué se refieren y por qué son importantes?

Un sistema de control de versiones registra los cambios que se hacen a los archivos de un proyecto a lo largo del tiempo. Permite volver a versiones anteriores, comparar

cambios, trabajar en paralelo mediante ramas y colaborar con otras personas sin pisar su trabajo (Chacon y Straub, 2014; GitHub, s. f.). En el desarrollo de software es importante porque:

- Deja un historial completo y trazable de quién cambió qué y por qué.
- Permite recuperar el trabajo ante un error o una modificación equivocada.
- Facilita el trabajo colaborativo y la revisión de cambios.
- Sirve como respaldo del proyecto si se usa un repositorio remoto.

Herramienta	**Qué es**	**Características principales**
Git	Sistema de control de versiones distribuido: cada persona tiene una copia completa del repositorio con su historial.	Commits, ramas (branches), fusiones (merge), trabajo local sin conexión, sincronización con repositorios remotos.
GitHub	Plataforma web que aloja repositorios Git y agrega herramientas de colaboración.	Repositorios remotos, pull requests, issues, revisión de código, automatización de flujos de trabajo.
GitLab	Plataforma web para alojar repositorios Git con herramientas de ciclo de desarrollo integradas.	Repositorios remotos, solicitudes de fusión (merge requests), integración y entrega continuas (CI/CD) incorporadas.

Mejores prácticas

- Hacer commits pequeños y frecuentes, cada uno con un mensaje que explique qué se hizo y por qué.
- Hacer al menos un commit por cada funcionalidad terminada (requisito del proyecto).
- Usar ramas para desarrollar funcionalidades nuevas sin afectar la versión estable.

- Revisar los cambios antes de hacer commit y no subir archivos temporales (usar .gitignore, por ejemplo para __pycache__).
- Usar pull requests y revisión de código cuando se trabaja en equipo.
- No subir contraseñas ni datos sensibles al repositorio.
- Sincronizar con el repositorio remoto con regularidad.

Estrategia de commits utilizada en el proyecto

Enlace del repositorio: https://github.com/amy10q/Proyecto_1_TP_2S-2026

Funcionalidad del menú	**Mensaje del commit**	**Fecha**
1. Registrar áreas	Avance de registrar áreas: funciones y documentación, Ajustes a eliminar áreas, Ajustar mensajes de Registrar áreas al enunciado	18/10/2026, 22/10/2026, 23/10/2026
2. Configuración	Implementar configuración de contactos y mejorar validaciones de áreas.	18/10/2026

Funcionalidad del menú	Mensaje del commit	Fecha
3. Registrar contactos	Línea 1: Avance registrar contactos: busqueda y validaciones iniciales Línea 2: Implementar agregar contactos: validaciones, confirmación y repetición de la ventana Línea 3: Implementar consultar contactos Línea 4: Implementación eliminar contactos Línea 5: Agrega modificar_contacto Línea 6: Agregue revisión de grupos al eliminar contacto	Línea 1: 20/10/2026 Línea 2: 21/10/2026 Línea 3: 22/10/2026 Línea 4: 23/10/2026 Línea 5: 24/10/2026 Línea 6: 25/10/2026
4. Administrar grupos de contactos	Línea 1: Implementación de agregar_grupo Línea 2: Agrega agregar_contacto_a_grupo Línea 3: Agregue eliminar_grupo Línea 4: Agregue eliminar_contacto_de_grupo Línea 5: Agregue modificar_grupo	Línea 1: 24/10/2026 Línea 2: 25/10/2026 Línea 3: 25/10/2026 Línea 4: 25/10/2026 Línea 5: 26/10/2026

Funcionalidad del menú	Mensaje del commit	Fecha
5. Lista de contactos (PDF)	Línea 1: Agregue lista_contactos con filtros Línea 2: Agregue emision del listado en PDF	Línea 1: 27/10/2026 Línea 2: 31/10/2026
6. Ayuda	Agregue ayuda	30/10/2026
7. Acerca de	Agregue acerca_de	28/10/2026

2.7 Registro y Uso Regulado de Inteligencia Artificial (IA)

El enunciado permite usar IA como apoyo para investigar conceptos técnicos, apoyar la investigación y comprender bibliotecas externas, siempre que cada uso quede documentado. La información de la IA puede ser imprecisa, por lo que cada respuesta se contrastó con fuentes oficiales (indicadas en cada tabla) (Anthropic, 2026; OpenAI, 2026).

Uso 1: revisión del apartado Registrar áreas

Campo	Contenido
Objetivo del uso	Revisar el menú de Registrar áreas y comprender cómo estructurar la opción de agregar área y sus funciones auxiliares, incluyendo la validación del código y la comprobación de que el área no esté registrada.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí revisar el código para ver si había posibles errores de nombres.

Campo	Contenido
Respuesta	Revisó mi código y señaló algunos errores, como una lista no definida, la comparación del nombre en lugar del código, el uso de append() sin argumentos, el uso de int(input()) que podía generar errores al ingresar la letra C, títulos de menús repetidos y mensajes de finalización del programa en los submenús. También indicó aspectos que debía revisar en la organización de las funciones auxiliares, sus nombres, entradas y salidas. No entregó código.
¿Cómo usó o adaptó la respuesta?	No copié código: la IA no entregó las funciones completas, así que las escribí yo a partir de los nombres, entradas y salidas propuestos. Corregí los errores señalados, definí las estructuras vacías (areas como lista de tuplas) y ajusté los títulos de pantalla al enunciado.
Reflexión crítica	Comprendí la importancia de validar el código antes de convertirlo a entero y de comprobar si un área está registrada antes de agregarla. También aprendí a dividir el problema en funciones más pequeñas.

Uso 2: estructura de la documentación

Campo	Contenido
Objetivo del uso	Estructurar la documentación del proyecto y identificar qué debía investigarse en cada tema.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí un machote para la parte escrita que cumpliera con la estructura del profesor y la tabla de uso de IA; luego pedí mejorar mi borrador.

Campo	Contenido
Respuesta	Entregó la estructura base del documento y campos marcados para completar con información propia del proyecto (horas, avance, problemas, casos de depuración, commits).
¿Cómo usó o adaptó la respuesta?	Utilicé la estructura propuesta como base para organizar el documento. Posteriormente fui completando los apartados con información del trabajo realizado, los problemas encontrados, el avance de la programación y los commits de GitHub.
Reflexión crítica	La respuesta me ayudó a organizar de una mejor manera los temas solicitados por el profesor.

Uso 3: Comprensión de listas y comprensión de configuración de contactos.

Campo	Contenido
Objetivo del uso	Comprender cómo almacenar y actualizar los valores predeterminados de área y tipo de teléfono mediante una lista.
Herramienta utilizada	ChatGPT (OpenAI)
Prompt o pregunta	No comprendí para qué es la lista, de que me sirve si no está, porque razón sería más complicado.
Respuesta	Explicó las diferencias entre utilizar una lista y variables individuales.
¿Cómo usó o adaptó la respuesta?	Revisé cada parte, agregué comentarios y comprobé los resultados ejecutando el programa.

Campo	Contenido
Reflexión crítica	Comprendí que la lista guarda solamente un área y un tipo de teléfono por omisión, y modificar la configuración remplaza los valores anteriores.
Otros	Comparé los requisitos con las páginas del enunciado.

Uso 4: Investigación sobre validación de cadenas y números en Python.

Campo	Contenido
Objetivo del uso	Entender como funciona .strip(), .isdigit() y len() para validar los datos introducidos por el usuario.
Herramienta utilizada	ChatGPT (OpenAI)
Prompt o pregunta	Entendido, pero a la hora de escribir algo de 40 caracteres, de que me sirve el .strip(), si al final los espacios en medio no cuentan como "Costa Rica" y que pasa si pongo también "50 6".
Respuesta	Explicó que .strip() elimina espacios de los extremos, .isdigit() comprueba si una cadena contiene únicamente dígitos y len() cuenta caracteres, incluidos los espacios internos.
¿Cómo usó o adaptó la respuesta?	Agregué .strip en las entradas de códigos de área de las operaciones Agregar, Consultar, Modificar y Eliminar. Conservé la validación con .isdigit().
Reflexión crítica	Aprendí que los espacios entre palabras cuentan como caracteres y que .strip() no elimina los espacios internos.

Campo	Contenido
Otros	Probé entradas como “506 “ y “5 06” para comprobar el comportamiento.

Uso 5: Orientación para realizar pruebas de Configuración

Campo	Contenido
Objetivo del uso	Comprender cómo comprobar los casos de guardar, actualizar, cancelar y validar entradas incorrectas.
Herramienta utilizada	ChatGPT (OpenAI)
Prompt o pregunta	Dime que es lo que tengo que probar para verificar que todo funciona de la manera correcta.
Respuesta	Propuso casos de prueba con valores específicos y explicó cómo verificar temporalmente el contenido de la lista configuracion usando print()
¿Cómo usó o adaptó la respuesta?	Ejecuté las pruebas en la consola y comprobé que guardar producía [506, 'M'], actualizar producía [47, 'C'] y cancelar conservaba los valores anteriores.
Reflexión crítica	Aprendí que no basta con mostrar un mensaje de éxito; también es necesario comprobar que los datos quedaron almacenados correctamente.
Otros	Utilicé un print() temporal durante las pruebas y lo retiré al terminar.

Uso 6: Revisión de redacción del manual de usuario.

Campo	Contenido
Objetivo del uso	Revisar la claridad de las instrucciones y la redacción del apartado Configuración de contactos.
Herramienta utilizada	ChatGPT (OpenAI)
Prompt o pregunta	Revisa si hay mala redacción, y si la hay, sugiéreme que podría poner en vez de lo que haya.
Respuesta	Señaló correcciones de ortografía y propuso explicaciones más claras sobre la selección de área, los tipos de teléfono y las opciones de aceptar o cancelar.
¿Cómo usó o adaptó la respuesta?	Revisé el texto de mi manual e hice ajustes necesarios, conservando las capturas de pantalla que obtuve al ejecutar el programa.
Reflexión crítica	Aprendí que el manual debe explicar de manera ordenada y utilizar los mismos nombres y mensajes que aparecen en el programa.
Otros	La revisión se aplicó al apartado de Configuración del manual.

Uso 7: herramientas de depuración (debugger)

Campo	Contenido
Objetivo del uso	Entender qué es un depurador, qué herramientas existen, qué funciones ofrece el debugger de mi IDE y cuáles usé para depurar el programa.
Herramienta utilizada	Claude (Anthropic).

Campo	Contenido
Prompt o pregunta	Pregunté qué es un depurador, para qué sirve, cuáles son sus funciones principales (puntos de interrupción, ejecución paso a paso, inspección de variables) y cómo se usan en Python
Respuesta	Un depurador permite ejecutar el programa de forma controlada para encontrar errores lógicos: pausar en puntos de interrupción (breakpoints), avanzar línea por línea, ver y observar variables, y revisar la pila de llamadas. Funciones típicas: Continue, Step Over (ejecuta la línea sin entrar en la función), Step Into (entra a la función), Step Out (sale de la función actual), Restart y Stop; también breakpoints condicionales y expresiones de observación (watch). Python incluye además pdb y la función breakpoint().
¿Cómo usó o adaptó la respuesta?	Usé la explicación como base y la reescribí con las funciones que realmente tiene mi IDE.
Reflexión crítica	Fue útil para pasar de depurar con print() a seguir el valor de las variables paso a paso.

Uso 8: importancia de la documentación de software

Campo	Contenido
Objetivo del uso	Comprender por qué se documenta el software, la diferencia entre documentación interna y externa, y las buenas prácticas para documentar código Python.
Herramienta utilizada	Claude (Anthropic).

Campo	Contenido
Prompt o pregunta	Pregunté por la importancia de la documentación de software y qué incluir en la documentación interna (comentarios) y externa (manuales, documentos del proyecto).
Respuesta	La documentación facilita el mantenimiento, la colaboración, la incorporación de nuevas personas al proyecto y reduce errores al dejar claro qué hace cada parte. La interna vive en el código (comentarios, docstrings con entradas, salidas y restricciones de cada función, nombres significativos); la externa es el material fuera del código (documento del proyecto, ayuda al usuario, manual).
¿Cómo usó o adaptó la respuesta?	Apliqué en los comentarios de cada función el formato Función, Propósito, Entrada (E) y Salida (S).
Reflexión crítica	Fue útil para ordenar la explicación y justificar el estilo de documentación interna. Aprendí que documentar mientras se programa ahorra tiempo al depurar y al ampliar el programa.

Uso 9: expresiones regulares

Campo	Contenido
Objetivo del uso	Entender qué son las expresiones regulares, cómo se usan en Python con el módulo re, cómo validar un correo electrónico con el formato del enunciado y en qué otras partes del programa se podrían aplicar.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pregunté la definición de expresión regular, cómo validar un correo con formato parte1@parte2 (sin espacios) en Python y qué otros datos del programa se podrían validar así.

Campo	Contenido
Respuesta	Una expresión regular es un patrón de caracteres que describe un conjunto de cadenas; sirve para buscar, validar y reemplazar texto. En Python se usa el módulo re (re.fullmatch, re.match, re.search, re.findall, re.sub, re.compile). Correo según el enunciado (parte1@parte2, sin espacios en blanco): re.fullmatch(r"[^@\s]+@[^@\s]+", correo). Para una validación más estricta de correos reales (RFC 5322) el patrón es mucho más complejo; aquí basta el formato pedido. Otras aplicaciones: teléfono de 5 a 12 dígitos r"\d{5,12}"; código de área de 1 a 999 r"[1-9]\d{0,2}"; tipo de teléfono r"[MCTO]"; formato de fecha r"\d{2}/\d{2}/\d{4}" (la validez real de la fecha la comprueba datetime). Verificación: probé el patrón del correo en el programa con correos válidos e inválidos (sin @, con dos @, con espacios y vacío).
¿Cómo usó o adaptó la respuesta?	Investigé el uso de expresiones regulares (Regex) para identificar y validar patrones de texto, enfocándome en los correos electrónicos, que deben contener un símbolo "@" y no tener espacios ni símbolos adicionales. Implementé una expresión regular simple con "re.fullmatch()" en las funciones "agregar_contacto()" y "modificar_contacto()" para validar el formato. Además, utilicé expresiones regulares para realizar búsquedas con el comodín "%" en la lista de contactos, aunque no usé los patrones para teléfonos, áreas y fechas propuestos, y opté por validar esos datos con métodos como "isdigit()", "len()" y "datetime".
Reflexión crítica	Fue útil, aunque entendí que una expresión regular revisa la forma del dato y no si existe de verdad (un correo puede cumplir el patrón y no

Campo	Contenido
	existir). Al principio el patrón me parecía difícil de leer, pero al separarlo símbolo por símbolo lo entendí.

Uso 10: generación de archivos PDF

Campo	Contenido
Objetivo del uso	Investigar cómo emitir información en un archivo PDF desde Python y comprender la biblioteca externa usada para generarlo.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pregunté qué bibliotecas de Python permiten generar PDF, cómo instalarlas y cómo escribir texto y tablas con ellas. Después pregunté si era recomendable usar fpdf2, por qué y si era sencilla, y pedí los pasos para hacer una prueba con tildes y ñ antes de usarla en el programa.
Respuesta	Python no trae soporte de PDF en su biblioteca estándar; las opciones más usadas son fpdf2 y ReportLab, que se instalan con pip. Con fpdf2 el flujo básico es: crear un objeto FPDF, add_page(), set_font(), escribir con cell() o multi_cell() y guardar con output("archivo.pdf"). Las fuentes básicas usan codificación Latin-1 (tildes y ñ funcionan, pero otros símbolos no); para Unicode completo se carga una fuente TTF con add_font(). Verificación: instalé fpdf2 (versión 2.8.9) en Python 3.14 y generé un PDF de prueba de diez líneas con tildes, ñ y los signos ¿ y ¡; todos los caracteres se mostraron correctamente (ver sección 2.5).
¿Cómo usó o adaptó la respuesta?	Usé el ejemplo de la respuesta para hacer una prueba en un archivo separado del proyecto (prueba_pdf.py), antes de modificar

Campo	Contenido
	contactos.py. Con esta prueba elegí fpdf2 como biblioteca para emitir el listado de la opción 5 (Lista de contactos).
Reflexión crítica	Fue útil para entender una biblioteca externa sin leer toda su documentación primero. Al hacer la prueba primero, fue para ver si funcionaba correctamente y de esa manera poder aplicarla al proyecto sin que genere ningún problema. Al inicio no entendía cómo funcionaba, pero ya luego de seguir los pasos fui entendiendo.

Uso 11: control de versiones y colaboración

Campo	Contenido
Objetivo del uso	Comprender qué son las herramientas de control de versiones y colaboración, su importancia, tres herramientas (incluyendo git y GitHub) y las mejores prácticas de uso.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pregunté qué es el control de versiones, por qué es importante, qué diferencia hay entre git y GitHub, otras dos herramientas parecidas y buenas prácticas.
Respuesta	Un control de versiones registra el historial de cambios del código, permite volver a versiones anteriores, trabajar en ramas y colaborar sin sobrescribir el trabajo de otras personas. Git es el sistema de control de versiones distribuido que funciona de forma local; GitHub es una plataforma web que aloja repositorios git y agrega colaboración (pull requests, issues, revisión de código). Otras herramientas: GitLab y Bitbucket (plataformas) o Mercurial y Subversion (otros sistemas).

Campo	Contenido
	Mejores prácticas: commits pequeños con mensajes descriptivos, un commit por cada funcionalidad terminada, ramas por funcionalidad, usar .gitignore, no subir contraseñas ni datos sensibles, traer cambios (pull) antes de enviar (push) y revisar el código antes de integrarlo. Verificación: lo apliqué en el repositorio del proyecto, haciendo un commit por cada funcionalidad terminada.
¿Cómo usó o adaptó la respuesta?	Lo apliqué en el repositorio Proyecto_1_TP_2S-2026: hice al menos un commit por cada funcionalidad del menú principal, con mensajes que describen el cambio, y subí cada uno a GitHub con git push.
Reflexión crítica	Fue útil para distinguir git de GitHub, que solía confundir. Aprendí a revisar con git status qué archivos se van a subir y a agregar solo contactos.py, para no subir por error archivos de prueba o documentos.

Uso 12: manejo de fechas con el módulo datetime

Campo	Contenido
Objetivo del uso	Entender cómo validar fechas de nacimiento con formato dd/mm/aaaa usando el módulo datetime.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pregunté cómo validar una fecha dd/mm/aaaa en Python con datetime, y cómo manejar los casos del enunciado (año 0000 y fecha incompleta).
Respuesta	datetime.strptime(texto, "%d/%m/%Y") convierte el texto en fecha y lanza ValueError si no es válida (por ejemplo 31/04/2024 o

Campo	Contenido
	29/02/2023), por lo que se puede usar dentro de try/except. También existe date(año, mes, día), que valida igual. Detalles a tener en cuenta: el año mínimo que acepta datetime es 1 (datetime.MINYEAR), así que el valor 0000 permitido por el enunciado no puede validarse con datetime y se debe tratar aparte; además strptime acepta días y meses de un solo dígito (1/1/2000), por lo que el formato exacto dd/mm/aaaa conviene revisarlo antes (con len/split o una expresión regular). Verificación: probé en el programa fechas inexistentes (29/02/2023 y 31/04/2024), mes y día inválidos (15/13/2000 y 00/01/2000), formatos incorrectos y el año 0000 (30/02/0000 rechazada y 29/02/0000 aceptada).
¿Cómo usó o adaptó la respuesta?	Validé el formato dd/mm/aaaa con split("/"), len() e isdigit(), y comprobé que la fecha exista con datetime.date() dentro de un try/except ValueError. Para el año 0000 uso 2000 solo al revisar día y mes. Una fecha vacía se guarda como 0.
Reflexión crítica	Comprendí que datetime solo admite años desde el 1, por lo que el año 0000, que el enunciado permite cuando no se conoce el año, no se puede validar directamente. Por eso lo reviso con el año 2000, que es bisiesto, y así 29/02/0000 se acepta porque ese día existe, mientras que 30/02/0000 se rechaza porque febrero nunca tiene 30 días. Al principio tuve dificultades para aprender el uso de como se usa el 2000 y al final pude entenderlo de una mejor manera.

Uso 13: Comprensión del módulo re y validación del correo electrónico

Campo	Contenido
Objetivo del uso	Comprender cómo funciona el módulo re de Python y cómo interpretar una expresión regular para validar el correo electrónico con el formato solicitado en el proyecto.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Quiero comprender mejor cómo funciona el módulo re de Python y cómo se utilizan las expresiones regulares para validar un correo electrónico. Explícame paso a paso qué significa cada parte de esa expresión, por qué se utiliza y cómo determina si un correo cumple el formato establecido.
Respuesta	La IA explicó que re es un módulo de Python para trabajar con expresiones regulares. Desglosó el funcionamiento de re.fullmatch(), el uso de cadenas sin procesar (r), el significado de [^@\s]+ y la función del símbolo @ como separador.
¿Cómo usó o adaptó la respuesta?	Analicé el significado de cada símbolo de la expresión regular y su relación con la condición if, el ciclo while True y la instrucción break. Después probé el correo en el programa con valores vacíos, con dos "@", sin parte final, sin parte inicial, con espacios y con el formato válido parte1@parte2.
Reflexión crítica	Comprendí que la expresión [^@\s]+@[^@\s]+ acepta una o más letras o símbolos que no sean "@" ni espacios, luego un "@", y otra vez una o más de esos caracteres, y que re.fullmatch() exige que todo el texto cumpla ese formato. Por eso un correo con un espacio, en

Campo	Contenido
	medio o al final, es rechazado, igual que uno con dos "@" o con una parte vacía.

Uso 14: pruebas con distintos valores

Campo	Contenido
Objetivo del uso	Diseñar casos de prueba con valores válidos, inválidos y límite para comprobar las validaciones de agregar_contacto(), y saber cómo documentar los resultados.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Le pregunté si me podía dar diferentes maneras en la cual pueda probar lo que llevaba hecho.
Respuesta	Claude propuso cuatro grupos de pruebas con los valores a escribir y el resultado esperado: teléfono, área y tipo; nombre y correo; dirección, fecha, pasatiempos y notas (con textos de 50/51, 60/61 y 80/81 caracteres y fechas inválidas); y fecha con año 0000 y fecha vacía. También indicó cómo ejecutarlas, qué capturas tomar y dónde documentarlas.
¿Cómo usó o adaptó la respuesta?	Ejecuté los grupos de prueba en el programa, tomé las capturas y comparé lo observado con lo esperado. Todos los casos coincidieron con lo esperado.
Reflexión crítica	Aprendí que al validar longitudes hay que probar tanto el valor máximo permitido como el siguiente, por ejemplo 50 y 51 caracteres en el nombre, porque ahí es donde suelen aparecer los errores. También

Campo	Contenido
	comprobé casos como un nombre con solo espacios, correos mal formados y fechas que no existen.

Uso 15: sesión de depuración de buscar_contacto() y documentación de la sección 2.2

Campo	Contenido
Objetivo del uso	Aprender a usar el depurador de VS Code sobre la función buscar_contacto() y documentar la sesión de depuración en la sección 2.2 del informe, eligiendo las capturas adecuadas.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí orientación sobre cómo hacer la sesión de depuración (dónde poner los breakpoints y qué teclas usar), cuáles de mis capturas incluir en el informe sin alargarlo, y que revisara la sección 2.2 de mi documento.
Respuesta	Me explicó paso a paso cómo poner breakpoints en consultar_contacto() y buscar_contacto(), iniciar con F5 y avanzar con F10 y F11, y qué mirar en el panel Variables. También me indicó qué dos capturas servían (antes y después del cambio de posicion) y me señaló correcciones del documento.
¿Cómo usó o adaptó la respuesta?	Hice la sesión de depuración yo misma en VS Code y tomé las capturas. Después corregí el nombre de la función, el acento, la frase introductoria.
Reflexión crítica	Comprendí mejor el uso del depurador, porque pude observar cómo cambiaba cada variable a medida que avanzaba paso a paso por el

Campo	Contenido
	código. Por ejemplo, vi cómo posicion pasaba de -1 a 1 al cumplirse la condición.

Uso 16: mejora de comentarios

Campo	Contenido
Objetivo del uso	Mejora en comentarios de código.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí revisar la redacción de los comentarios del programa, tanto los que describen las entradas y salidas de cada función como los comentarios internos, buscando que fueran claros, precisos y fáciles de comprender.
Respuesta	Me sugirió mejoras que se podrían implementar en los comentarios.
¿Cómo usó o adaptó la respuesta?	De acuerdo con lo que me sugirió, fui función por función cambiando lo que me indicó.
Reflexión crítica	Me ayudó a optimizar comentarios y ver que tenía errores de redacción.

Uso 17: relación 1:1

Campo	Contenido
Objetivo del uso	Comprender la relación 1:1 entre las listas grupos y contactos_por_grupo.

Campo	Contenido
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí que me explicara en detalle en qué consiste la relación 1:1.
Respuesta	Explicó que el grupo que está en el índice n de grupos corresponde a la lista que está en el índice n de contactos_por_grupo (por eso ambas listas se actualizan juntas) y propuso una tabla de pruebas.
¿Cómo usó o adaptó la respuesta?	Usé la explicación para entender por qué, al agregar, modificar o eliminar un grupo, siempre se cambian las dos listas en la misma posición. Lo comprobé en las pruebas mostrando temporalmente el contenido de grupos y contactos_por_grupo.
Reflexión crítica	Al principio no entendí para qué servían dos listas separadas. Con el ejemplo pude entender que la posición es lo que las une, como si fueran dos columnas de una misma tabla.

Uso 18: revisar informe y manual

Campo	Contenido
Objetivo del uso	Apoyo en la redacción del informe y del manual de usuario.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí que revisara el informe y el manual de usuario, que me dijera si cumplían con todo lo que piden las instrucciones del profesor y revisar si hay errores de redacción u ortografía, indicando dónde están.

Campo	Contenido
Respuesta	Revisó el informe y el manual, señaló partes incompletas o con errores (análisis intercambiados, viñetas repetidas, marcadores sin completar, textos de plantilla).
¿Cómo usó o adaptó la respuesta?	Corroboré con lo que yo tenía realmente e hice los cambios respectivos de acuerdo con las sugerencias dadas.
Reflexión crítica	Me ayudó a encontrar errores que yo no había visto, como textos repetidos o espacios sin llenar. Aprendí que hay que revisar que lo escrito coincida con lo que de verdad hace el programa.

Uso 19: Lista de contactos y comodín %

Campo	Contenido
Objetivo del uso	Comprender la opción Lista de contactos con sus seis filtros y el comodín %.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí que me explicara de qué trataba esta opción y cómo probarlo.
Respuesta	Explicó que cada % se cambia por ".*" en una expresión regular y que re.IGNORECASE hace que no importen mayúsculas ni minúsculas.
¿Cómo usó o adaptó la respuesta?	Hice las pruebas con los ejemplos del enunciado (%castillo, castillo%, %castillo%, william%mata)

Campo	Contenido
Reflexión crítica	Me costó entender cómo el % se convierte en una expresión regular, pero con los ejemplos del enunciado lo pude ver funcionando.

Uso 20: revisión del código

Campo	Contenido
Objetivo del uso	Comprobar que el código estuviera completo y de acuerdo con los requerimientos antes de hacer los commits.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí verificar que el código del Programa 1 cumpla con todos los requisitos e instrucciones del enunciado.
Respuesta	El código fue revisado y se verificó su correspondencia con los requerimientos del programa, señalando aspectos que cumplían con las instrucciones y errores en comentarios que necesitaban corrección. No se generó ni modificó código.
¿Cómo usó o adaptó la respuesta?	Corregí lo señalado, volví a probar el programa y después hice el commit.
Reflexión crítica	Aprendí a revisar el código completo antes de cada commit.

Uso 21: pruebas de las funciones y textos de resultados

Campo	Contenido
Objetivo del uso	Diseñar las pruebas de cada función (modificar, eliminar, grupos, lista, PDF y ayuda) y redactar los textos de las pruebas de funcionamiento.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí las pruebas paso a paso para cada función, en qué momento tomar cada captura y el texto para describir los resultados en el informe.
Respuesta	Entregó tablas de pruebas con lo que había que escribir y lo que debía aparecer, bloques de datos temporales para no registrar todo a mano y el texto de los resultados según lo que se observó en las capturas.
¿Cómo usó o adaptó la respuesta?	Hice cada prueba yo misma, tomé las capturas y comparé con lo esperado. Cuando una prueba salió mal (por ejemplo, un filtro en el campo equivocado), la repetí. Revisé que cada texto coincidiera con mis capturas antes de pegarlo.
Reflexión crítica	Aprendí que probar no es solo ver que funciona, sino también probar datos incorrectos y casos raros. Tener las pruebas ordenadas me ayudó a no saltarme ninguna.

Uso 22: commits

Campo	Contenido
Objetivo del uso	Hacer correctamente cada commit en Git y subirlo a GitHub.
Herramienta utilizada	Claude (Anthropic).

Campo	Contenido
Prompt o pregunta	Pedí los pasos para hacer cada commit después de terminar cada función.
Respuesta	Indicó los comandos en orden (git status, git add contactos.py, git commit, git push y git log) y recomendó borrar los bloques de prueba antes de cada commit y no subir archivos que no fueran del programa.
¿Cómo usó o adaptó la respuesta?	Seguí los pasos en cada commit, escribí mis propios mensajes y copié cada mensaje real de git log a la tabla de commits.
Reflexión crítica	Al principio no entendía la diferencia entre add, commit y push. Ahora sé que add escoge el archivo, commit guarda el cambio en mi computadora y push lo sube a GitHub.

Uso 23: orden del trabajo

Campo	Contenido
Objetivo del uso	Organizar el orden de trabajo del Programa 1 y saber cuál era el siguiente paso.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí que me sugiriera el orden para hacer cada parte del programa y la documentación, cuánto tiempo podía tomar cada una y, al terminar cada parte, cuál seguía.
Respuesta	Propuso un plan por días, con lo que faltaba del código, del informe y del manual, y al terminar cada parte indicó el siguiente paso y qué documentar.

Campo	Contenido
¿Cómo usó o adaptó la respuesta?	Ajusté el plan a mis horarios y exámenes y lo seguí parte por parte.
Reflexión crítica	Me ayudó a no perderme con tantas partes y a avanzar un poco cada día. Aprendí que dividir el trabajo en pasos pequeños hace más fácil terminarlo.

Uso 24: numeración de figuras y nombres de figuras

Campo	Contenido
Objetivo del uso	Numerar las figuras del informe y del manual y revisar que sus nombres fueran claros.
Herramienta utilizada	Claude (Anthropic).
Prompt o pregunta	Pedí ayuda para numerar las figuras y cambiar los nombres que no fueran claros.
Respuesta	Indicó cómo numerarlas en orden y sugirió nombres más claros para algunas figuras.
¿Cómo usó o adaptó la respuesta?	Numeré las figuras en el documento y cambié los nombres que me parecieron necesarios.
Reflexión crítica	Aprendí que unas figuras bien numeradas y con nombres claros hacen más fácil leer el documento.

2.8 Otros aspectos investigados

Módulo datetime (fechas de nacimiento)

En el programa, la fecha de nacimiento se solicita con el formato dd/mm/aaaa. Primero separo el texto con split("/") y compruebo que tenga tres partes de 2, 2 y 4 dígitos numéricos, usando len() e isdigit(). Después compruebo que la fecha exista con datetime.date(año, mes, día) dentro de un try/except ValueError: si la fecha no existe, por ejemplo 29/02/2023 o 31/04/2024, datetime lanza el error y el programa muestra un mensaje y vuelve a pedir el dato (Python Software Foundation, s. f.-a).

Como datetime no admite el año 0, cuando el año es 0000 (año desconocido) uso el año 2000 solo para revisar el día y el mes; por eso 29/02/0000 se acepta y 30/02/0000 se rechaza. La fecha válida se guarda como texto dd/mm/aaaa. Si el usuario deja el campo vacío, se guarda el número 0, que indica que no se tiene la fecha.

Manejo de errores con try/except

try/except es una estructura de Python que permite ejecutar un bloque de código y, si ocurre un error, controlarlo en lugar de que el programa se detenga. El código que podría fallar se escribe dentro de try, y en except se indica qué hacer cuando ocurre un error de cierto tipo, por ejemplo ValueError (Python Software Foundation, s. f.-b).

En el programa lo utilicé para validar la fecha de nacimiento: dentro de un try se llama a datetime.date(año, mes, día) y, si la fecha no existe, como 29/02/2023 o 31/04/2024, datetime lanza un ValueError. El except lo captura, muestra el mensaje "ERROR: La fecha no es válida." y el programa vuelve a pedir el dato.

Estructura match-case

La instrucción match-case (disponible desde Python 3.10) compara un valor contra varios patrones y ejecuta el bloque del que coincide; el caso case _ funciona como opción por defecto. En el proyecto se usa para los menús, y el caso por defecto emite el mensaje de error cuando se digita una opción que no existe (Bucher y van Rossum, 2020; Python Software Foundation, s. f.-c).

Listas y tuplas

Las listas son estructuras de datos que permiten agregar, eliminar y modificar elementos, mientras que las tuplas no se pueden modificar directamente.

En el programa utilicé una lista llamada areas, cuyos elementos son tuplas que contienen el código y el nombre de cada área. Cuando necesito modificar un área, reemplazo la tupla completa en la posición correspondiente.

También utilicé una lista llamada configuracion para guardar el área y el tipo de teléfono por omisión. Para agregar los primeros valores utilicé append() y, cuando ya existen, los reemplazo mediante las posiciones [0] y [1].

Cada contacto es una lista dentro de la lista contactos. Para modificar un contacto reemplazo directamente sus valores por posición, por ejemplo, contactos[posicion][3] = nuevo_nombre, sin crear un contacto nuevo.

Integridad referencial

La integridad referencial permite mantener relaciones válidas entre los datos del programa. Hasta el momento se implementó la comprobación de que el área seleccionada en Configuración exista en la lista de áreas. También, al agregar un contacto, se comprueba que el área ingresada esté registrada y que la combinación de teléfono y área no esté repetida.

Al eliminar un área, el programa revisa con la función area_en_uso si algún contacto la utiliza; en ese caso muestra el mensaje “EN ESTA ÁREA HAY CONTACTOS REGISTRADOS, NO SE PUEDE ELIMINAR” y vuelve a pedir el dato. De forma similar, al agregar un contacto a un grupo se comprueba con la función buscar_grupo que el grupo exista y con buscar_contacto que el contacto esté registrado en la lista contactos; además, con buscar_contacto_en_grupo se evita repetir un contacto dentro del mismo grupo.

Cuando se elimina un contacto, el programa revisa con la función contacto_en_algun_grupo si ese contacto pertenece a algún grupo. Si pertenece, decidí rechazar la eliminación y mostrar el mensaje “ESTE CONTACTO PERTENECE A UN GRUPO, NO SE PUEDE ELIMINAR”, en lugar de borrarlo automáticamente de los grupos. El enunciado deja libre esta decisión. Escogí rechazar porque así el usuario decide de forma consciente quitar primero el contacto de los grupos (con la opción Eliminar contactos de los grupos) y se evita modificar los grupos sin que se dé cuenta. Además, es el mismo criterio que se usa con las áreas, que no se pueden eliminar si algún contacto las utiliza.

Conclusiones y Trabajo Futuro

3.1 Problemas Encontrados y Soluciones Aplicadas

Problema	Cómo se detectó	Solución aplicada
El programa fallaba si introducía una letra donde esperaba un código numérico.	Durante las primeras pruebas de Registrar áreas, al introducir C o un dato no numérico.	Primero leí la entrada como texto, validé su contenido y después la convertí a entero con int().
No se detectaban correctamente las áreas repetidas.	Al revisar cómo se comprobaban los códigos dentro de la lista de tuplas.	Utilicé buscar_area() y existe_area() para comprobar si un código ya estaba registrado.
Un código como 506 mostraba un error por el espacio final.	Al introducir códigos con espacios durante las pruebas.	Agregué .strip() para eliminar espacios al principio y al final antes de validar con .isdigit().
Configuración solicitaba un código, aunque no hubiera áreas registradas.	Al revisar qué sucedía al entrar a Configuración con la lista areas vacía.	Agregué if len(areas) == 0 para mostrar un aviso y utilizar return para regresar al menú principal.
El módulo datetime no acepta el año 0000, que el enunciado permite cuando no se conoce el año de nacimiento.	Al investigar cómo validar fechas con datetime y al probar la fecha 29/02/0000.	Cuando el año es 0000 se usa el año 2000 (bisiesto) solo para comprobar que el día y el mes existan; la fecha se guarda como el usuario la escribió.

Problema	Cómo se detectó	Solución aplicada
Cuatro mensajes de Registrar áreas no coincidían con el texto exacto del enunciado.	Al revisar el enunciado para completar la Lista de Revisión, con ayuda de IA.	Se cambiaron los mensajes de agregar, consultar, modificar y eliminar áreas por los del enunciado y se volvieron a probar.
El título del menú de grupos no coincidía con el del enunciado (decía "REGISTRAR GRUPOS DE CONTACTO").	Al comparar las pantallas del programa con el enunciado.	Se cambió el título a "ADMINISTRAR GRUPOS DE CONTACTOS" y se volvió a ejecutar el menú.
Un contacto que pertenecía a un grupo se podía eliminar de la lista contactos, y en el grupo quedaba una tupla (teléfono, área) de un contacto que ya no existía.	Al revisar la integridad referencial después de desarrollar los grupos; en eliminar_contacto() había quedado marcado como pendiente.	Se creó la función contacto_en_algun_grupo() y eliminar_contacto() ahora rechaza la eliminación con un mensaje. Se probó con contactos que están en dos grupos, en un grupo y en ninguno.
Al abrir el manual con Ayuda parecía que el programa no regresaba al menú.	Al probar la opción 6, el visor de PDF quedó encima de la terminal.	Se comprobó que el programa sí regresa al menú de inmediato; el PDF solo tapa la ventana. Se indicó en el manual que hay que volver a la terminal para seguir usando el programa.

3.2 Aprendizajes Obtenidos

- Dividir el programa en módulos y funciones auxiliares reutilizables facilita desarrollar y probar cada parte por separado.
- Validar los datos antes de convertirlos o usarlos evita errores en tiempo de ejecución.
- Aprendí a usar el depurador del IDE en lugar de print para observar cómo cambian las variables paso a paso, por ejemplo en buscar_area() y buscar_contacto(), y Git para llevar el historial de cambios del programa.
- Aprendí a trabajar con listas y tuplas, y a buscar información dentro de ellas mediante funciones auxiliares.
- Comprendí mejor la diferencia entre break y return. El primero permite salir de un ciclo, mientras que el segundo termina la ejecución de una función.
- También aprendí a utilizar append() para agregar elementos a una lista y a modificar valores mediante sus posiciones. Además, comprendí la importancia de probar entradas incorrectas y verificar los resultados antes de continuar con otra funcionalidad.
- Comprendí la relación 1:1 entre dos listas paralelas: cada vez que se agrega o se elimina un grupo hay que actualizar las dos listas (grupos y contactos_por_grupo) en la misma posición.
- Aprendí que la integridad referencial se debe pensar antes de eliminar datos: antes de borrar un área o un contacto hay que revisar si otros datos dependen de él.
- Aprendí a usar una biblioteca externa (fpdf2) para generar archivos PDF, a revisar que esté instalada antes de usarla y a probarla primero en un archivo aparte antes de integrarla al programa.

3.3 Aspectos a Mejorar / Hacer Diferente en Próximos Proyectos

- Diseñar primero las funciones auxiliares y sus entradas y salidas antes de programar los menús.
- Llevar el registro de horas y de problemas desde el primer día.

- Probar cada funcionalidad con casos límite antes de pasar a la siguiente y hacer commit al terminarla.
- Empezar la investigación y el manual de usuario en paralelo con el desarrollo, no al final.

Estadística de Tiempos

El cuadro muestra las actividades realizadas y las horas invertidas en cada una, como base para estimar el esfuerzo de trabajos futuros.

Actividad realizada	**Período**	**Horas**
Análisis del problema	04/10/2026	3 horas
Diseño de algoritmos	04/10/2026, 18/10/2026	4 horas
Investigación: depuración y documentación	18/10/2026	3 horas
Investigación: expresiones regulares	21/10/2026	1 hora
Investigación: PDF	28/10/2026 29/10/2026	1 hora
Investigación: control de versiones	04/10/2026, 18/10/2026	3 horas
Programación: Registrar áreas	04/10/2026, 18/10/2026	10 horas
Programación: Configuración	18/10/2026	5 horas

Actividad realizada	Período	Horas
Programación: Registrar contactos	20/10/2026 21/10/2026 22/10/2026 23/10/2026 24/10/2026	10 horas
Programación: Grupos de contactos	24/10/2026 25/10/2026 26/10/2026	12 horas
Programación: Lista de contactos y PDF	27/10/2026 31/10/2026	6 horas
Documentación interna	04/10/2026, 18/10/2026, 20/10/2026, 21/10/2026 22/10/2026 23/10/2026 24/10/2026 25/10/2026	10 horas

Actividad realizada	Período	Horas
Pruebas y depuración	04/10/2026, 18/10/2026 21/10/2026 22/10/2026 23/10/2026 24/10/2026 25/10/2026 26/10/2026 27/10/2026 28/10/2026 31/10/2026	10 horas
Elaboración del manual de usuario	18/10/2026 22/10/2026 23/10/2026 24/10/2026 25/10/2026 26/10/2026 27/10/2026 28/10/2026 31/10/2026 01/11/2026	5 horas

Actividad realizada	**Período**	**Horas**
Elaboración de la documentación del proyecto	04/10/2026, 18/10/2026, 19/10/2026 21/10/2026 22/10/2026 23/10/2026 24/10/2026 25/10/2026 26/10/2026 27/10/2026 28/10/2026 29/10/2026 31/10/2026 01/11/2026	15 horas
Programación: Acerca de y Ayuda	28/10/2026 30/10/2026	3 horas
TOTAL		101 horas

Lista de Revisión y Análisis de Resultados

Avance: 100 = totalmente desarrollado (no hace falta análisis, salvo observaciones). Un porcentaje específico = desarrollo parcial; en el análisis se indica qué hace, qué falta y por qué no se completó. 0 = no desarrollado; se indica por qué.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Registrar áreas	10	100		Totalmente desarrollado. Permite agregar, consultar, modificar y eliminar áreas, que se guardan como tuplas (código, nombre) en la lista areas. Todas las operaciones validan los datos de entrada y se repiten hasta que se escribe C. Un área que está siendo usada por algún contacto no se puede eliminar (integridad referencial).
Agregar	3	100		Se implementó el registro de áreas mediante una lista de tuplas. Se validó que el código sea numérico, esté entre 1 y 999 y no se encuentre registrado. También se comprobó que el nombre tenga entre 1 y 40 caracteres y que el usuario pueda aceptar o cancelar el registro.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Consultar	1	100		Se implementó la búsqueda de áreas mediante su código. El programa muestra el código y el nombre cuando encuentra el área, e informa cuando no está registrada. También permite regresar al menú anterior.
Modificar	3	100		Se implementó la modificación del nombre de un área registrada, manteniendo su código original. Se validó la longitud del nuevo nombre y se comprobó que, al dejarlo vacío, se conserve el nombre anterior. La modificación requiere confirmación del usuario.
Eliminar	3	100		Impide eliminar áreas usadas por contactos (integridad referencial)
Configuración	5	100		Totalmente desarrollado. Guarda el área y el tipo de teléfono por omisión en la lista configuracion, validando que el área esté registrada y que el tipo sea M, C, T u O, y permite aceptar o cancelar. Observación: si no hay áreas registradas, el programa lo informa y regresa al menú; el enunciado no lo indica, pero se agregó para evitar un ciclo infinito pidiendo un área.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Registrar contactos	20	100		Totalmente desarrollado. Permite agregar, consultar, modificar y eliminar contactos, que se guardan como listas de nueve datos en la lista contactos y se identifican por la combinación de teléfono y área. Todos los datos se validan, y el área y el tipo de teléfono por omisión se usan cuando esos datos se dejan vacíos. Observación: un contacto que pertenece a algún grupo no se puede eliminar; el enunciado deja libre esta decisión y se optó por rechazar la eliminación con un mensaje.
Agregar	6	100		Totalmente desarrollado. Solicita los nueve datos en el orden del enunciado, valida cada uno, usa el área y el tipo por omisión cuando se deja vacío, rechaza un teléfono repetido con la misma área, permite aceptar o cancelar, y repite la ventana hasta que se escribe C en el teléfono. Se probó con valores válidos, inválidos y límite.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Consultar	2	100		Totalmente desarrollado. Solicita el teléfono y el área (si el área se deja vacía usa el área por omisión), rechaza un contacto no registrado con el mensaje "ESTE CONTACTO NO ESTA REGISTRADO, NO SE PUEDE CONSULTAR" y vuelve a pedir los datos. Si el contacto existe muestra sus nueve datos sin permitir modificarlos, solicita aceptar con A y vuelve a desplegar la ventana para más consultas. Con C en el teléfono regresa al menú anterior.
Modificar	6	100		Totalmente desarrollado. Solicita el teléfono y el área (o usa el área por omisión), rechaza un contacto no registrado con el mensaje del enunciado, muestra cada dato actual seguido de su línea de MODIFICACIÓN, conserva el valor anterior cuando se deja en blanco, valida el nombre, el correo, la dirección, la fecha, los pasatiempos y las notas, permite aceptar o cancelar (cancelar no cambia ningún dato) y repite la ventana hasta que se escribe C.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Eliminar	6	100		Totalmente desarrollado. Solicita el teléfono y el área (o usa el área por omisión), rechaza un contacto no registrado con el mensaje del enunciado y vuelve a pedir los datos. Si el contacto existe, muestra sus nueve datos y permite aceptar o cancelar. Al aceptar, comprueba que el contacto no pertenezca a ningún grupo; si pertenece, rechaza la eliminación con un mensaje y no la realiza, y si no pertenece, pide una confirmación (SI/NO) y elimina el contacto de la lista contactos. Repite la ventana hasta que se escribe C en el teléfono.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Administrar grupos de contactos	20	100		Totalmente desarrollado. Permite agregar grupos, agregar contactos a un grupo, modificar el nombre de un grupo, eliminar grupos y eliminar contactos de los grupos. Los nombres se guardan en la lista grupos y los contactos de cada grupo, como tuplas (teléfono, área), en contactos_por_grupo, en la misma posición (relación 1:1). Si el nuevo nombre de un grupo ya existe, sus contactos pasan al grupo existente sin repetirse. Eliminar un grupo o quitar un contacto de un grupo no elimina el contacto de la lista contactos.
Agregar grupos	2	100		Totalmente desarrollado. Solicita el nombre del grupo (de 1 a 40 caracteres), rechaza uno que ya está registrado con el mensaje del enunciado y vuelve a pedirlo, y permite aceptar o cancelar. Al aceptar agrega el nombre a la lista grupos y una lista vacía a contactos_por_grupo, para mantener la relación 1:1 entre ambas. Repite la ventana hasta que se escribe C.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Agregar contactos	4	100		Totalmente desarrollado. Solicita el nombre del grupo y rechaza uno que no existe con el mensaje del enunciado; luego solicita el teléfono y el área (con el área por omisión si se deja vacía) y rechaza un contacto que no existe con el mensaje del enunciado. Muestra el nombre del área y del contacto, permite aceptar o cancelar, y al aceptar agrega la tupla (teléfono, área) a la lista del grupo en contactos_por_grupo. Además, impide agregar un contacto repetido en un mismo grupo, y repite la ventana hasta que se escribe C.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Modificar grupos	6	100		Totalmente desarrollado. Solicita el nombre del grupo y rechaza uno que no está registrado con el mensaje del enunciado, volviendo a pedirlo. Solicita el nuevo nombre (de 1 a 40 caracteres; con Enter se conserva el actual) y, si ya existe en otro grupo, muestra el aviso del enunciado de que los contactos se le agregarán. Permite aceptar o cancelar. Al aceptar con un nombre nuevo, reemplaza el nombre en la lista grupos; si el nombre ya existe, pasa los contactos del grupo modificado al grupo existente sin repetirlos y elimina el grupo modificado de las listas grupos y contactos_por_grupo, manteniendo la relación 1:1. Repite la ventana hasta que se escribe C.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Eliminar grupos	5	100		Totalmente desarrollado. Solicita el nombre del grupo, rechaza un grupo no registrado con el mensaje del enunciado y vuelve a pedirlo. Permite aceptar o cancelar y pide una confirmación (SI/NO); al confirmar elimina el grupo de la lista grupos y su lista de contactos de contactos_por_grupo, usando la misma posición para mantener la relación 1:1. Los contactos registrados no se eliminan. Repite la ventana hasta que se escribe C.
Eliminar contactos de grupos	3	100		Totalmente desarrollado. Solicita el nombre del grupo, el teléfono y el área; rechaza un grupo inexistente y un contacto que no pertenece al grupo con los mensajes del enunciado y vuelve a pedir el dato. Muestra el nombre del área y del contacto, permite aceptar o cancelar y pide una confirmación (SI/NO). Al confirmar elimina la tupla del contacto de la lista del grupo en contactos_por_grupo, sin eliminar el contacto de la lista contactos ni de otros grupos. Repite la ventana hasta que se escribe C.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Lista de contactos	10	100		Totalmente desarrollado. Solicita los seis filtros (nombre, teléfono, área, fecha de nacimiento, pasatiempos y grupo), los aplica todos a la vez y permite aceptar o cancelar; al cancelar vuelve a mostrar la ventana con los filtros vacíos, y con C en el primer filtro regresa al menú. La emisión del listado en PDF se evalúa en su propio concepto.
Uso de comodín %	8	100		Totalmente desarrollado. Un filtro puede tener uno o varios % en cualquier posición (al inicio, al final, en el medio o en ambos lados) y cada % reemplaza cero o más caracteres. Se implementó con expresiones regulares en la función coincide_con_filtro(): se separa el filtro por los %, se protege cada parte con re.escape(), se unen con .* y se compara con re.fullmatch() y re.IGNORECASE. Funciona en los seis filtros.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
No usa comodines (%)	2	100		Totalmente desarrollado. Un filtro sin % solo encuentra los contactos cuyo dato es exactamente igual, sin distinguir mayúsculas de minúsculas, y un filtro vacío acepta cualquier valor. Se probó en el nombre, el área y el grupo.
Emitir listado en formato PDF	10	100		Totalmente desarrollado. Genera un archivo PDF en hoja horizontal con una tabla de las ocho columnas que pide el enunciado, solo con los contactos que cumplen los filtros. Usa la biblioteca fpdf2; si no está instalada, muestra un mensaje con el comando para instalarla. El archivo se guarda en la carpeta del programa con un número para no reemplazar listados anteriores.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Validación de datos de entrada	10	100		Totalmente desarrollado. Todos los datos se validan antes de guardarse y, si un dato no es válido, se muestra un mensaje y se vuelve a pedir: código de área (solo números, de 1 a 999), teléfono (de 5 a 12 dígitos), tipo de teléfono (M, C, T u O), nombre del contacto (de 1 a 50 caracteres), nombre del área y del grupo (de 1 a 40), correo con el formato parte1@parte2 sin espacios (expresión regular), dirección (máximo 80 caracteres), pasatiempos y notas (máximo 60), fecha dd/mm/aaaa que exista (módulo datetime, con el año 0000 permitido) y las opciones A/C y SI/NO. Los números se revisan antes de convertirlos, por lo que el programa no se detiene si se escribe una letra.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Validación de integridad referencial	10	100		Totalmente desarrollado. No se puede eliminar un área que usa algún contacto ni un contacto que pertenece a algún grupo; en ambos casos se muestra un mensaje y no se elimina. Solo se pueden agregar a un grupo contactos que estén registrados, y el área de un contacto debe estar registrada. Las listas grupos y contactos_por_grupo se actualizan siempre juntas, en la misma posición (relación 1:1), al agregar, modificar o eliminar grupos, y al eliminar un grupo sus contactos siguen registrados. Observación: al eliminar un área no se revisa si es el área por omisión de la configuración; si se elimina, al agregar un contacto con el área vacía el programa indica que el área no está registrada y pide otra.

Concepto	Puntos	Avance	Obtenidos	Análisis de resultados
Ayuda: manual de usuario en pantalla	5	100		Totalmente desarrollado. Abre el manual de usuario en PDF (manual_de_usuario_contactos.PDF) con el visor predeterminado del computador. Si el archivo no está en la carpeta del programa, muestra un mensaje de error y regresa al menú. Observación: la apertura usa os.startfile(), que funciona en Windows.
TOTAL	100	100		

Pruebas de funcionamiento de la función existe_contacto().

Se realizaron cuatro pruebas para comprobar que la función identifica correctamente los contactos mediante la combinación del número de teléfono y el código de área.

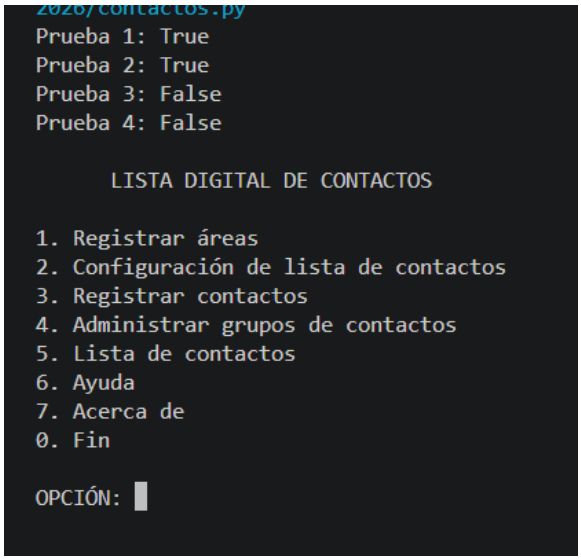


Figura 10. Pruebas de funcionamiento de la función existe_contacto()

Resultado: 1 y 2 devolvieron True porque los contactos estaban registrados. Las pruebas 3 y 4 devolvieron False porque no se encontró una coincidencia exacta entre el teléfono y el área.

Pruebas de funcionamiento de buscar_contacto()

Se realizaron cuatro pruebas para comprobar que la función devuelve la posición del contacto dentro de la lista, o -1 cuando no está registrado, usando la combinación del número de teléfono y el código de área.

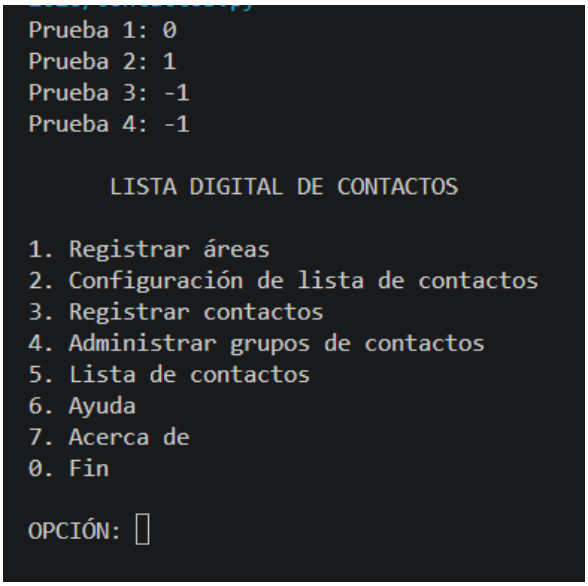


Figura 11. Pruebas de funcionamiento de la función buscar_contacto()

Resultado: la prueba 1 devolvió 0 y la prueba 2 devolvió 1, es decir, las posiciones de los contactos registrados en la lista. Las pruebas 3 y 4 devolvieron -1 porque no se encontró una coincidencia exacta entre el teléfono y el área. Los resultados fueron los esperados.

Pruebas de validación y valores por omisión en Agregar contactos

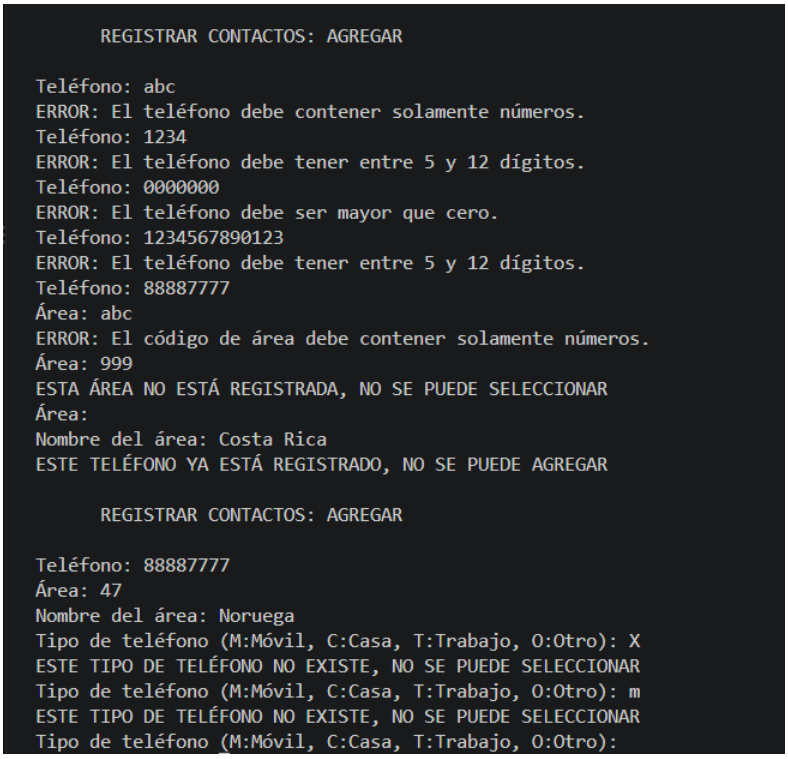


Figura 12. Validación de teléfono, área, tipo y duplicado en Agregar contactos

```
      REGISTRAR CONTACTOS: AGREGAR

Teléfono: 88887778
Área: 506
Nombre del área: Costa Rica
Tipo de teléfono (M:Móvil, C:Casa, T:Trabajo, O:Otro):
Nombre contacto: 12345678901234567890123456789012345678901234567890123456789012345678901
ERROR: El nombre debe tener entre 1 y 50 caracteres.
Nombre contacto:
ERROR: El nombre debe tener entre 1 y 50 caracteres.
Nombre contacto: 12345678901234567890123456789012345678901234567890
Correo electrónico:
ERROR: El correo debe tener el formato parte1@parte2, sin espacios.
Correo electrónico: ana@@gmail.com
ERROR: El correo debe tener el formato parte1@parte2, sin espacios.
Correo electrónico: ana@
ERROR: El correo debe tener el formato parte1@parte2, sin espacios.
Correo electrónico: @gmail.com
ERROR: El correo debe tener el formato parte1@parte2, sin espacios.
Correo electrónico: ana correo@gmail.com
ERROR: El correo debe tener el formato parte1@parte2, sin espacios.
Correo electrónico: ana@gmail.com
ERROR: El correo debe tener el formato parte1@parte2, sin espacios.
Correo electrónico: a@b
Dirección física:
```

Figura 13. Validación de nombre y correo en Agregar contactos

```
      REGISTRAR CONTACTOS: AGREGAR

Teléfono: 88887778
Área: 506
Nombre del área: Costa Rica
Tipo de teléfono (M:Móvil, C:Casa, T:Trabajo, O:Otro):
Nombre contacto: Ana
Correo electrónico: a@b
Dirección física: 12345678901234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901
ERROR: La dirección debe tener máximo 80 caracteres.
Dirección física: 12345678901234567890123456789012345678901234567890123456789012345678901234567890
Fecha de nacimiento (dd/mm/aaaa): 29/02/2023
ERROR: La fecha no es válida.
Fecha de nacimiento (dd/mm/aaaa): 31/04/2024
ERROR: La fecha no es válida.
Fecha de nacimiento (dd/mm/aaaa): 15/13/2000
ERROR: La fecha no es válida.
Fecha de nacimiento (dd/mm/aaaa): 00/01/2000
ERROR: La fecha no es válida.
Fecha de nacimiento (dd/mm/aaaa): 1/1/2000
ERROR: La fecha debe tener el formato dd/mm/aaaa.
Fecha de nacimiento (dd/mm/aaaa): 30/01/01
ERROR: La fecha debe tener el formato dd/mm/aaaa.
Fecha de nacimiento (dd/mm/aaaa): 2000-01-01
ERROR: La fecha debe tener el formato dd/mm/aaaa.
Fecha de nacimiento (dd/mm/aaaa): abc
ERROR: La fecha debe tener el formato dd/mm/aaaa.
Fecha de nacimiento (dd/mm/aaaa): 30/01/2001
Pasatiempos: 12345678901234567890123456789012345678901234567890123456789012345678901
ERROR: Los pasatiempos deben tener máximo 60 caracteres.
Pasatiempos: 123456789012345678901234567890123456789012345678901234567890
Notas: 12345678901234567890123456789012345678901234567890123456789012345678901234567890
¿DESEA REGISTRAR EL CONTACTO? A-ACEPTAR C-CANCELAR:
```

Figura 14. Validación de dirección, fecha, pasatiempos y notas en Agregar contactos

Se probaron valores inválidos y válidos en cada campo: teléfono (letras, longitud incorrecta y ceros), área (letras y no registrada), tipo (X y minúscula), nombre (vacío y de 51 caracteres), correo (vacío, dos "@", sin parte final, sin parte inicial y con espacios), dirección (81 caracteres), fecha (fechas inexistentes y formatos incorrectos) y pasatiempos y notas (61 caracteres). En todos los casos el programa mostró el mensaje de error y volvió a pedir el dato, y los valores en el límite (50, 80 y 60 caracteres) fueron aceptados. Con el área vacía se usó el área por omisión, y el mismo teléfono con la misma área fue rechazado como duplicado, mientras que con otra área fue aceptado.

Pruebas de fecha de nacimiento, confirmación y guardado en Agregar contactos

```
     REGISTRAR CONTACTOS: AGREGAR

Teléfono: 88887778
Área: 506
Nombre del área: Costa Rica
Tipo de teléfono (M:Móvil, C:Casa, T:Trabajo, O:Otro): C
Nombre contacto: Ana
Correo electrónico: a@b
Dirección física:
Fecha de nacimiento (dd/mm/aaaa): 30/02/0000
ERROR: La fecha no es válida.
Fecha de nacimiento (dd/mm/aaaa): 29/02/0000
Pasatiempos:
Notas:
¿DESEA REGISTRAR EL CONTACTO? A-ACEPTAR C-CANCELAR: A
[[88887777, 506, 'M', 'María', 'maria@correo.com', '', 0, '', ''], [88887778, 506, 'C', 'Ana', 'a@b', '', '29/02/0000', '', '']]
Contacto registrado correctamente.
```

Figura 15. Fecha con año 0000 y guardado del contacto

```
      REGISTRAR CONTACTOS: AGREGAR

Teléfono: 88887777
Área: 47
Nombre del área: Noruega
Tipo de teléfono (M:Móvil, C:Casa, T:Trabajo, O:Otro): X
ESTE TIPO DE TELÉFONO NO EXISTE, NO SE PUEDE SELECCIONAR
Tipo de teléfono (M:Móvil, C:Casa, T:Trabajo, O:Otro): m
ESTE TIPO DE TELÉFONO NO EXISTE, NO SE PUEDE SELECCIONAR
Tipo de teléfono (M:Móvil, C:Casa, T:Trabajo, O:Otro):
Nombre contacto: Amy
Correo electrónico: amy@m
Dirección física:
Fecha de nacimiento (dd/mm/aaaa):
Pasatiempos:
Notas:
¿DESEA REGISTRAR EL CONTACTO? A-ACEPTAR C-CANCELAR: A
[[88887777, 506, 'M', 'María', 'maria@correo.com', '', 0, '', ''], [88887777, 47, 'M', 'Amy', 'amy@m', '', 0, '', '']]
Contacto registrado correctamente.

      REGISTRAR CONTACTOS: AGREGAR

Teléfono: C

      REGISTRAR CONTACTOS

1. Agregar contactos
2. Consultar contactos
3. Modificar contactos
4. Eliminar contactos
0. Fin

OPCIÓN:
```

Figura 16. Fecha vacía, tipo por omisión, repetición de la ventana y regreso al menú

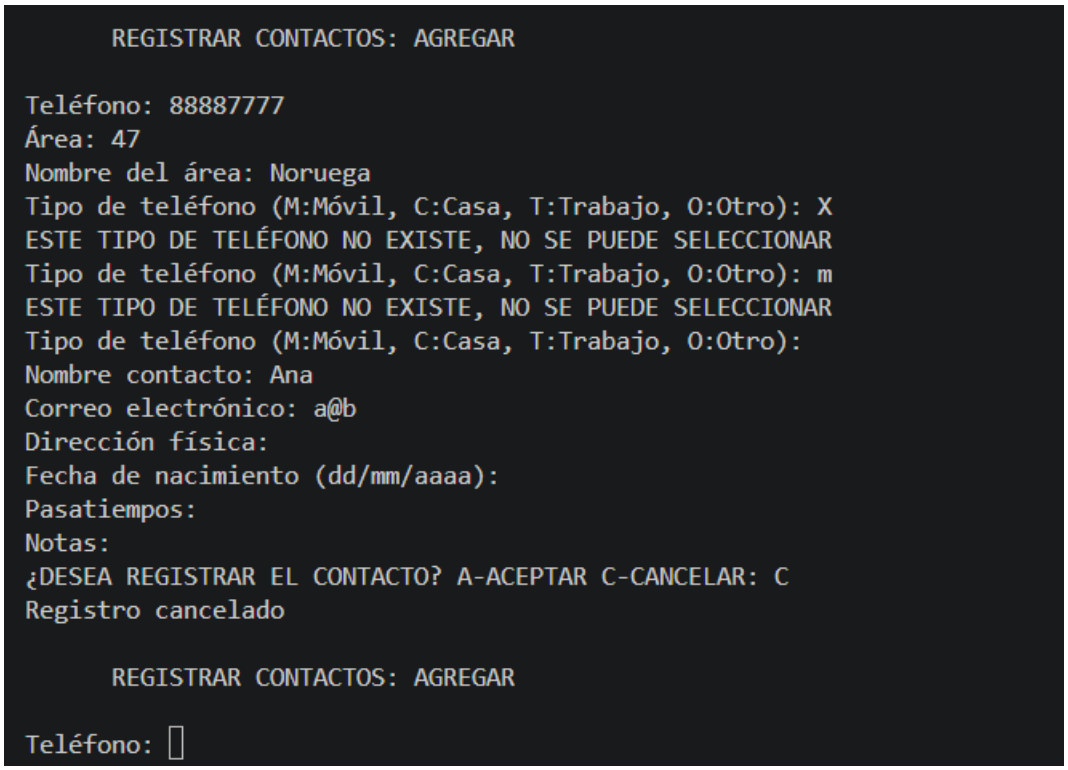


Figura 17. Cancelación del registro

Se comprobó que 30/02/0000 es rechazada y que 29/02/0000 es aceptada, porque el año 0000 significa año desconocido y solo se revisan el día y el mes (con un año bisiesto de prueba, ya que datetime no admite el año 0); la fecha se guardó como texto “29/02/0000”. Una fecha vacía se guardó como el número 0 y un tipo vacío se guardó con el tipo por omisión M. Con la opción A el contacto se agregó a la lista contactos y se mostró “Contacto registrado correctamente.”, y la ventana de Agregar se repitió; con C en la confirmación se mostró “Registro cancelado” y la ventana también se repitió. Con C en el campo teléfono se regresó al menú de Registrar contactos.

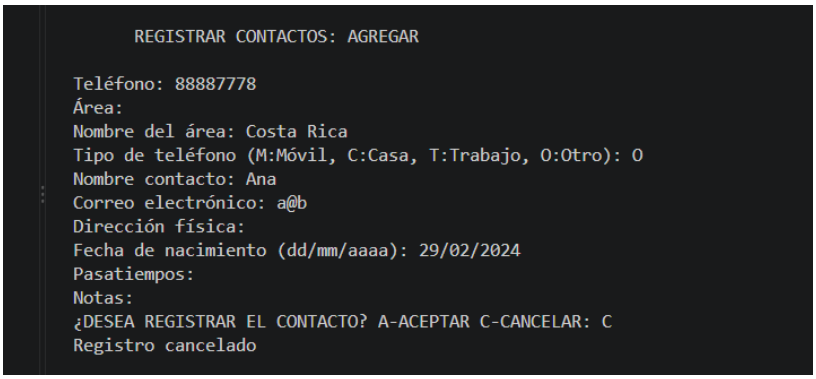


Figura 18. Prueba de año bisiesto, tipo O y cancelación

Eliminar áreas: integridad referencial

Se probó la función eliminar_area() con dos áreas registradas: la 506 (Costa Rica), que tenía un contacto registrado, y la 47, que no tenía ningún contacto. Estos datos se cargaron solo para la prueba. Primero se intentó eliminar el área 506, y después el área 47.

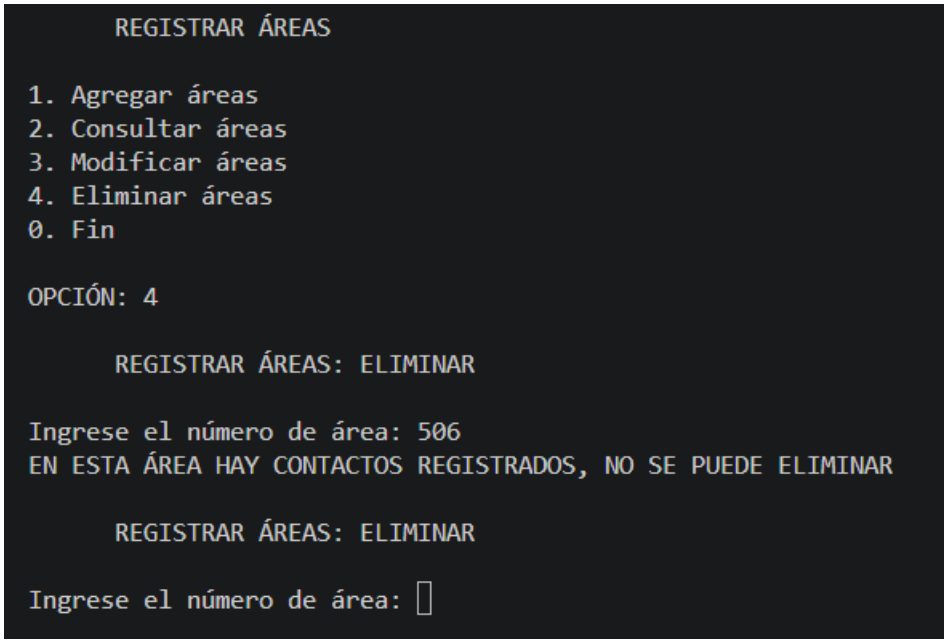


Figura 19. Intento de eliminar el área 506, que tiene contactos registrados: se rechaza y se vuelve a pedir el número de área.

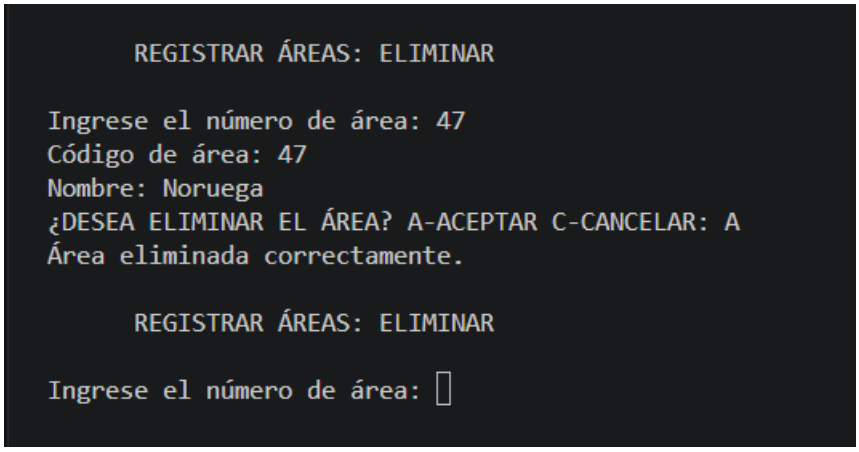


Figura 20. Salió Eliminación del área 47, que no tiene contactos: se muestra "Área eliminada correctamente" y se vuelve a pedir el número de área.

Resultado: la prueba coincidió con lo esperado. Al intentar eliminar el área 506 se mostró "EN ESTA ÁREA HAY CONTACTOS REGISTRADOS, NO SE PUEDE ELIMINAR" y se volvió a pedir el número de área, sin eliminar el área. Al eliminar el área 47, que no tenía contactos, se mostró "Área eliminada correctamente" y se volvió a pedir el número de área.

Pruebas de funcionamiento de Consultar contactos

Se probó la función consultar_contacto() con dos contactos registrados en el área 506: María, con la fecha de nacimiento vacía, y Ana, con la fecha 15/06/1990. Estos datos se cargaron temporalmente solo para las pruebas.

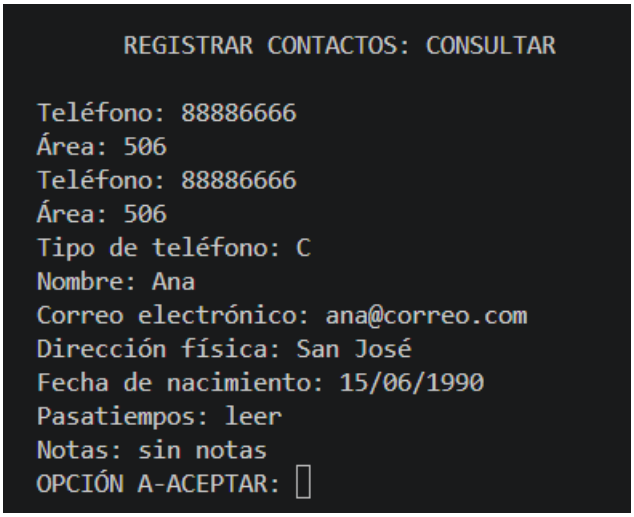


Figura 21. Consulta de un contacto registrado

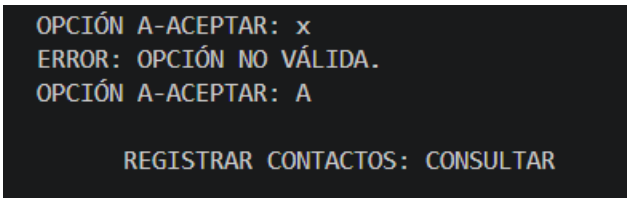


Figura 22. Opción no válida al aceptar y repetición de la ventana

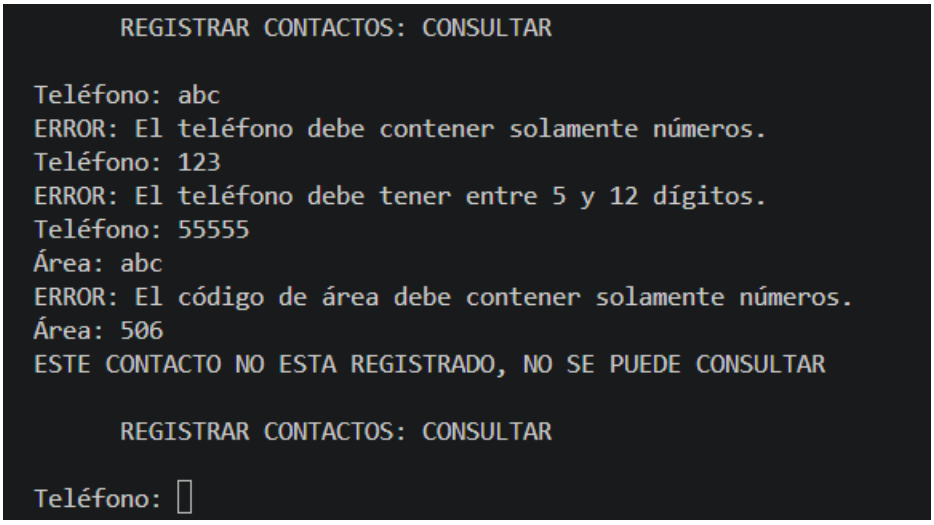


Figura 23. Validación de teléfono y área, y contacto no registrado

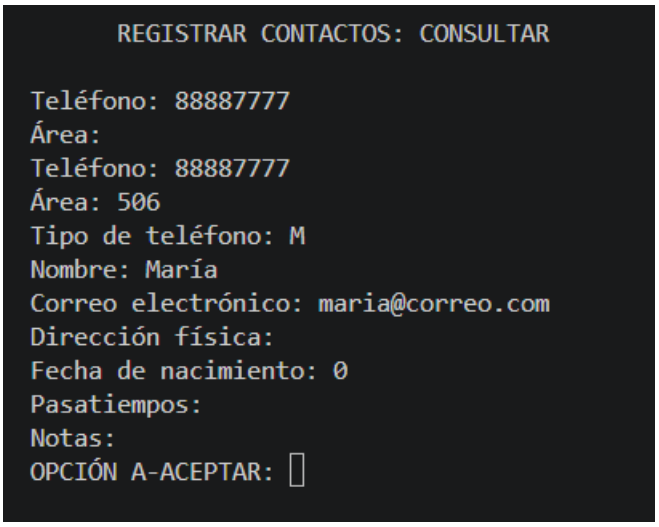


Figura 24. Consulta con área por omisión y fecha vacía

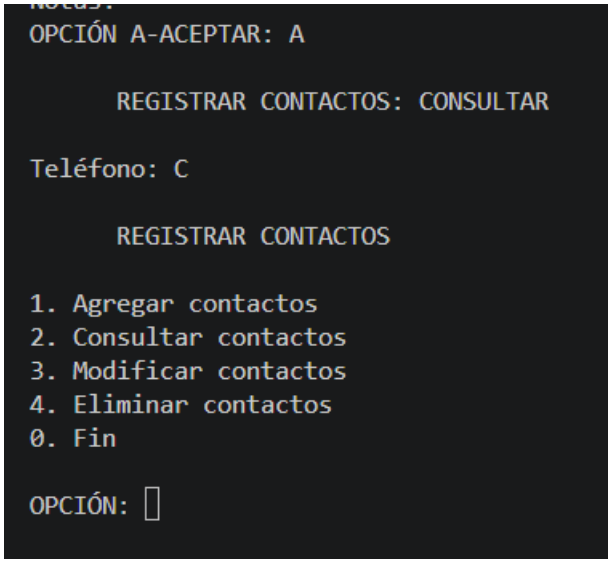


Figura 25. Regreso al menú con C

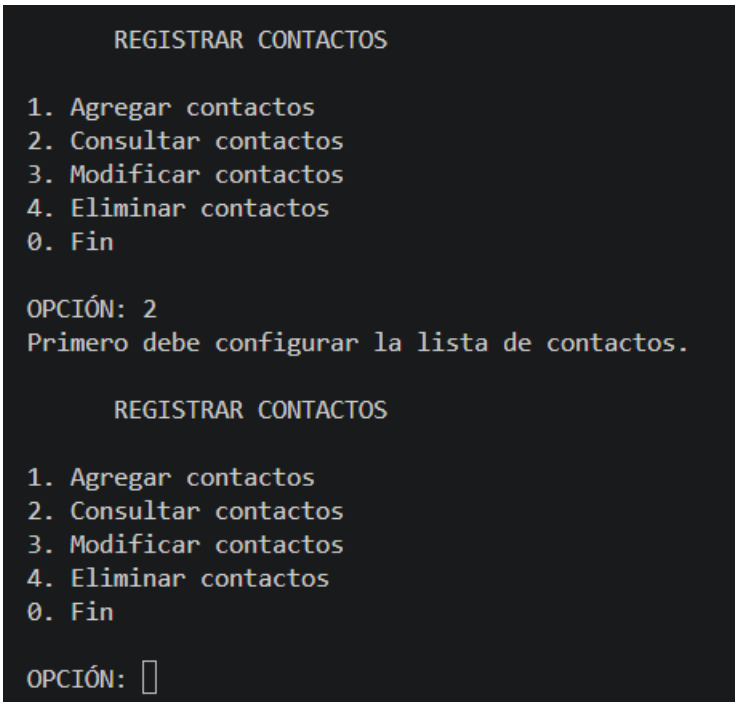


Figura 26. Aviso al consultar sin configuración guardada

Resultado: todas las pruebas coincidieron con lo esperado. Al consultar un contacto registrado (Ana) se mostraron sus nueve datos y el programa esperó la confirmación A; al escribir una opción distinta de A se mostró "ERROR: OPCIÓN NO VÁLIDA." y, al aceptar, se repitió la ventana. Los datos inválidos de teléfono (letras y cantidad de dígitos incorrecta) y de área (letras) mostraron su mensaje de error, y un teléfono no registrado mostró "ESTE CONTACTO NO ESTA REGISTRADO, NO SE PUEDE CONSULTAR" y volvió a pedir teléfono y área. Con el área vacía se utilizó el área por omisión (506) y el contacto sin fecha mostró el valor 0. Al escribir C en el teléfono se regresó al menú de Registrar contactos, y sin configuración guardada se mostró el aviso "Primero debe configurar la lista de contactos." y se regresó al menú.

Pruebas de funcionamiento de Eliminar contactos

Se probó la función eliminar_contacto() con tres contactos registrados en el área 506 (María, Ana y Luis), cargados temporalmente solo para las pruebas. Se eliminó el contacto Luis para comprobar la cancelación y la eliminación.

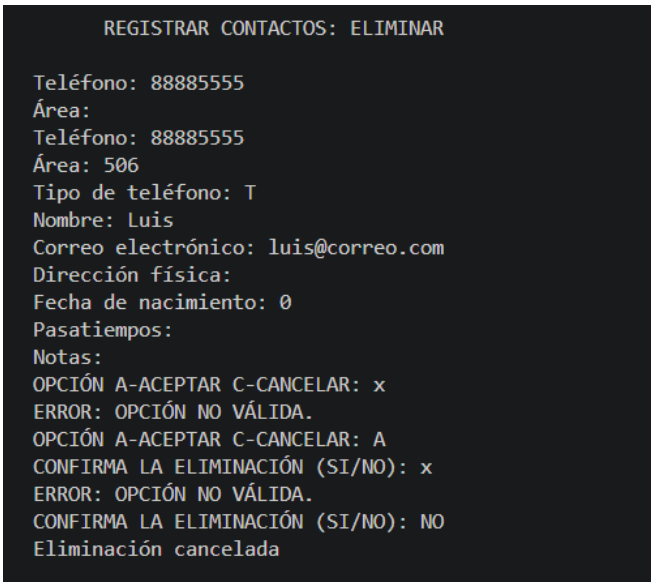


Figura 27. Opción no válida y cancelación con NO al eliminar un contacto

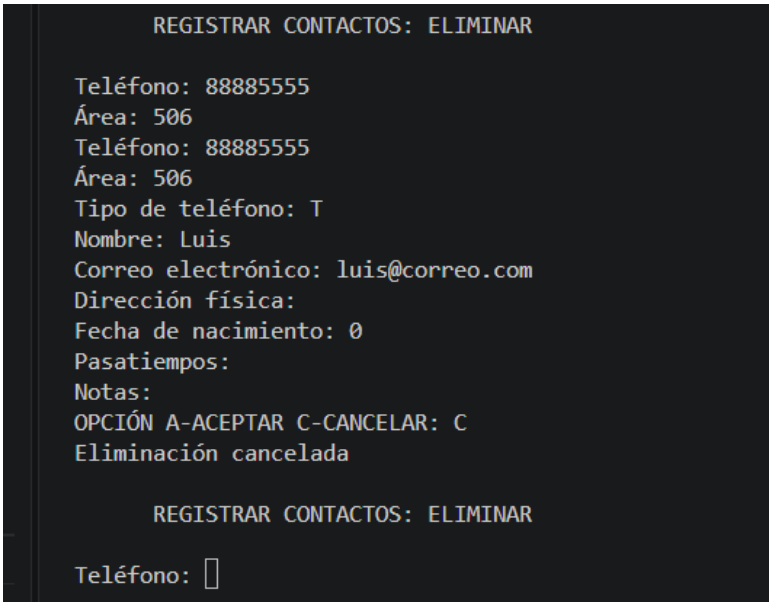


Figura 28. Cancelación con C al eliminar un contacto

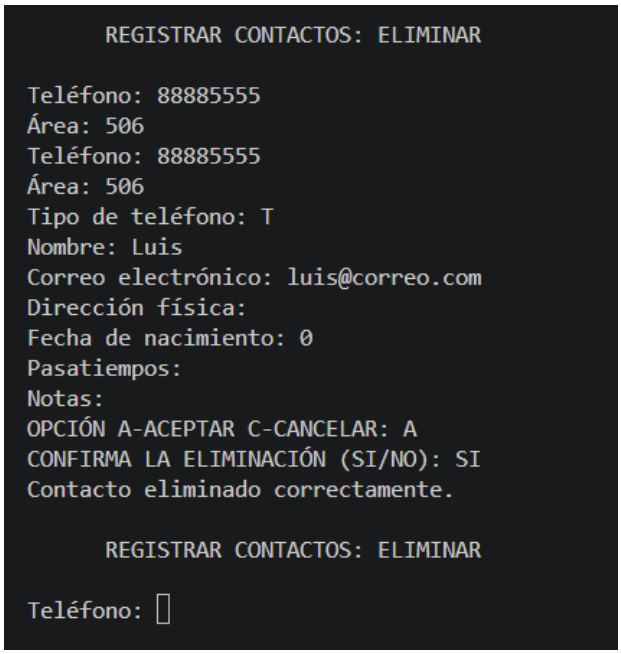


Figura 29. Eliminación de un contacto confirmada

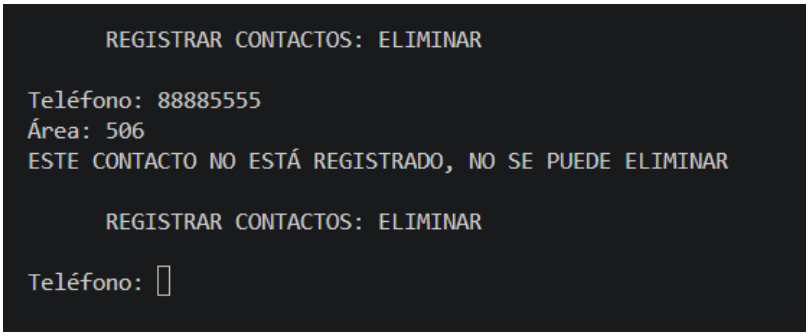


Figura 30. Intento de eliminar un contacto que ya no existe

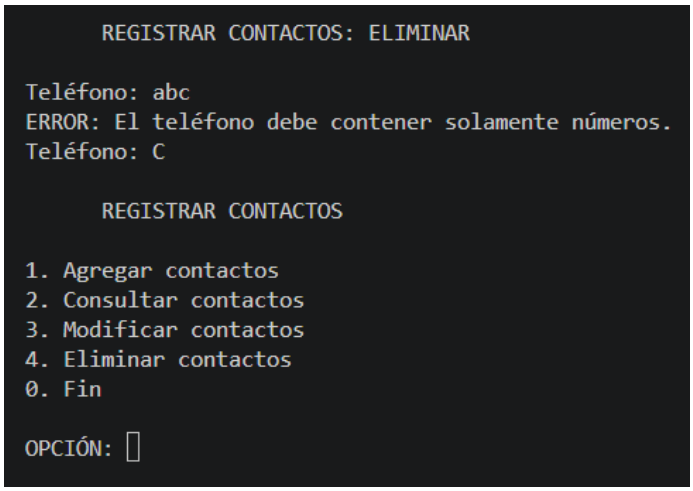


Figura 31. Teléfono inválido y regreso al menú con C

Resultado: todas las pruebas coincidieron con lo esperado. Al escribir una opción distinta de A o C, y una confirmación distinta de SI o NO, se mostró "ERROR: OPCIÓN NO VÁLIDA." y se volvió a pedir el dato. Con la confirmación NO, y con C en la opción, se mostró "Eliminación cancelada" y el contacto siguió registrado. Con la confirmación SI se mostró "Contacto eliminado correctamente.", y al intentar eliminarlo de nuevo se mostró "ESTE CONTACTO NO ESTÁ REGISTRADO, NO SE PUEDE ELIMINAR" y se volvieron a pedir el teléfono y el área. Con un teléfono inválido se mostró el mensaje de error, y con C se regresó al menú de Registrar contactos.

Pruebas de funcionamiento de Modificar contactos

Se probó la función modificar_contacto() con dos contactos registrados en el área 506: María, con la fecha de nacimiento vacía, y Ana, con la fecha 15/06/1990. Estos datos se cargaron temporalmente solo para las pruebas. Para comprobar los resultados se utilizó la opción Consultar contactos.



Figura 32. Modificación aceptada



Figura 33. Verificación con Consultar

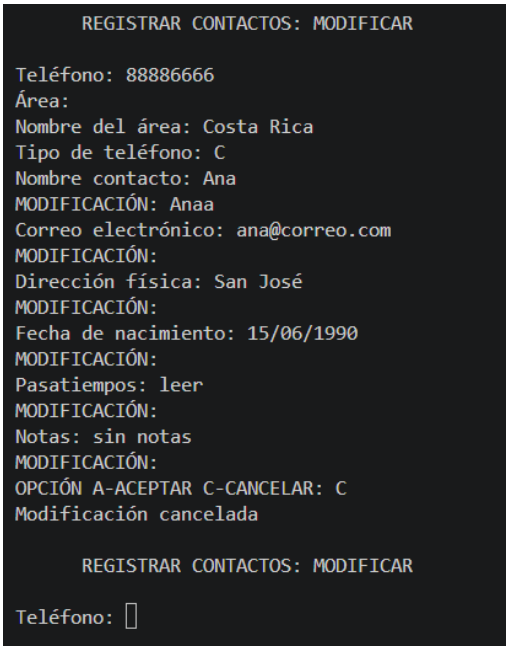


Figura 34. Cancelación

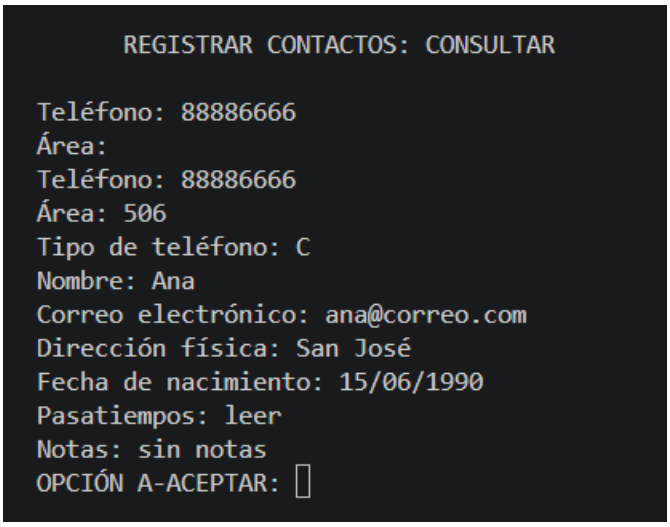


Figura 35. Los datos no cambiaron

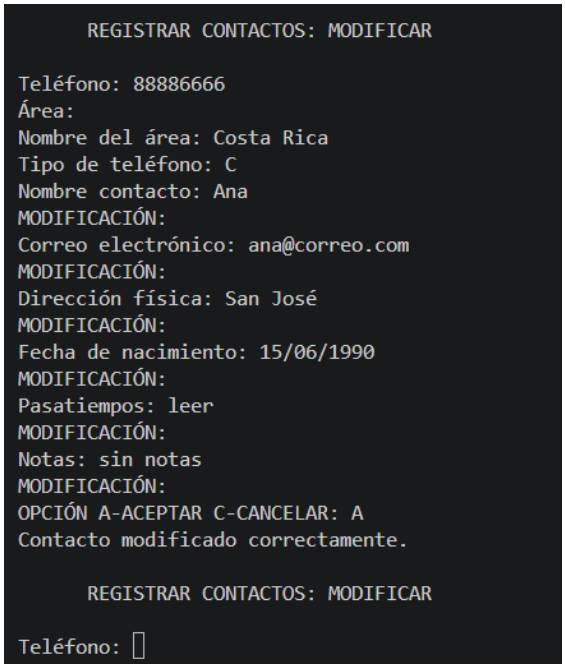


Figura 36. Todo vacío conserva los valores

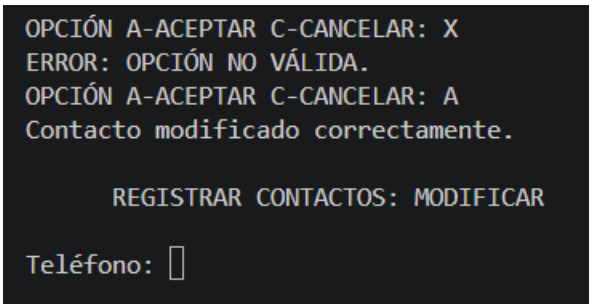


Figura 37. Opción no válida

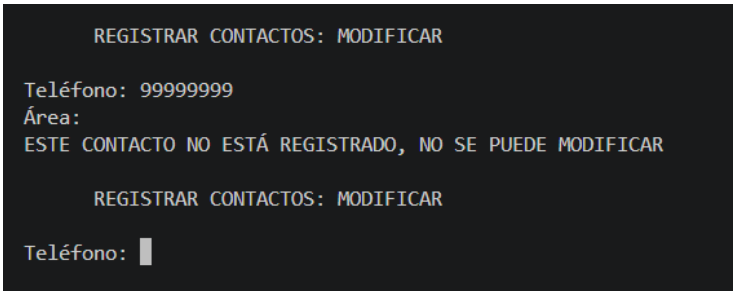


Figura 38. Contacto no registrado

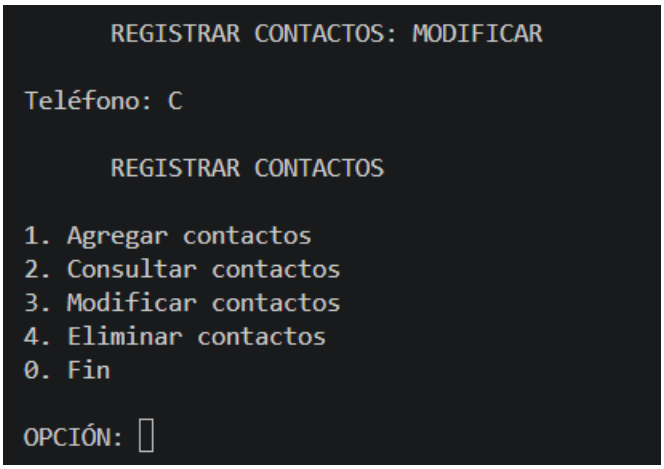


Figura 39. Regreso con C

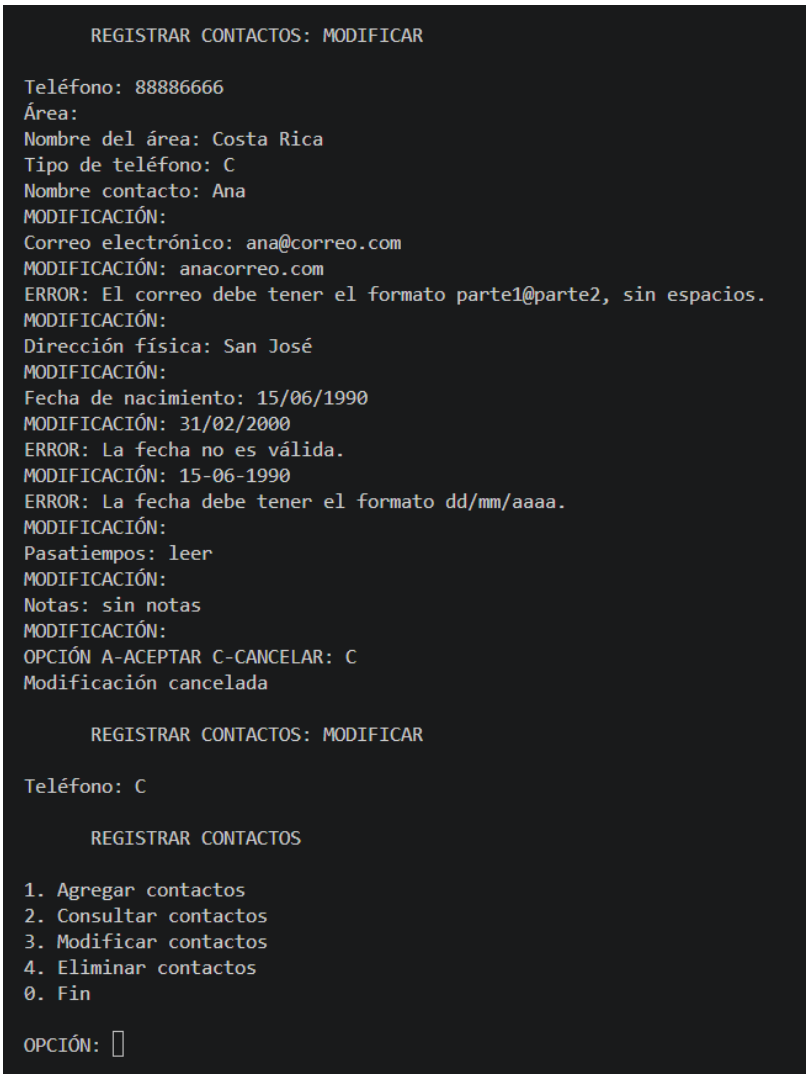


Figura 40. Validaciones al modificar: correo y fecha

Resultado: todas las pruebas coincidieron con lo esperado. Al modificar varios datos y aceptar con A se mostró “Contacto modificado correctamente.”, y al consultar el contacto se vieron los nuevos valores, mientras que los datos que se dejaron en blanco conservaron su valor anterior. Con la opción C se mostró “Modificación cancelada” y el contacto quedó sin cambios. Con todos los datos en blanco y la opción A no se modificó nada. Una opción distinta de A o C mostró “ERROR: OPCIÓN NO VÁLIDA.” y se volvió a preguntar. Un teléfono no registrado mostró “ESTE CONTACTO NO ESTÁ REGISTRADO, NO SE PUEDE MODIFICAR” y se volvieron a pedir el teléfono y el área. Un correo sin arroba y las fechas 31/02/2000 y 15-06-1990 fueron rechazadas con su mensaje de error y se volvió a pedir el dato. Con C en el teléfono se regresó al menú de Registrar contactos.

Pruebas de funcionamiento de Agregar grupos

Se probó la función agregar_grupo(), que utiliza la función auxiliar buscar_grupo() para comprobar si un nombre ya está registrado. Se agregaron los grupos “COMPAS DEL COLEGIO”, “BARRIO” y un grupo con un nombre de 40 caracteres. Para comprobar el contenido de las listas grupos y contactos_por_grupo se mostró temporalmente su valor al finalizar el programa; ese código era solo para la prueba y no forma parte del programa.

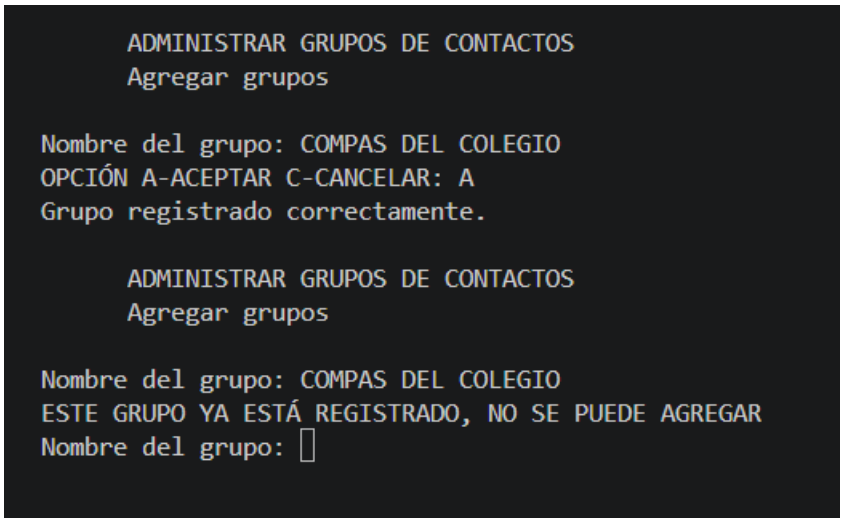


Figura 41. Grupo agregado y rechazo de un nombre repetido

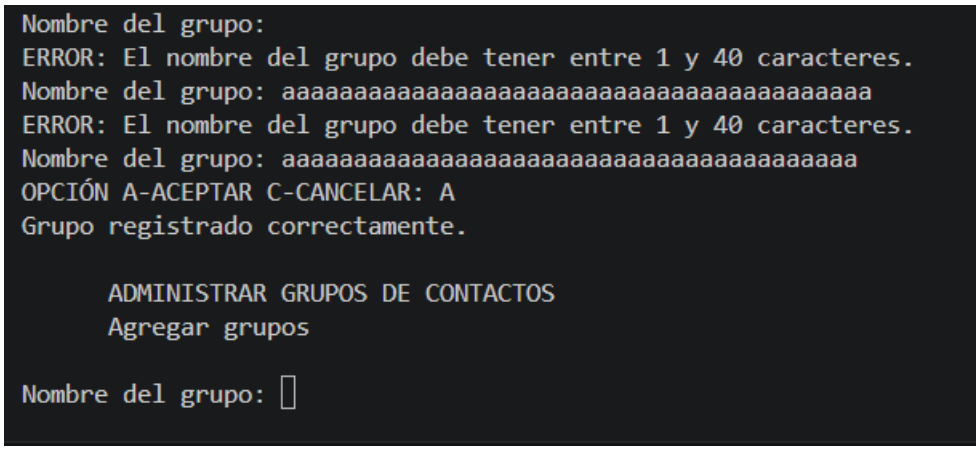


Figura 42. Validación de la longitud del nombre del grupo (vacío, 41 y 40 caracteres)

```
      ADMINISTRAR GRUPOS DE CONTACTOS
      Agregar grupos

Nombre del grupo: BARRIO
OPCIÓN A-ACEPTAR C-CANCELAR: X
ERROR: OPCIÓN NO VÁLIDA.
OPCIÓN A-ACEPTAR C-CANCELAR: C
Registro cancelado

      ADMINISTRAR GRUPOS DE CONTACTOS
      Agregar grupos

Nombre del grupo:
```

Figura 43. Opción no válida y cancelación al agregar un grupo

```
       ADMINISTRAR GRUPOS DE CONTACTOS
       Agregar grupos

  Nombre del grupo: BARRIO
  OPCIÓN A-ACEPTAR C-CANCELAR: A
  Grupo registrado correctamente.

       ADMINISTRAR GRUPOS DE CONTACTOS
       Agregar grupos

  Nombre del grupo: C

       ADMINISTRAR GRUPOS DE CONTACTOS

  1. Agregar grupos
  2. Agregar contactos a los grupos
  3. Modificar grupos
  4. Eliminar grupos
  5. Eliminar contactos de los grupos
  0. Fin
```

Figura 44. Grupo agregado y regreso al menú con C

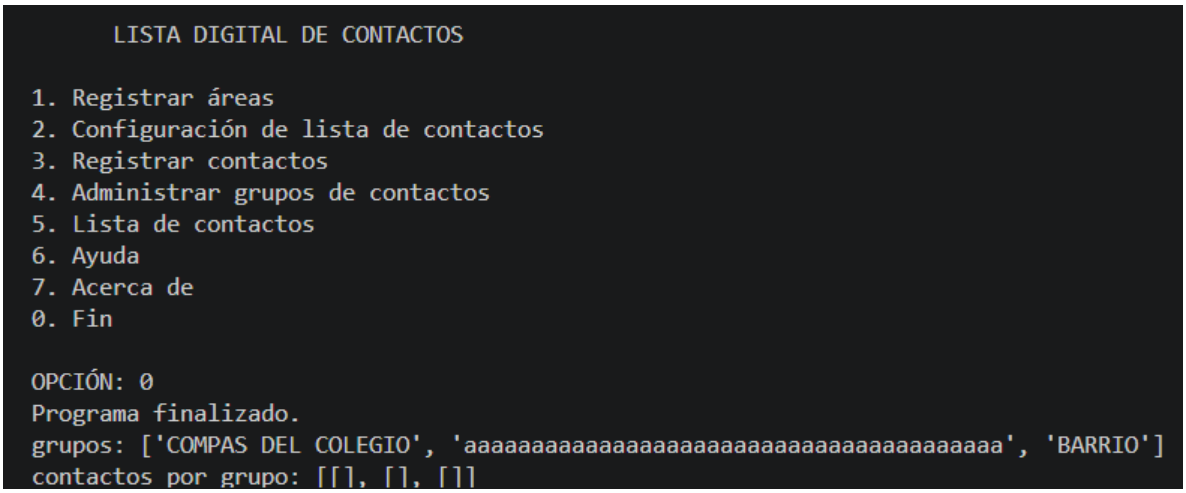


Figura 45. Contenido de las listas grupos y contactos_por_grupo al finalizar el programa

Resultado: todas las pruebas coincidieron con lo esperado. Al agregar un grupo y aceptar con A se mostró "Grupo registrado correctamente.", y al intentar agregar de nuevo "COMPAS DEL COLEGIO" se mostró "ESTE GRUPO YA ESTÁ REGISTRADO, NO SE PUEDE AGREGAR" y se volvió a pedir el nombre. Un nombre vacío y uno de 41 caracteres mostraron el mensaje de error de longitud, y uno de 40 caracteres fue aceptado. Una opción distinta de A o C mostró "ERROR: OPCIÓN NO VÁLIDA." y se volvió a preguntar; con C se mostró "Registro cancelado" y el grupo no se agregó. Con C en el nombre se regresó al menú de grupos. Al finalizar, las listas grupos y contactos_por_grupo tenían la misma cantidad de elementos (tres grupos y tres listas vacías), lo que confirma la relación 1:1.

Pruebas de funcionamiento de Agregar contactos a los grupos

Se probó la función agregar_contacto_a_grupo(), que utiliza las funciones auxiliares buscar_grupo(), buscar_contacto() y buscar_contacto_en_grupo(). Se cargaron temporalmente solo para las pruebas dos contactos en el área 506 (María y Ana) y dos grupos ("COMPAS DEL COLEGIO" y "BARRIO"). Para comprobar el contenido de las listas grupos y contactos_por_grupo se mostró temporalmente su valor al finalizar el programa; ese código era solo para la prueba y no forma parte del programa.

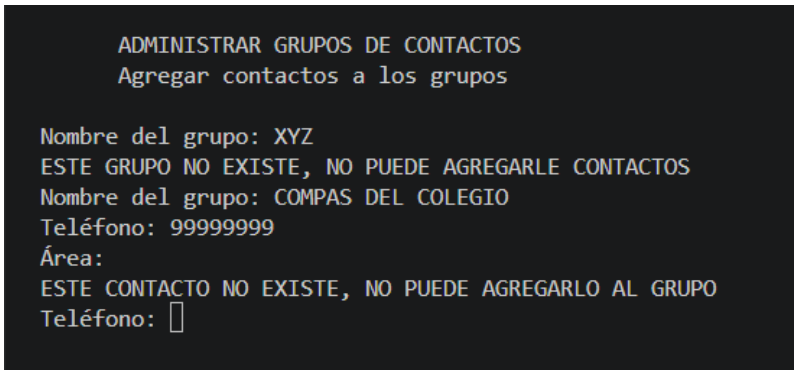


Figura 46. Grupo y contacto que no existen al agregar contactos a un grupo

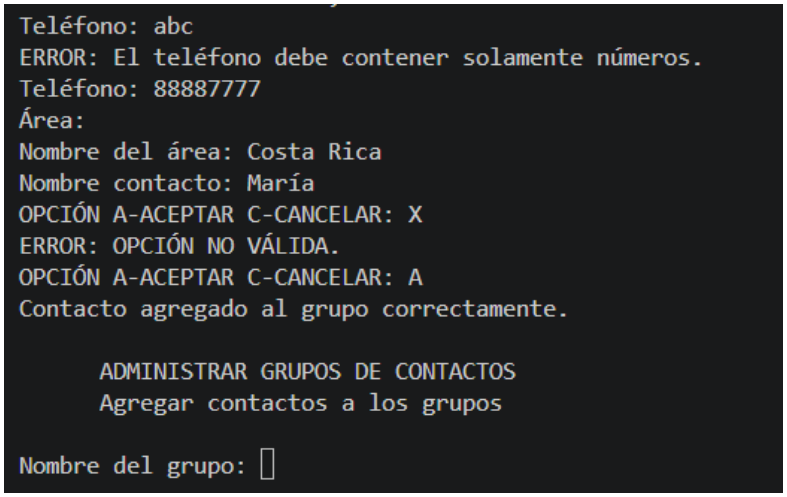


Figura 47. Teléfono inválido, opción no válida y contacto agregado al grupo

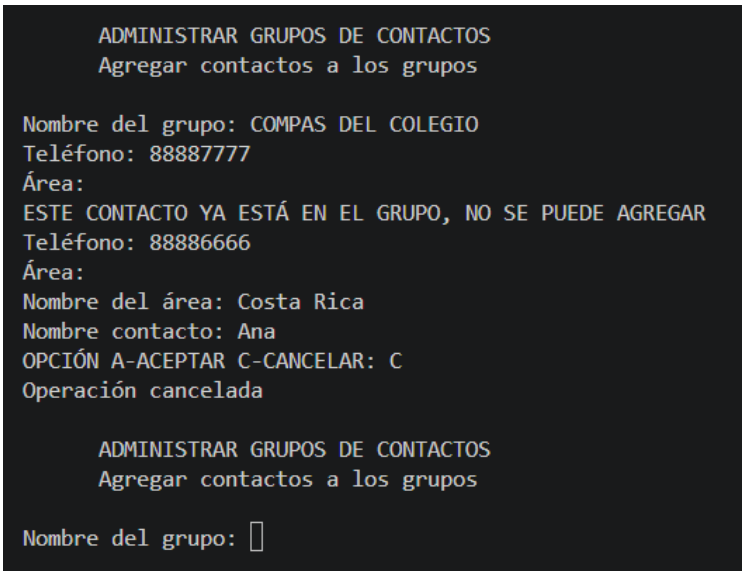


Figura 48. Contacto repetido en el grupo y cancelación de la operación

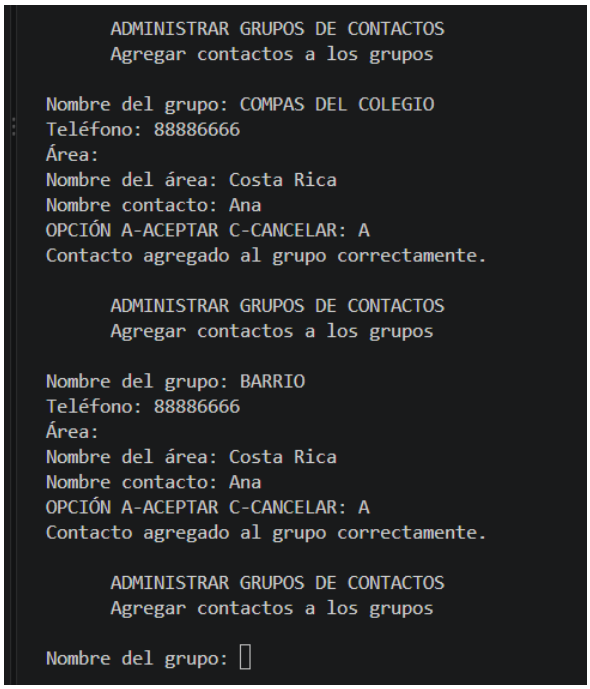


Figura 49. Un mismo contacto agregado a dos grupos distintos

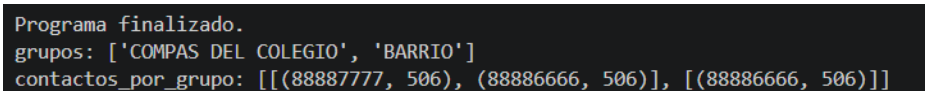


Figura 50. Contenido de las listas grupos y contactos_por_grupo al finalizar el programa

Resultado: todas las pruebas coincidieron con lo esperado. Un grupo que no existe mostró "ESTE GRUPO NO EXISTE, NO PUEDE AGREGARLE CONTACTOS" y se volvió a pedir el nombre, y un contacto que no existe mostró "ESTE CONTACTO NO EXISTE, NO PUEDE AGREGARLO AL GRUPO" y se volvieron a pedir el teléfono y el área. Un teléfono con letras mostró el mensaje de error. Con un contacto existente se mostraron el nombre del área y el nombre del contacto; una opción distinta de A o C mostró "ERROR: OPCIÓN NO VÁLIDA." y, con A, se mostró "Contacto agregado al grupo correctamente.". Al intentar agregar de nuevo a un contacto que ya estaba en el grupo se mostró "ESTE CONTACTO YA ESTÁ EN EL GRUPO, NO SE PUEDE AGREGAR", y con C se mostró "Operación cancelada" sin agregar el contacto. Un mismo contacto se pudo agregar a dos grupos distintos. Al finalizar, las listas mostraron las tuplas (teléfono, área) en la posición de cada grupo.

Pruebas de funcionamiento de Modificar grupos

Se probó la función modificar_grupo(), que utiliza las funciones auxiliares buscar_grupo() y buscar_contacto_en_grupo(). Se cargaron temporalmente solo para las pruebas tres contactos en el área 506 (María, Ana y Luis) y tres grupos: “COMPAS DEL COLEGIO” (con María y Ana), “BARRIO” (con Ana y Luis) y “AMIGOS” (sin contactos). Para comprobar el contenido de las listas grupos y contactos_por_grupo se mostró temporalmente su valor antes de iniciar las pruebas y al finalizarlas. Todas las pruebas se hicieron en una sola ejecución. El código temporal era solo para las pruebas y no forma parte del programa.

```
ANTES grupos: ['COMPAS DEL COLEGIO', 'BARRIO', 'AMIGOS']
ANTES contactos_por_grupo: [[(88887777, 506), (88886666, 506)], [(88886666, 506), (88885555, 506)], []]
```

Figura 51. Contenido inicial de las listas grupos y contactos_por_grupo

```
      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo: XYZ
GRUPO NO ESTÁ REGISTRADO, NO SE PUEDE MODIFICAR
Nombre del grupo: AMIGOS
MODIFICACIÓN: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
ERROR: El nombre del grupo debe tener entre 1 y 40 caracteres.
MODIFICACIÓN:
OPCIÓN A-ACEPTAR C-CANCELAR: X
ERROR: OPCIÓN NO VÁLIDA.
OPCIÓN A-ACEPTAR C-CANCELAR: A
Grupo modificado correctamente.

      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo:
```

Figura 52. Grupo no registrado, nombre de 41 caracteres, nombre sin cambios y opción no válida

```
      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo: AMIGOS
MODIFICACIÓN: PRIMOS
OPCIÓN A-ACEPTAR C-CANCELAR: A
Grupo modificado correctamente.

      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo:
```

Figura 53. Cambio de nombre del grupo “AMIGOS” a “PRIMOS”

```
      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo: BARRIO
MODIFICACIÓN: COMPAS DEL COLEGIO
GRUPO YA EXISTE, EN CASO DE ACEPTAR LA OPERACIÓN LOS CONTACTOS SE LE AGREGARÁN
OPCIÓN A-ACEPTAR C-CANCELAR: C
Modificación cancelada

      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo:
```

Figura 54. Nombre de un grupo que ya existe y cancelación

```
      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo: BARRIO
MODIFICACIÓN: COMPAS DEL COLEGIO
GRUPO YA EXISTE, EN CASO DE ACEPTAR LA OPERACIÓN LOS CONTACTOS SE LE AGREGARÁN
OPCIÓN A-ACEPTAR C-CANCELAR: A
Grupo modificado correctamente.

      ADMINISTRAR GRUPOS DE CONTACTOS
      Modificar grupos

Nombre del grupo: C

DESPUÉS grupos: ['COMPAS DEL COLEGIO', 'PRIMOS']
DESPUÉS contactos_por_grupo: [[(88887777, 506), (88886666, 506), (88885555, 506)], []]
Contactos registrados: 3
```

Figura 55. Nombre de un grupo que ya existe y aceptación, y contenido final de las listas

Resultado: todas las pruebas coincidieron con lo esperado. Un grupo que no existe mostró "GRUPO NO ESTÁ REGISTRADO, NO SE PUEDE MODIFICAR" y se volvió a pedir el nombre. Un nuevo nombre de 41 caracteres mostró el mensaje de error de longitud y se volvió a pedir el dato. Con el dato vacío se conservó el nombre actual, sin mostrar el aviso de que el grupo ya existe, y una opción distinta de A o C mostró "ERROR: OPCIÓN NO VÁLIDA." y se volvió a preguntar; con A se mostró "Grupo modificado correctamente." y el nombre quedó igual. Al cambiar "AMIGOS" por "PRIMOS" solo se reemplazó el nombre, y el grupo conservó su posición en las listas. Al escribir el nombre de un grupo que ya existe ("BARRIO" por "COMPAS DEL COLEGIO") se mostró "GRUPO YA EXISTE, EN CASO DE ACEPTAR LA OPERACIÓN LOS CONTACTOS SE LE AGREGARÁN"; con C se mostró "Modificación cancelada" y nada cambió, y con A los contactos de "BARRIO" pasaron a "COMPAS DEL COLEGIO" sin repetir a Ana, que ya estaba en ese grupo, y el grupo "BARRIO" se eliminó. Al finalizar, las listas grupos y contactos_por_grupo quedaron con dos elementos cada una ("COMPAS DEL COLEGIO" con María, Ana y Luis, y "PRIMOS" sin contactos), lo que confirma la relación 1:1, y los tres contactos seguían registrados en la lista contactos.

Pruebas de funcionamiento de Eliminar grupos

Se probó la función eliminar_grupo() con tres grupos cargados temporalmente solo para las pruebas: "COMPAS DEL COLEGIO" (con dos contactos, María y Ana), "BARRIO" (con un contacto, Ana) y "AMIGOS" (sin contactos). En una primera ejecución se eliminó el grupo "BARRIO", que está en una posición intermedia, para comprobar que las listas grupos y contactos_por_grupo se mantienen emparejadas; para ello se mostró temporalmente su valor al finalizar el programa. En una segunda ejecución se eliminó el grupo "COMPAS DEL COLEGIO" y después se consultó el contacto Ana, que pertenecía a ese grupo. El código temporal era solo para las pruebas y no forma parte del programa.

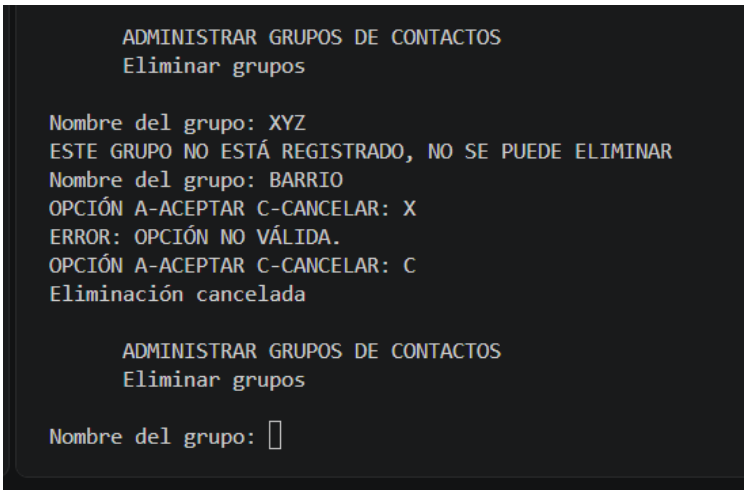


Figura 56. Grupo no registrado, opción no válida y cancelación al eliminar un grupo

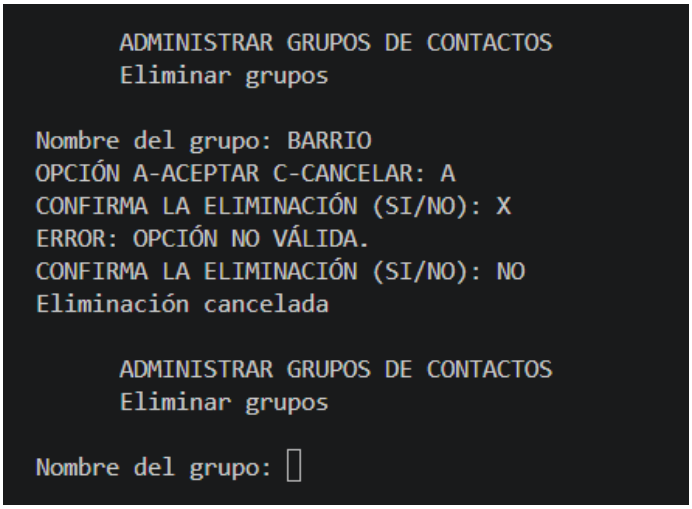


Figura 57. Confirmación no válida y cancelación con NO

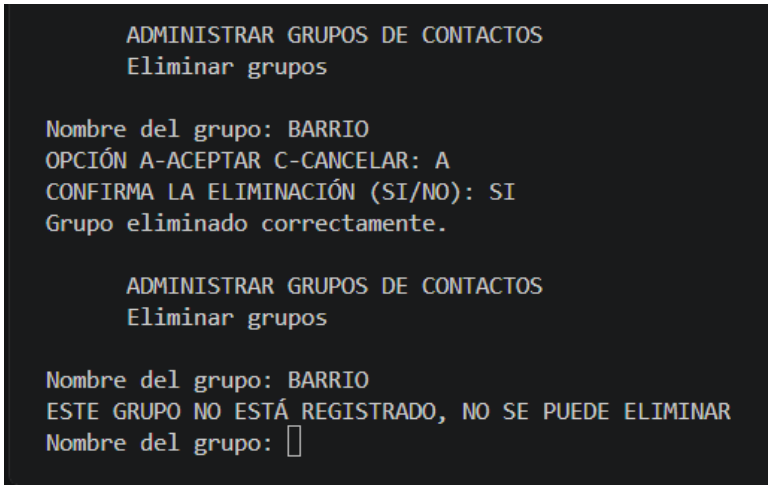


Figura 58. Grupo eliminado y rechazo al intentar eliminarlo de nuevo

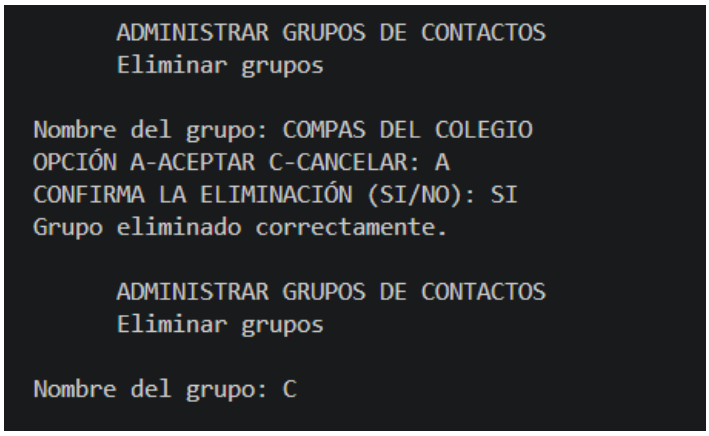


Figura 59. Contenido de las listas grupos y contactos_por_grupo después de eliminar un grupo

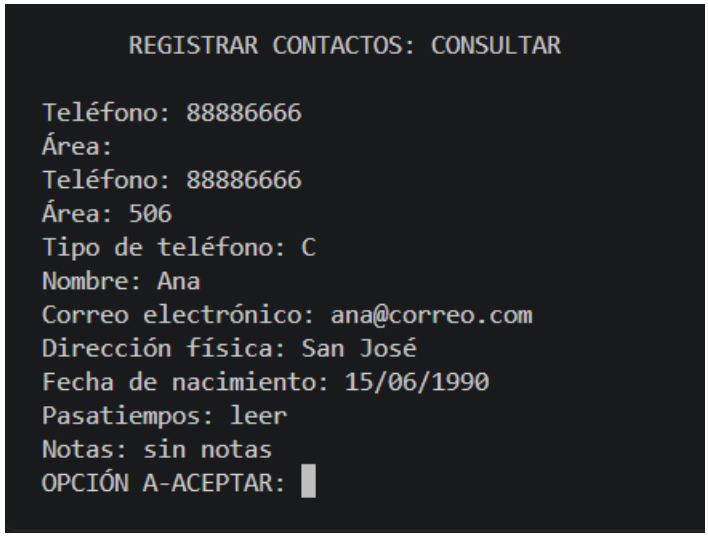


Figura 60. Eliminación del grupo “COMPAS DEL COLEGIO”

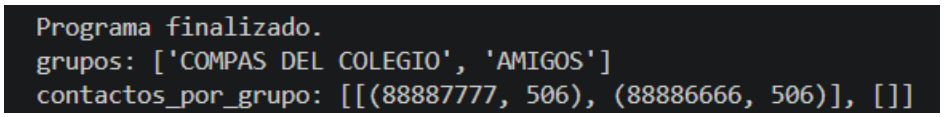


Figura 61. Contacto consultado después de eliminar un grupo al que pertenecía

Resultado: todas las pruebas coincidieron con lo esperado. Un grupo que no existe mostró “ESTE GRUPO NO ESTÁ REGISTRADO, NO SE PUEDE ELIMINAR” y se volvió a pedir el nombre. Una opción distinta de A o C, y una confirmación distinta de SI o NO, mostraron “ERROR: OPCIÓN NO VÁLIDA.” y se volvió a preguntar. Con C en la opción, y con NO en la confirmación, se mostró “Eliminación cancelada” y el grupo no se eliminó. Con SI se mostró “Grupo eliminado correctamente.”, y al intentar eliminar de nuevo el mismo grupo se mostró el mensaje de que no está registrado. Al finalizar, el grupo eliminado ya no aparecía

en ninguna de las dos listas, y los grupos restantes conservaron sus contactos en la posición correcta, lo que confirma la relación 1:1. Además, en una segunda ejecución se eliminó el grupo “COMPAS DEL COLEGIO” y el contacto Ana, que pertenecía a ese grupo, siguió registrado con todos sus datos: eliminar un grupo no elimina los contactos de la lista contactos.

Pruebas de funcionamiento de Eliminar contactos de los grupos

Se probó la función eliminar_contacto_de_grupo() con tres contactos (María, Ana y Luis) y tres grupos cargados temporalmente solo para las pruebas: “COMPAS DEL COLEGIO” (con María y Ana), “BARRIO” (con Ana) y “AMIGOS” (sin contactos). Luis está registrado en la lista de contactos, pero no pertenece a ningún grupo. Para comprobar el resultado, se mostró temporalmente el valor de las listas grupos y contactos_por_grupo al finalizar el programa. El código temporal era solo para las pruebas y no forma parte del programa.

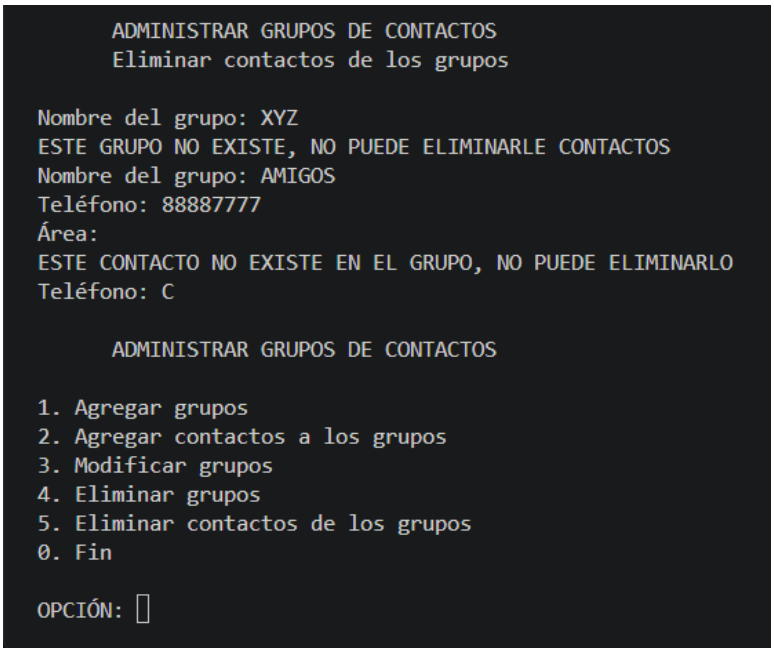


Figura 62. Grupo inexistente y contacto que no está en un grupo vacío

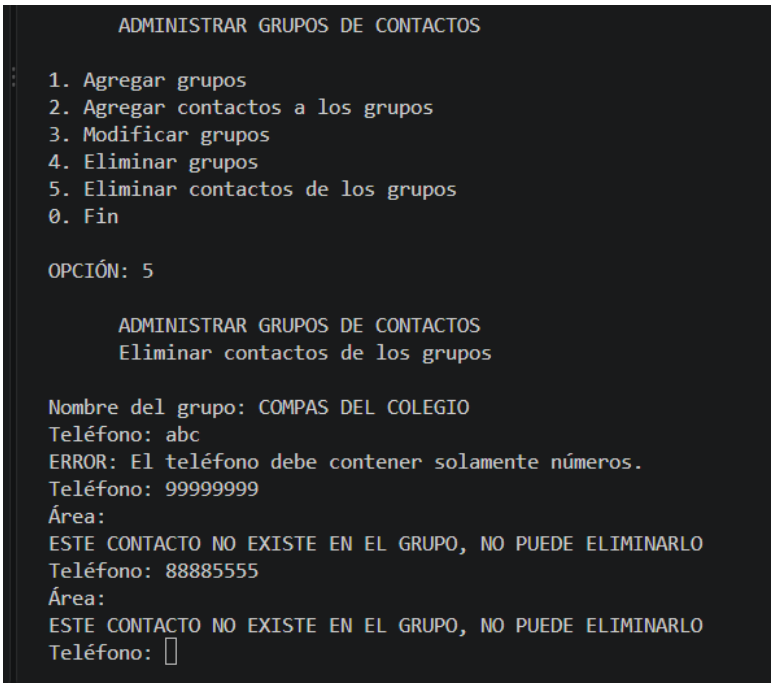


Figura 63. Teléfono no válido y contactos que no pertenecen al grupo

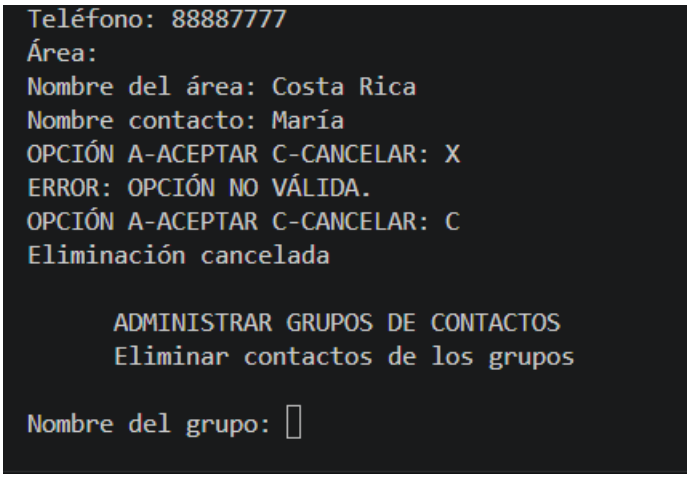


Figura 64. Datos del contacto, opción no válida y cancelación con C

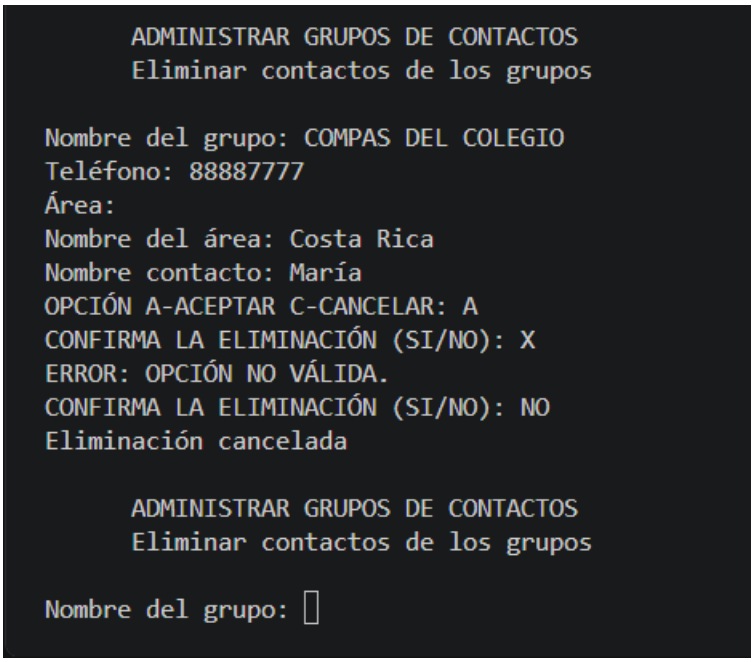


Figura 65. Confirmación no válida y cancelación con NO

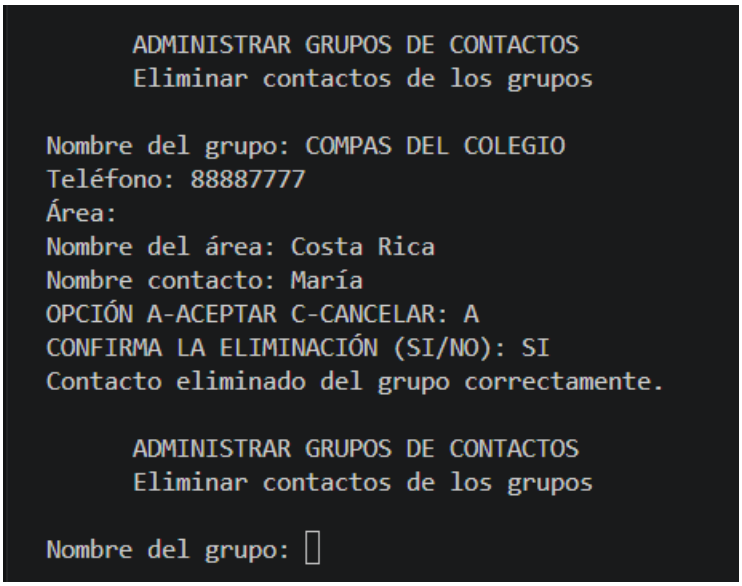


Figura 66. Contacto eliminado de un grupo

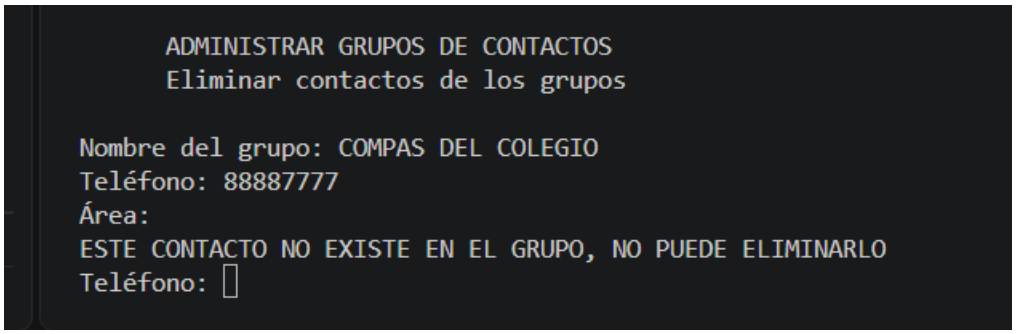


Figura 67. Comprobación de que el contacto ya no está en el grupo

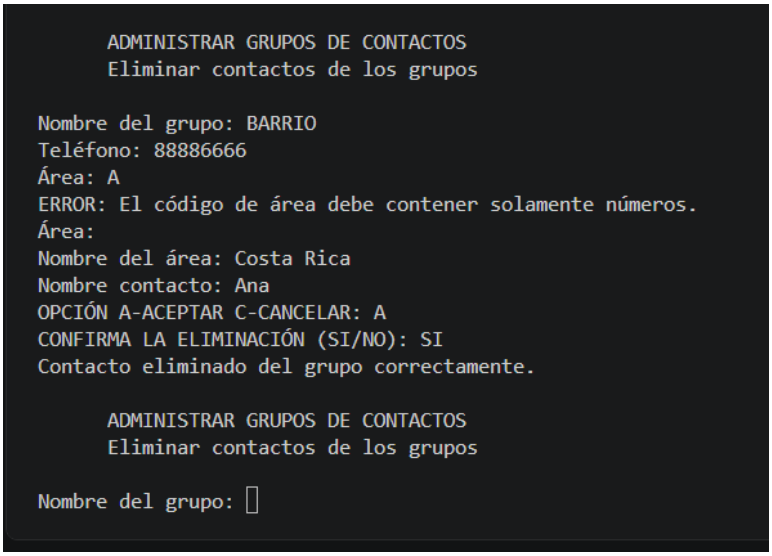


Figura 68. Área no válida y eliminación de un contacto de otro grupo

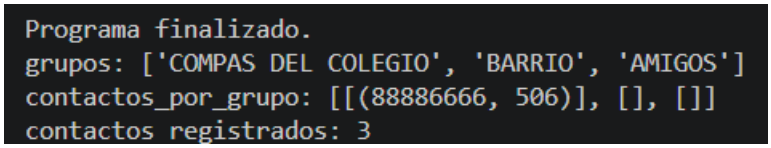


Figura 69. Contenido de las listas después de eliminar contactos de los grupos

Resultado: todas las pruebas coincidieron con lo esperado. Un grupo que no existe mostró "ESTE GRUPO NO EXISTE, NO PUEDE ELIMINARLE CONTACTOS", y un contacto que no pertenece al grupo (incluso uno que sí está registrado, como Luis, o el que se acababa de eliminar) mostró "ESTE CONTACTO NO EXISTE EN EL GRUPO, NO PUEDE ELIMINARLO"; en ambos casos se volvió a solicitar el dato. Un teléfono o un área no válidos mostraron el error correspondiente. Una opción distinta de A o C, y una confirmación distinta de SI o NO, mostraron "ERROR: OPCIÓN NO VÁLIDA.". Con C en la opción y con NO en la confirmación se mostró "Eliminación cancelada" y el contacto no se eliminó.

Con SI se mostró “Contacto eliminado del grupo correctamente.”. Al finalizar, María ya no estaba en “COMPAS DEL COLEGIO”, Ana seguía en ese grupo aunque se eliminó de “BARRIO”, y los tres contactos seguían registrados en la lista contactos: eliminar un contacto de un grupo no lo elimina de la lista general ni de otros grupos.

Pruebas de funcionamiento de la revisión de grupos al eliminar un

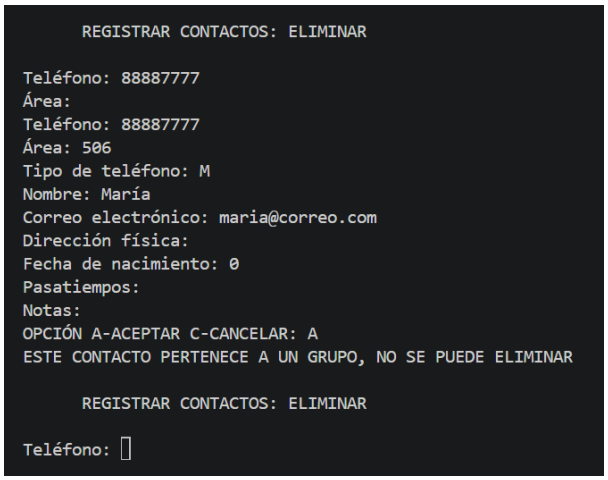


Figura 70. Rechazo al eliminar un contacto que pertenece a un grupo

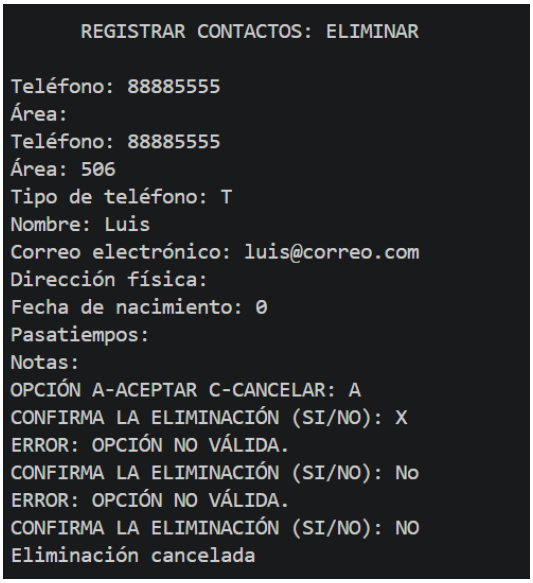


Figura 71. Confirmación no válida y cancelación al eliminar un contacto que no está en ningún grupo

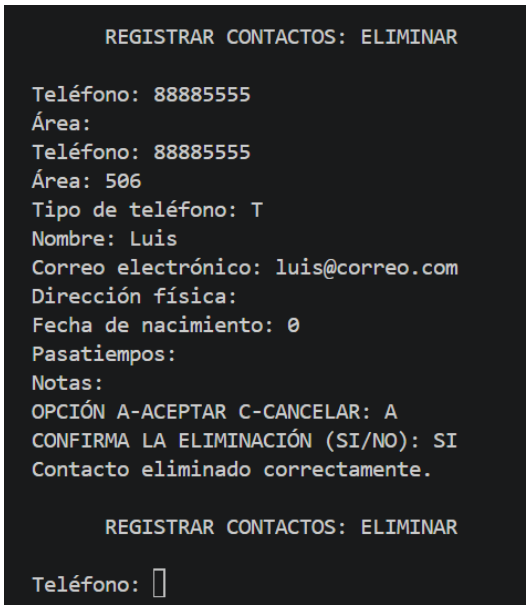


Figura 72. Eliminación de un contacto que no pertenece a ningún grupo

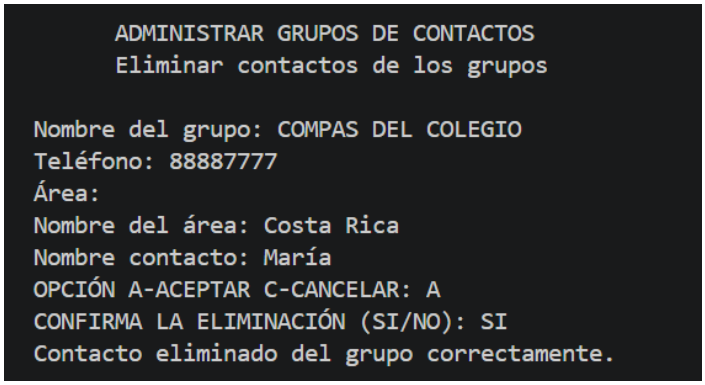


Figura 73. Contacto eliminado de un grupo con Eliminar contactos de los grupos

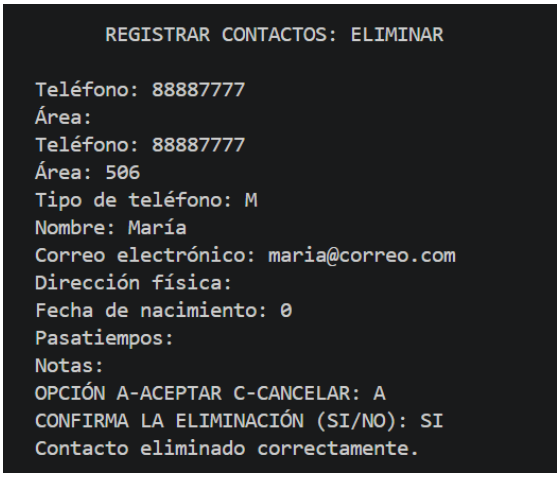


Figura 74. Eliminación del contacto después de quitarlo del grupo

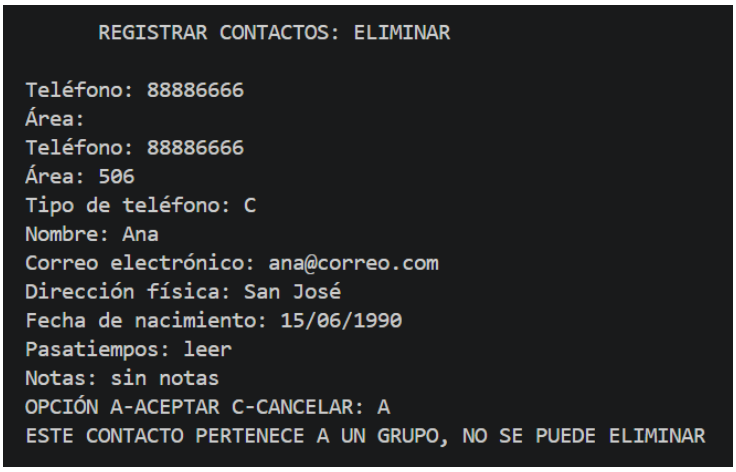


Figura 75. Rechazo al eliminar un contacto que pertenece a dos grupos

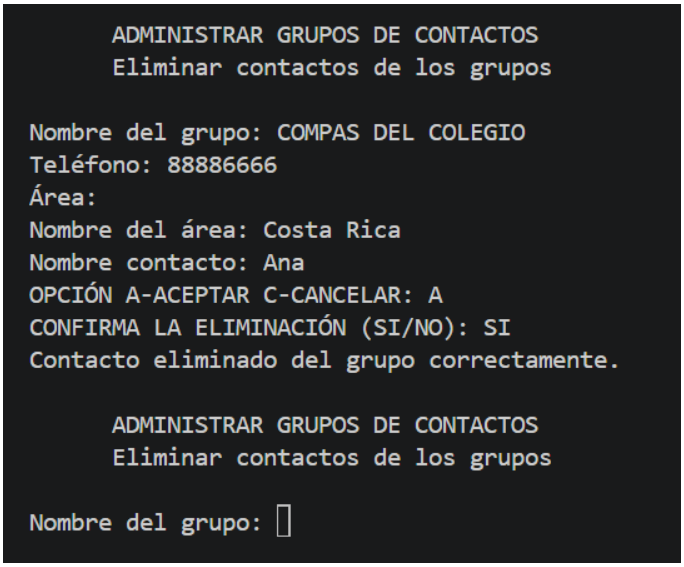


Figura 76. Contacto eliminado de uno de sus dos grupos

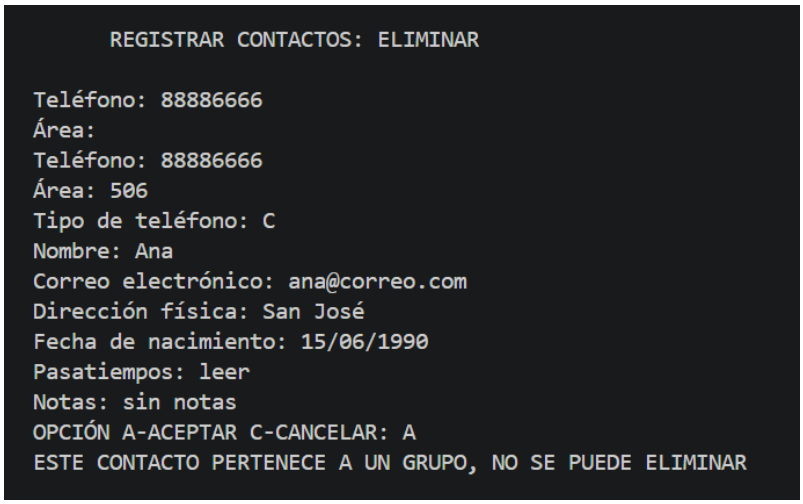


Figura 77. Rechazo al eliminar un contacto que todavía pertenece a un grupo



Figura 78. Contenido de las listas grupos, contactos_por_grupo y contactos al finalizar

Resultado: todas las pruebas coincidieron con lo esperado. Al intentar eliminar a María, que pertenecía a "COMPAS DEL COLEGIO", se mostró "ESTE CONTACTO PERTENECE A UN GRUPO, NO SE PUEDE ELIMINAR" sin pedir la confirmación. Luis, que no pertenece a ningún grupo, llegó a la confirmación: una confirmación distinta de SI o NO (por ejemplo "X" o "No") mostró "ERROR: OPCIÓN NO VÁLIDA.", con NO se mostró "Eliminación cancelada" y con SI se mostró "Contacto eliminado correctamente.". Después de quitar a María del grupo con la opción Eliminar contactos de los grupos, sí pudo eliminarse de la lista de contactos. Ana, que pertenecía a dos grupos, fue rechazada; incluso después de quitarla de "COMPAS DEL COLEGIO" siguió rechazándose porque aún pertenecía a "BARRIO". Al finalizar, solo quedaba Ana en la lista contactos, y las listas grupos y contactos_por_grupo conservaban la misma cantidad de elementos (tres grupos y tres listas), lo que confirma la relación 1:1.

Pruebas de funcionamiento de Lista de contactos

Se probó la función lista_contactos(), que utiliza las funciones auxiliares coincide_con_filtro(), contacto_cumple_grupo(), filtrar_contactos() y emitir_listado(). Se cargaron temporalmente solo para las pruebas cinco contactos (María, Ana, William, Alberto y Luis) y cuatro grupos (“COMPAS DEL COLEGIO”, “BARRIO”, “AMIGOS” y “FUTBOL”); Luis no pertenece a ningún grupo y William no tiene fecha de nacimiento (valor 0). Por ahora el listado se muestra en pantalla con el nombre, el teléfono y el área de cada contacto encontrado; la emisión del archivo PDF corresponde a otra funcionalidad y se documenta en su propia sección. El código temporal era solo para las pruebas y no forma parte del programa.

```
DATOS DE PRUEBA (nombre | teléfono | área | fecha | pasatiempos)
María Fernández Solís | 88887777 | 506 | 15/06/1990 | leer, bailar
Ana Mora | 88886666 | 506 | 20/03/2001 | Futbol
William Mata Rodríguez | 88885555 | 47 | 0 | videojuegos, futbol
Alberto Castillo Matamoros | 22223333 | 506 | 30/01/2001 | futbol, basquet, videojuegos
Luis Castillo | 88884444 | 502 | 15/08/0000 |
grupos: ['COMPAS DEL COLEGIO', 'BARRIO', 'AMIGOS', 'FUTBOL']
contactos_por_grupo: [[(88887777, 506), (88886666, 506), (22223333, 506)], [(88886666, 506)], [], [(22223333, 506), (88885555, 47)]]
```

Figura 79. Datos de prueba: contactos, grupos y contactos_por_grupo

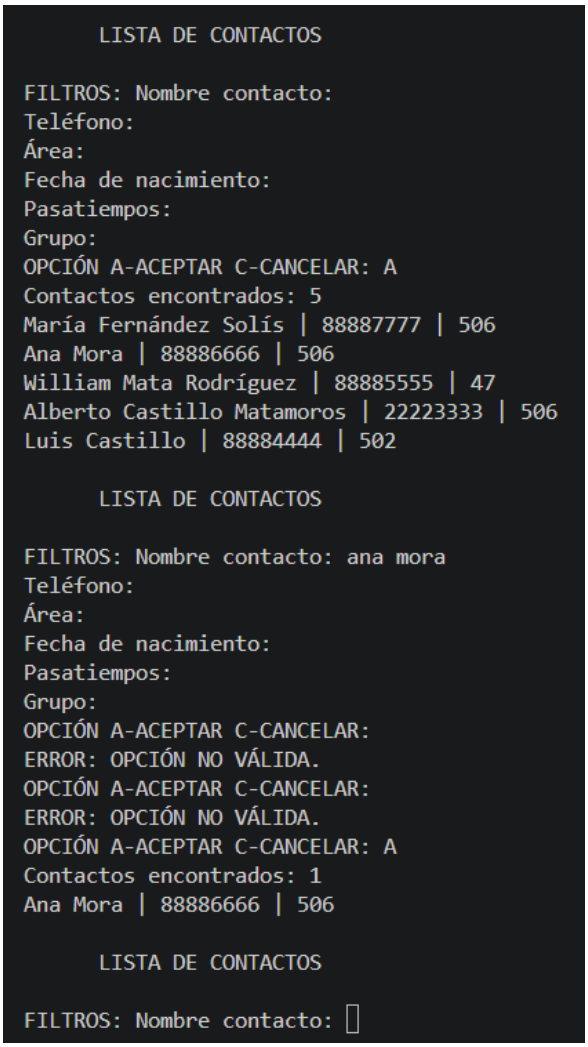


Figura 80. Sin filtros (todos los contactos) y nombre exacto sin distinguir mayúsculas

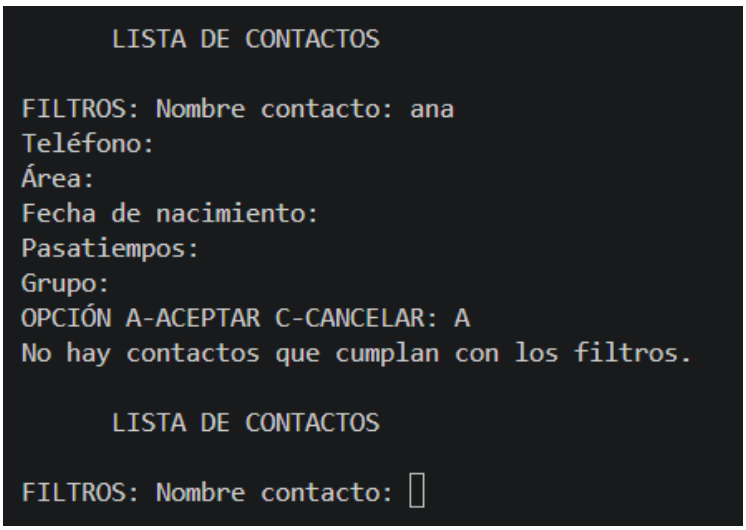


Figura 81. Nombre sin comodín que no es exactamente igual al dato

```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto: %castillo
Teléfono:
Área:
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 1
Luis Castillo | 88884444 | 502

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto: alberto castillo%
Teléfono:
Área:
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 1
Alberto Castillo Matamoros | 22223333 | 506

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto: %castillo%
Teléfono:
Área:
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 2
Alberto Castillo Matamoros | 22223333 | 506
Luis Castillo | 88884444 | 502

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

Figura 82. Comodín % al inicio, al final y en ambos lados del nombre

```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto: william%mata
Teléfono:
Área:
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
No hay contactos que cumplan con los filtros.

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto: %william %mata%
Teléfono:
Área:
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 1
William Mata Rodríguez | 88885555 | 47

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

Figura 83. Comodín % en el medio del nombre

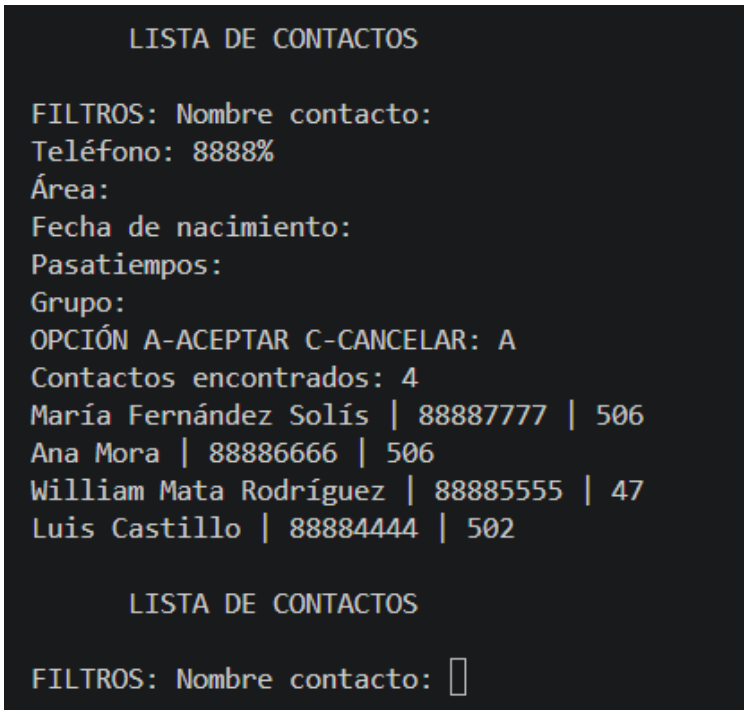
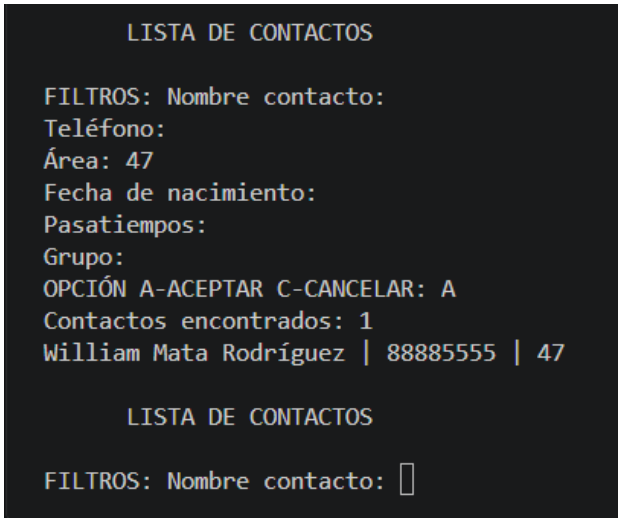


Figura 84. Comodín % en el teléfono



```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
Teléfono:
Área: 50%
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 4
María Fernández Solís | 88887777 | 506
Ana Mora | 88886666 | 506
Alberto Castillo Matamoros | 22223333 | 506
Luis Castillo | 88884444 | 502

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

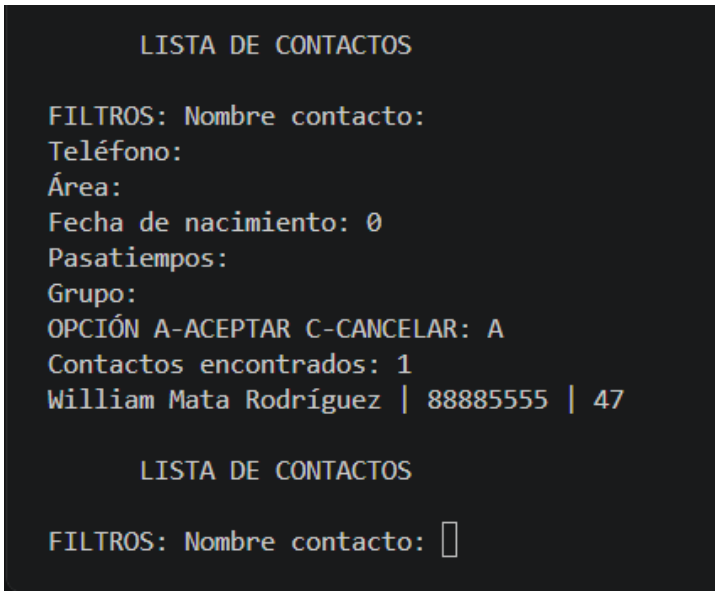
Figura 85. Filtro de área con valor exacto y con comodín

```
       LISTA DE CONTACTOS

FILTROS: Nombre contacto:
Teléfono:
Área:
Fecha de nacimiento: %1990
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 1
María Fernández Solís | 88887777 | 506

       LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```



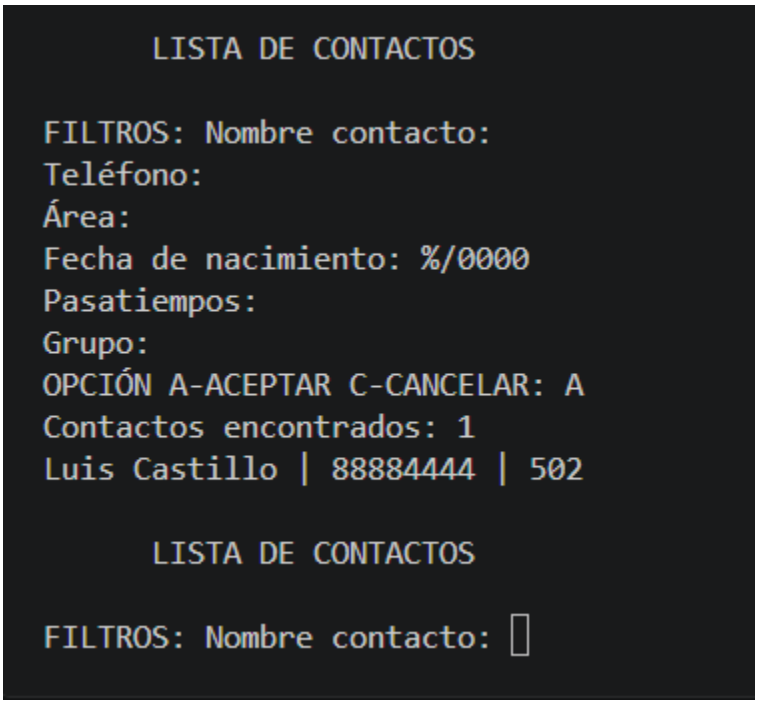


Figura 86. Filtro de fecha de nacimiento con comodín y contacto sin fecha

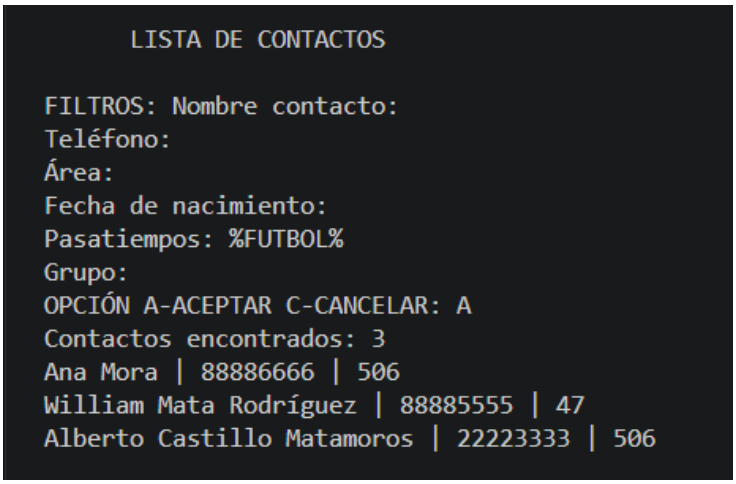
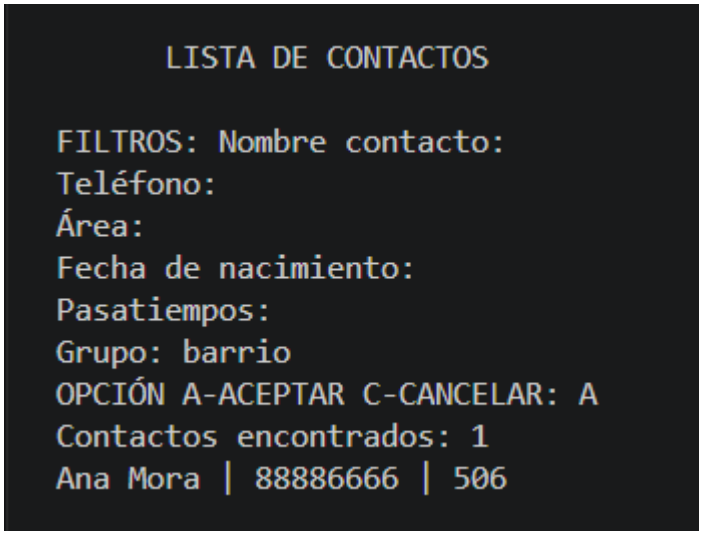
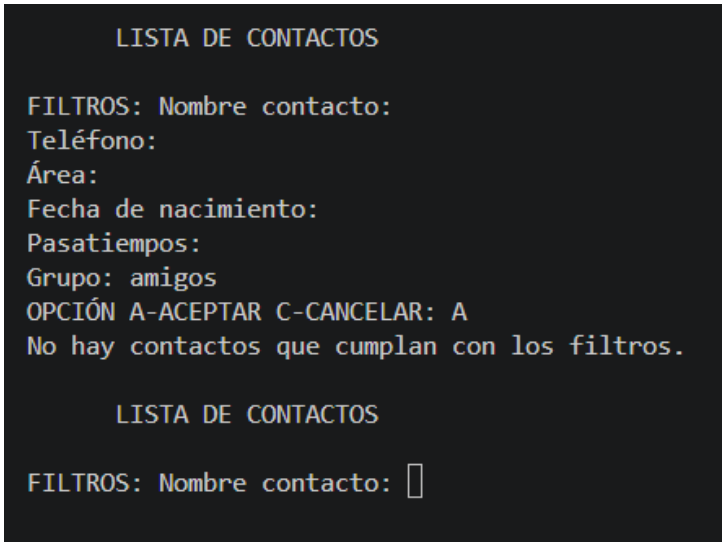


Figura 87. Filtro de pasatiempos sin distinguir mayúsculas





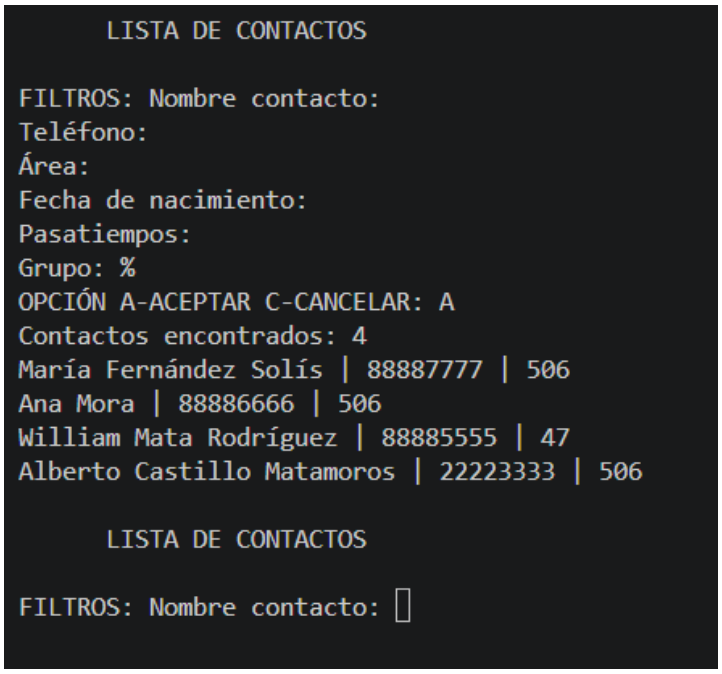
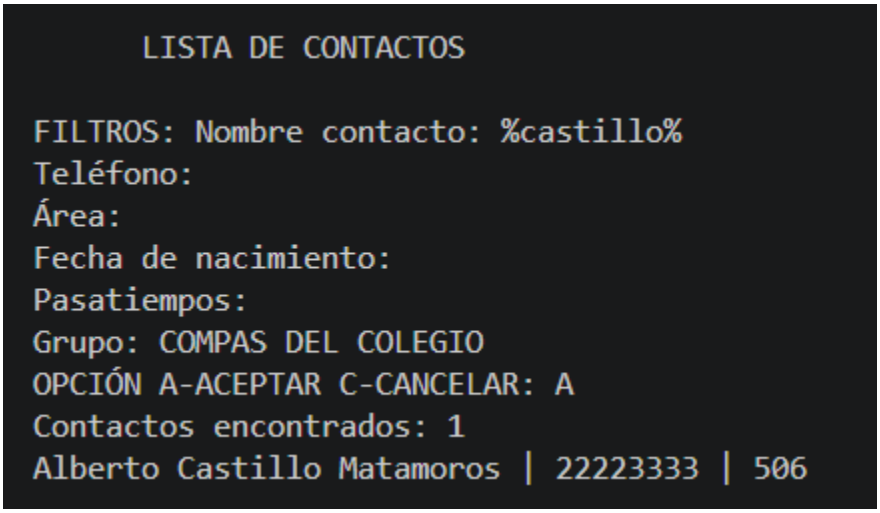


Figura 88. Filtro de grupo: nombre exacto, grupo vacío y comodín %



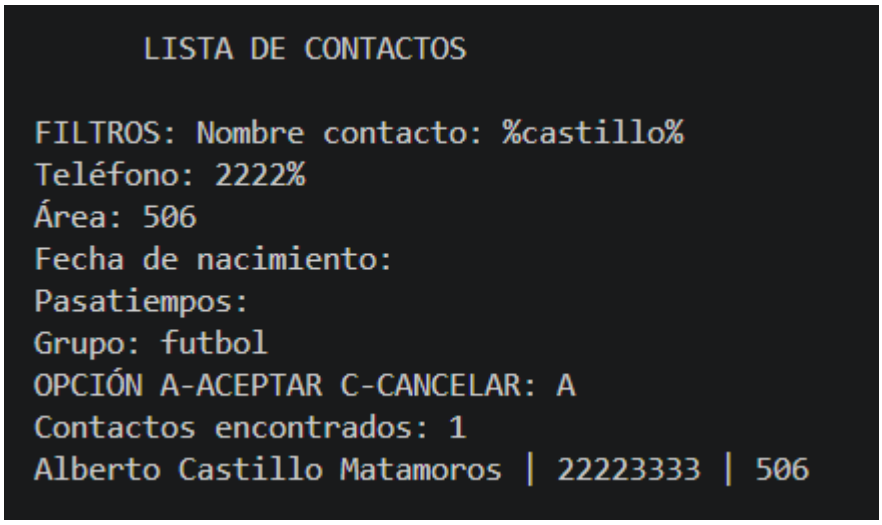


Figura 89. Varios filtros a la vez

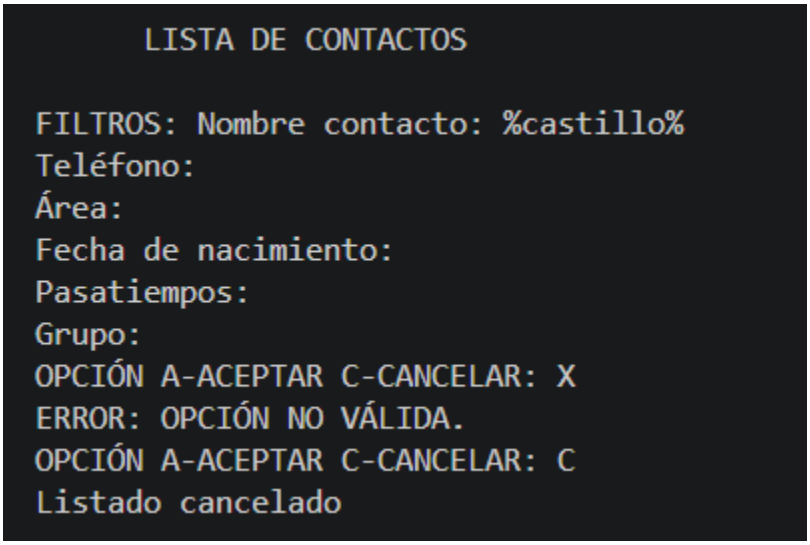


Figura 90. Opción no válida, cancelación y regreso al menú con C

Resultado: todas las pruebas coincidieron con lo esperado. Sin filtros se listaron los cinco contactos. Un nombre escrito en minúsculas (“ana mora”) encontró a “Ana Mora”, por lo que no se distingue entre mayúsculas y minúsculas, y “ana” sin comodín no encontró nada porque, sin %, el dato debe ser exactamente igual. El comodín % funcionó al inicio (“%castillo”: Luis Castillo), al final (“alberto castillo%”: Alberto), en ambos lados (“%castillo%”: Alberto y Luis) y en el medio (“%william %mata%”: William), mientras que “william%mata” no encontró a nadie porque el nombre no termina en “mata”. En el teléfono, “8888%” encontró cuatro contactos; en el área, “47” encontró uno y “50%” cuatro; en la fecha, “%1990” encontró uno, “0” al contacto sin fecha y “%/0000” al de año desconocido;

en los pasatiempos, “%FUTBOL%” encontró tres contactos. En el grupo, “barrio” encontró a Ana, “amigos” no encontró nada porque el grupo está vacío y “%” encontró a los cuatro contactos que pertenecen a algún grupo, sin incluir a Luis. Con varios filtros a la vez solo se listaron los contactos que cumplían todos. Una opción distinta de A o C mostró “ERROR: OPCIÓN NO VÁLIDA.” y se volvió a preguntar; con C se mostró “Listado cancelado” y la ventana volvió a aparecer con los filtros vacíos, y con C en el nombre se regresó al menú.

Pruebas de funcionamiento de la emisión del listado en PDF

Se probó la función emitir_listado(), que utiliza las funciones auxiliares nombre_tipo() y texto_pdf(). Se cargaron temporalmente, solo para las pruebas, seis contactos (María, Ana, William, Alberto, Luis y Ñandú) y dos grupos (“COMPAS DEL COLEGIO” y “BARRIO”). Los datos incluyen tildes, ñ, los signos ¿ y ¡, un emoji, un contacto sin fecha de nacimiento (0) y uno con el año desconocido (0000). El código temporal era solo para las pruebas y no forma parte del programa.

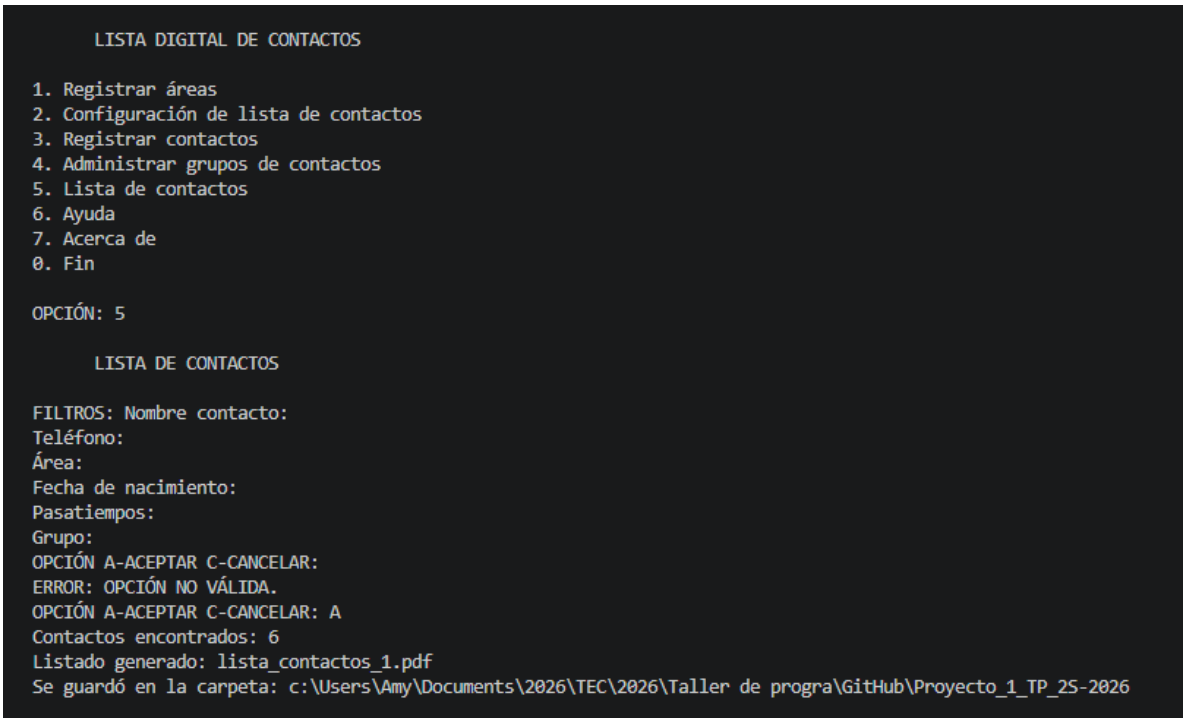


Figura 91. Emisión del listado sin filtros

LISTA DIGITAL DE CONTACTOS - LISTA DE CONTACTOS

Contactos encontrados: 6

Nombre	Teléfono y área	Tipo	Correo electrónico	Dirección física	Fecha de nacimiento	Pasatiempos	Notas
María Fernández	88887777 (506)	Móvil	maria@correo.com	San José, del parque 200 m al norte, casa esquinera	15/06/1990	leer, música	Compañera del colegio
Ana Mora	88886666 (506)	Casa	ana@correo.com	Heredia	20/03/1995	futbol, cine	¿Llamar mañana? ¡Sí!
William Mata	88885555 (47)	Trabajo	william@correo.no	Oslo	0	futbol	Nota con emoji ?
Alberto Castillo Matamoros	22223333 (506)	Otro	a_castillo@yahoo.com	Cartago centro	30/01/2001	futbol, basquet, videojuegos	está de vacaciones
Luis Castillo	88884444 (502)	Móvil	luis@correo.gt		15/08/0000		
Ñandú Núñez Ávila	88883333 (506)	Móvil	nandu@correo.com	Nicoya, Guanacaste	01/12/1988	pesca	Prueba de ñ y tildes

Figura 92. PDF con los seis contactos

```
      LISTA DE CONTACTOS

FILTROS: Nombre contacto: %castillo%
Teléfono:
Área:
Fecha de nacimiento:
Pasatiempos:
Grupo:
OPCIÓN A-ACEPTAR C-CANCELAR: A
Contactos encontrados: 2
Listado generado: lista_contactos_2.pdf
Se guardó en la carpeta: c:\Users\Amy\Documents\2026\TEC\2026\Taller de progra\GitHub\Proyecto_1_TP_2S-2026

      LISTA DE CONTACTOS

FILTROS: Nombre contacto:
```

Figura 93. Listado con el filtro %castillo%

```
      LISTA DE CONTACTOS

 FILTROS: Nombre contacto:
 Teléfono:
 Área:
 Fecha de nacimiento:
 Pasatiempos:
 Grupo: barrio
 OPCIÓN A-ACEPTAR C-CANCELAR: A
 Contactos encontrados: 1
 Listado generado: lista_contactos_3.pdf
 Se guardó en la carpeta: c:\Users\Amy\Documents\2026\TEC\2026\Taller de progra\GitHub\Proyecto_1_TP_2S-2026

      LISTA DE CONTACTOS

 FILTROS: Nombre contacto:
```

Figura 94. Listado con el filtro de grupo barrio

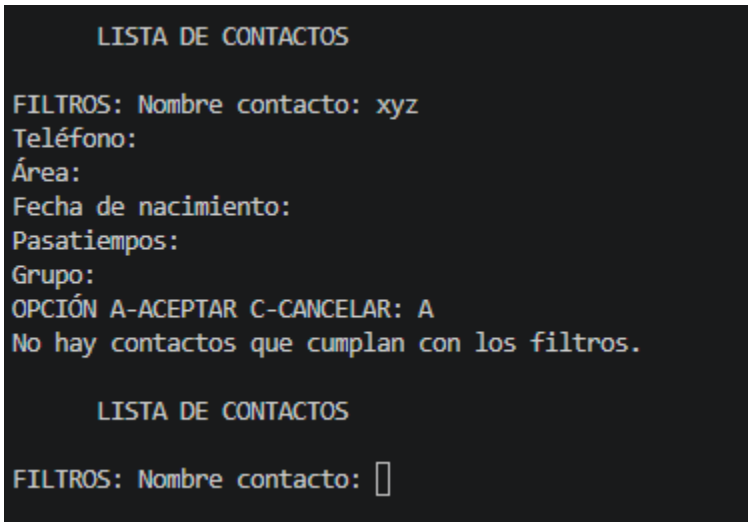


Figura 95. Filtros sin resultados

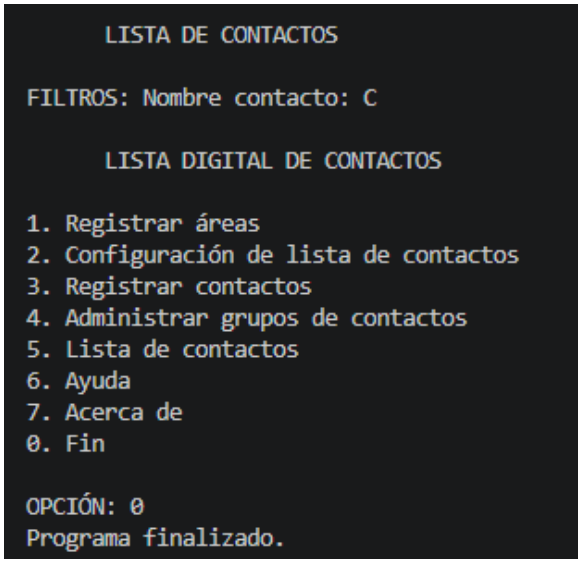


Figura 96. Regreso al menú principal

Resultado: todas las pruebas coincidieron con lo esperado. Sin filtros se generó lista_contactos_1.pdf con los seis contactos en una tabla de ocho columnas en hoja horizontal; las tildes, la ñ y los signos ¿ y ¡ se mostraron correctamente, el emoji se reemplazó por "?", el contacto sin fecha mostró 0, el año desconocido se mostró como 15/08/0000 y el tipo de teléfono apareció con su descripción. Con el filtro "%castillo%" se generó lista_contactos_2.pdf solo con Alberto y Luis, y con el filtro de grupo "barrio" se generó lista_contactos_3.pdf solo con Ana; cada listado se guardó con un número nuevo sin

reemplazar el anterior. Con un filtro sin resultados (“xyz”) se mostró “No hay contactos que cumplan con los filtros.” y no se creó ningún archivo. Una opción vacía mostró “ERROR: OPCIÓN NO VÁLIDA.”, y al escribir C en el primer filtro se regresó al menú principal.

Pruebas de funcionamiento de Ayuda

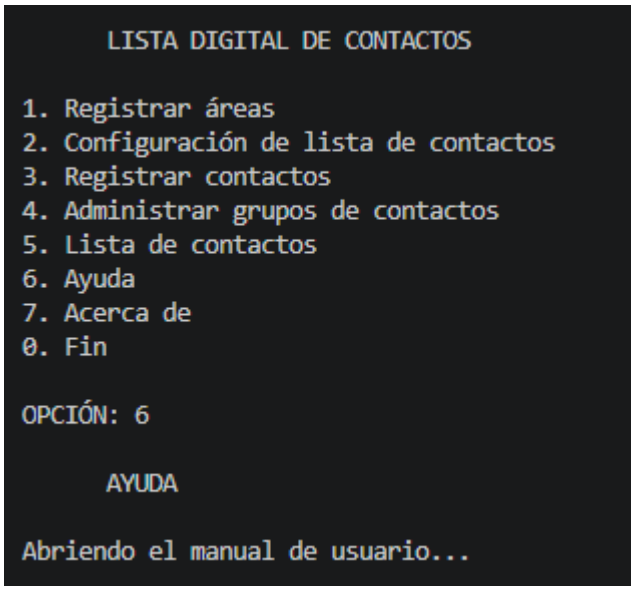


Figura 97. Apertura del manual de usuario con la opción Ayuda

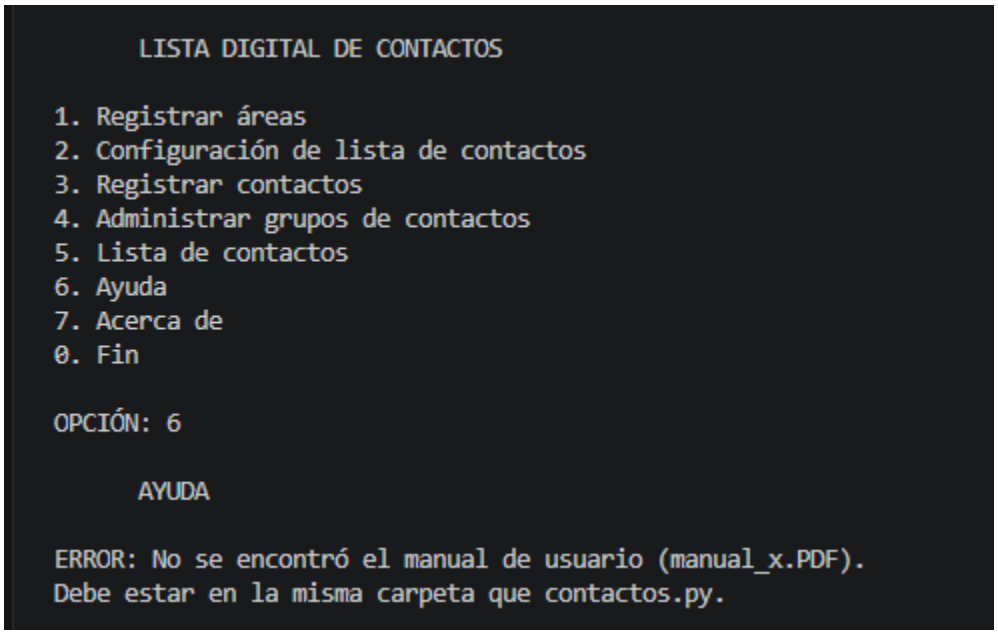


Figura 98. Mensaje cuando no se encuentra el manual

Resultado: al seleccionar la opción 6 se mostró “Abriendo el manual de usuario...” y el manual se abrió en el visor de PDF del computador. Cuando el archivo no se encontró, se mostró “ERROR: No se encontró el manual de usuario (manual_x.PDF).” y “Debe estar en la misma carpeta que contactos.py.”, y el programa regresó al menú sin detenerse. Después de la prueba se restableció el nombre correcto del archivo.

Funcionalidades y Partes Adicionales Desarrolladas

Se agregó una validación adicional que comprueba si existen áreas registradas antes de permitir ingresar a Configuración. Si no hay áreas, el programa muestra un mensaje y regresa al menú principal.

También se utilizó .strip() en las entradas de los códigos de área para eliminar espacios accidentales al inicio o al final, sin permitir espacios entre los números.

Además, se mejoró la presentación de Configuración agregando separaciones entre los mensajes para facilitar la lectura de la información en consola.

En Agregar contactos a los grupos se agregó una validación que impide agregar un contacto que ya pertenece al grupo, mostrando el mensaje “ESTE CONTACTO YA ESTÁ EN EL GRUPO, NO SE PUEDE AGREGAR”, porque el enunciado indica que en los grupos no hay contactos repetidos. También se permite regresar al menú con C en el campo del teléfono, para evitar que el usuario quede atrapado en la ventana cuando no tiene un contacto que agregar. Un mismo contacto sí puede pertenecer a varios grupos.

En Lista de contactos, el enunciado indica que con CANCELAR se vuelve a mostrar la ventana con los filtros vacíos, pero no indica cómo regresar al menú principal; se usó la letra “C” en el primer filtro (Nombre contacto), igual que en las demás ventanas del programa. También se redactaron los mensajes “No hay contactos que cumplan con los filtros.” y “Listado cancelado”, que no aparecen en el enunciado. En Configuración de la lista de contactos, si no hay áreas registradas se muestra el mensaje “No hay áreas registradas. Primero debe registrar un área.” y se regresa al menú, para evitar un ciclo infinito pidiendo un código de área.

Al eliminar un contacto que pertenece a algún grupo, el enunciado deja libre la decisión. Se optó por rechazar la eliminación y mostrar el mensaje “ESTE CONTACTO

PERTENECE A UN GRUPO, NO SE PUEDE ELIMINAR", que no aparece en el enunciado y fue redactado para este proyecto. También fue redactado para el proyecto el mensaje "Contacto eliminado del grupo correctamente.", que se muestra al quitar un contacto de un grupo.

En la emisión del listado en PDF se agregaron detalles que el enunciado no indica: el programa comprueba que la biblioteca fpdf2 esté instalada y, si no lo está, muestra el mensaje "ERROR: Falta instalar la biblioteca fpdf2. Escriba en la terminal: python -m pip install fpdf2" en lugar de detenerse con un error. Cada listado se guarda con un número (lista_contactos_1.pdf, lista_contactos_2.pdf, …) para no reemplazar uno anterior, y al terminar se muestran el nombre del archivo y la carpeta donde se guardó. El tipo de teléfono se muestra con su descripción (por ejemplo, Móvil en lugar de M), y los caracteres que la fuente del PDF no puede mostrar, como los emojis, se reemplazan por "?". Si ningún contacto cumple los filtros, no se crea el archivo y se muestra "No hay contactos que cumplan con los filtros.".

En Ayuda, si el manual de usuario no está en la carpeta del programa, se muestra "ERROR: No se encontró el manual de usuario (…)" y "Debe estar en la misma carpeta que contactos.py.", y el programa regresa al menú.

Referencias

Anthropic. (2026). *Claude* [Modelo de lenguaje grande]. https://claude.ai

Bucher, B., y van Rossum, G. (2020). *PEP 634 – Structural pattern matching: Specification*. Python Enhancement Proposals. https://peps.python.org/pep-0634/

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book

GitHub. (s. f.). *GitHub Docs*. https://docs.github.com

Goodger, D., y van Rossum, G. (2001). *PEP 257 – Docstring conventions*. Python Enhancement Proposals. https://peps.python.org/pep-0257/

Microsoft. (s. f.-a). *Debugging*. Visual Studio Code Documentation. https://code.visualstudio.com/docs/editor/debugging

Microsoft. (s. f.-b). *Python debugging in VS Code*. Visual Studio Code Documentation. https://code.visualstudio.com/docs/python/debugging

OpenAI. (2026). *ChatGPT* [Modelo de lenguaje grande]. https://chatgpt.com

py-pdf. (s. f.). *fpdf2 documentation*. https://py-pdf.github.io/fpdf2/

Python Software Foundation. (s. f.-a). *datetime — Basic date and time types*. Python 3 documentation. https://docs.python.org/3/library/datetime.html

Python Software Foundation. (s. f.-b). *Errors and exceptions*. Python 3 documentation. https://docs.python.org/3/tutorial/errors.html

Python Software Foundation. (s. f.-c). *More control flow tools: match statements*. Python 3 documentation. https://docs.python.org/3/tutorial/controlflow.html#match-statements

Python Software Foundation. (s. f.-d). *pdb — The Python debugger*. Python 3 documentation. https://docs.python.org/3/library/pdb.html

Python Software Foundation. (s. f.-e). *re — Regular expression operations*. Python 3 documentation. https://docs.python.org/3/library/re.html

Python Software Foundation. (s. f.-f). *Regular expression HOWTO*. Python 3 documentation. https://docs.python.org/3/howto/regex.html

ReportLab. (s. f.). *ReportLab PDF library user guide*. https://www.reportlab.com/docs/reportlab-userguide.pdf

van Rossum, G., Warsaw, B., y Coghlan, A. (2001). *PEP 8 – Style guide for Python code*. Python Enhancement Proposals. https://peps.python.org/pep-0008/